

AĞAÇLAR

Dr. Öğr. Üyesi Fatma AKALIN

Ağaç veri yapıları, düğümler (node) ve bu düğümler arasındaki ebeveyn-çocuk (parent-child) ilişkileri üzerinden tanımlanan, hiyerarşik organizasyona sahip soyut veri modelleridir. Bu yapılar, doğrusal olmayan veri organizasyonunu mümkün kılarak, özellikle karmaşık ilişkilerin modellenmesinde ve büyük veri kümelerinin etkin bir biçimde yönetilmesinde önemli bir rol üstlenmektedir. Her bir ağaç yapısı, kök (root) düğümden başlayarak alt düğümlere doğru dallanan bir yapı sergiler ve bu yönüyle doğal dünyadaki ağaç formuna benzer bir soyutlama sunar.

Bilgisayar bilimleri ve yazılım mühendisliği alanında ağaç veri yapıları, birçok temel problemin çözümünde yaygın olarak kullanılmaktadır. Özellikle veritabanı sistemlerinde indeksleme ve hızlı arama işlemleri için B-ağaçları ve türevleri tercih edilirken, işletim sistemlerinde dosya ve izin organizasyonu ağaç yapıları aracılığıyla gerçekleştirilmektedir. Bunun yanı sıra, yapay zekâ ve oyun geliştirme alanlarında karar ağaçları, olası durumların modellenmesi ve en uygun hamlenin belirlenmesi süreçlerinde etkin bir araç olarak kullanılmaktadır. Aynı şekilde, veri sıkıştırma tekniklerinde (örneğin Huffman kodlama) ve çeşitli arama algoritmalarında (örneğin ikili arama ağaçları) ağaç yapılarının sağladığı hiyerarşik düzen ve erişim verimliliği büyük avantaj sağlamaktadır.

Sonuç olarak, ağaç veri yapıları, hem teorik hem de pratik bağlamda geniş bir kullanım alanına sahip olup, veri organizasyonu, erişim hızı ve algoritmik verimlilik açısından bilgisayar bilimlerinin temel yapı taşlarından biri olarak kabul edilmektedir.

Birçok uygulama probleminde, veriler arasındaki ilişkilerin doğası gereği hiyerarşik bir yapı sergilemesi, ağaç veri modelini son derece uygun ve etkili bir çözüm aracı haline getirmektedir. Özellikle karar süreçlerinin modellenmesi, veri organizasyonu ve hızlı erişim gerektiren durumlarda ağaç yapıları, sundukları esneklik ve sistematik düzen sayesinde ön plana çıkmaktadır. Bu bağlamda, problemin yapısına uygun ağaç türünün seçilmesi, çözümün doğruluğu ve performansı açısından kritik bir öneme sahiptir.

Literatürde farklı ihtiyaçlara yönelik olarak geliştirilmiş çeşitli ağaç türleri bulunmaktadır. İkili ağaçlar (binary trees), her düğümün en fazla iki çocuğa sahip olduğu yapılarıyla özellikle arama ve sıralama algoritmalarında yaygın olarak kullanılmaktadır. Sözlük ağaçları (trie), karakter tabanlı veri kümelerinde (örneğin metin işleme ve otomatik tamamlama sistemlerinde) etkin bir çözüm sunarken; kodlama ağaçları, veri sıkıştırma süreçlerinde (örneğin Huffman algoritması) optimal kod üretimi için tercih edilmektedir. Aile işaretçisi (parent pointer) ağaçları ise her düğümün ebeveynine doğrudan erişimini mümkün kılarak, yukarı yönlü gezinme gerektiren algoritmalarda avantaj sağlamaktadır.

Sonuç olarak, farklı ağaç türleri, problem alanına özgü gereksinimleri karşılayacak şekilde özelleştirilmiş olup, her biri belirli uygulama senaryolarında en uygun ve verimli çözümü sunmayı amaçlamaktadır. Bu çeşitlilik, ağaç veri yapılarının bilgisayar bilimlerindeki merkezi konumunu ve önemini pekiştirmektedir.

Ağaç veri modeli birçok bilişim uygulaması için tartışılmaz çözüm modelidir. Örneğin N elemanlı bir küme içerisinde bilgi arama işlevini gerçekleştireceğim.

Bu durum,

Bağlantılı liste veri modelinde $O(n)$

Ağaç veri modelinde $O(\log_2^N)$ karmaşıklığında yapılmaktadır.

Bu durum da n'nin büyük değerleri için çok büyük zaman kazancı sağlanır.

Ağaçlar Üzerine Temel Kavramlar

Ağaç bir **kök işaretçisi**, **sonlu sayıda düğümleri** ve **onları birbirine bağlayan dalları** olan bir veri modelidir; aynı aile soy ağacında olduğu için **hiyerarşik** bir yapısı vardır ve orada geçen birçok kavram buradaki ağaç veri modelinde de tanımlıdır.

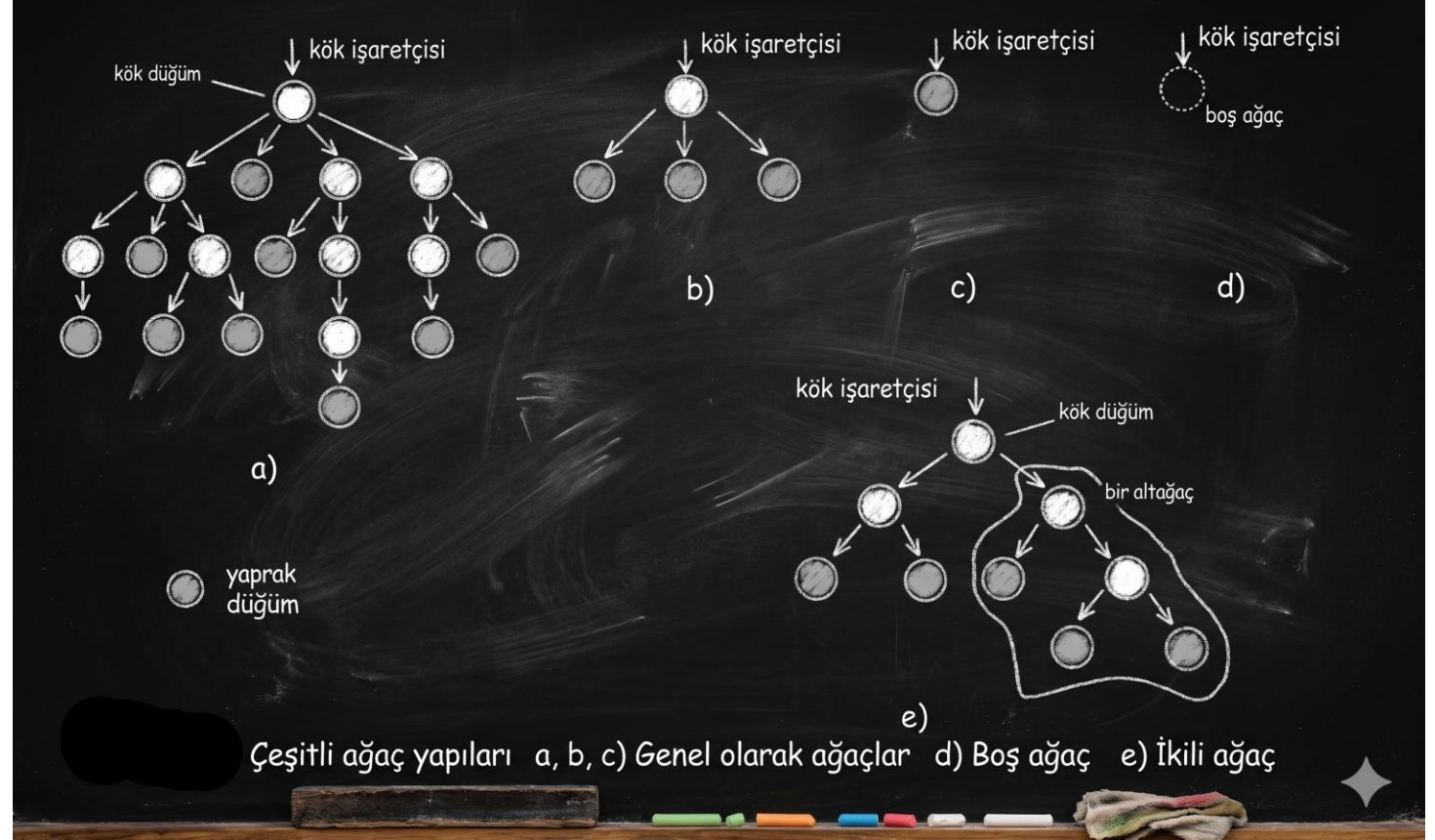
Genel olarak, **veri** ağacın düğümlerinde tutulur; **dallarda** ise bağlantı(geçiş) koşulları vardır. Ağaç veri yapısında her düğüm (node) için bir veri yapısı tanımlanır. Eğer ağaçta farklı türde düğümler varsa, her düğüm türü için ayrı bir veri yapısı oluşturulur. Normal düğüm → sadece veri + çocuklar

Özel düğüm → veri + çocuklar + ekstra bilgi

İkili ağaç kodlama ağacı sözlük ağacı kümeleme ağacı gibi çeşitli ağaç şekilleri vardır. Üstelik uygulamaya yönelik özel ağaç şekilleri de çıkarılabilir.

Ağaç Üzerine Bazı Tanımlar

Ağaç veri modeli ile çalışılırken bazı **tanımlar** ve **kavramlarla** karşılaşılır; bunların yalın olarak bilinmesi ağaçlar üzerinde çeşitli işlemler yapan algoritmaların incelenmesinde kolaylık sağlar.

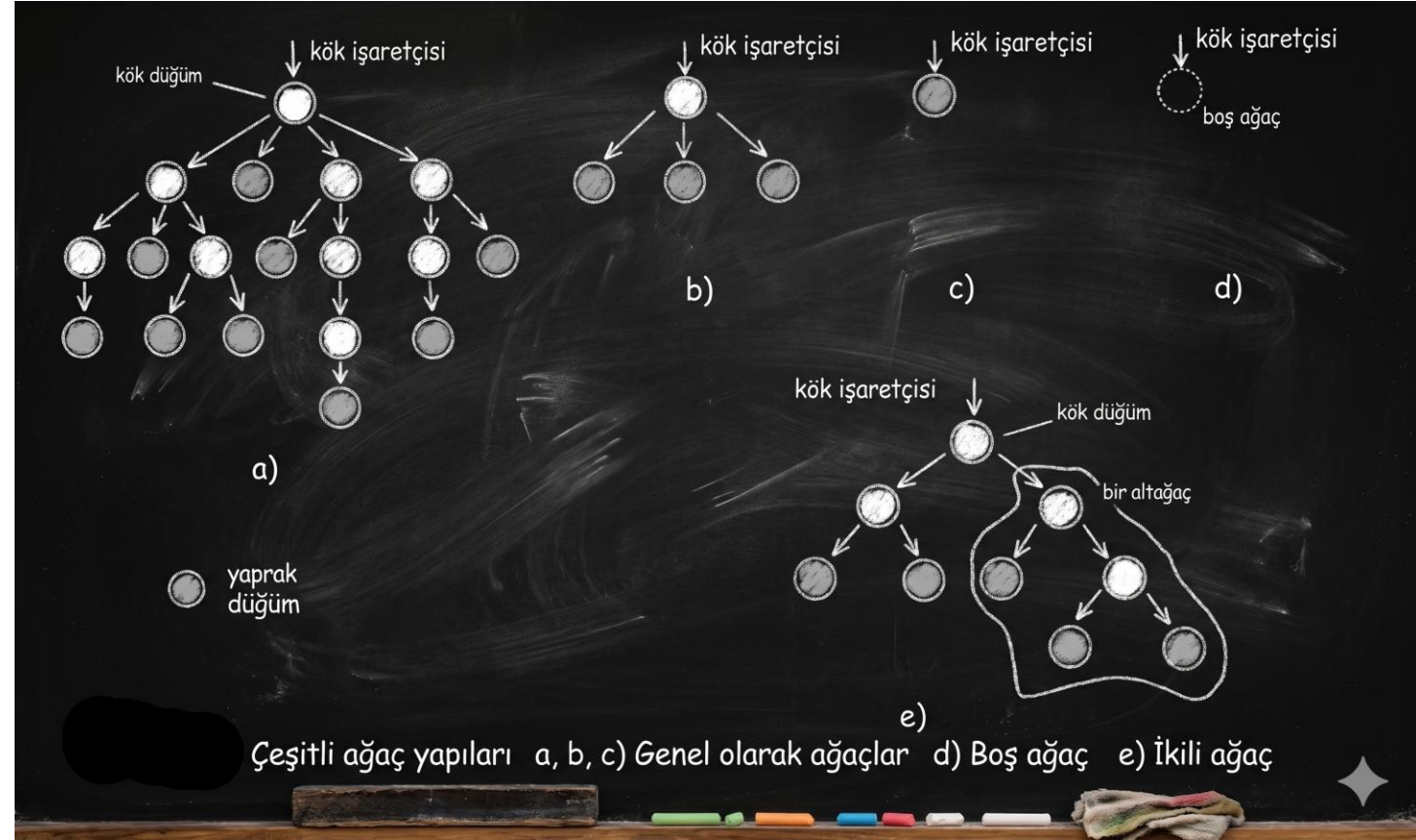


Çocuk, Oğul veya Evlat / Child

Bir **düğüm**e doğrudan bağlı olan düğümlere o düğümün **çocukları** denilir. Bir düğümüne bağlı alt ağaçların kökleri o düğümün çocuklarıdır

Yandaki şekilde verilen (a için) ağacın kök düğüm derecesi 4, (b için) ağacın kök düğüm derecesi 3.

Yaprak düğümler çocuksuz düğümlerdir.

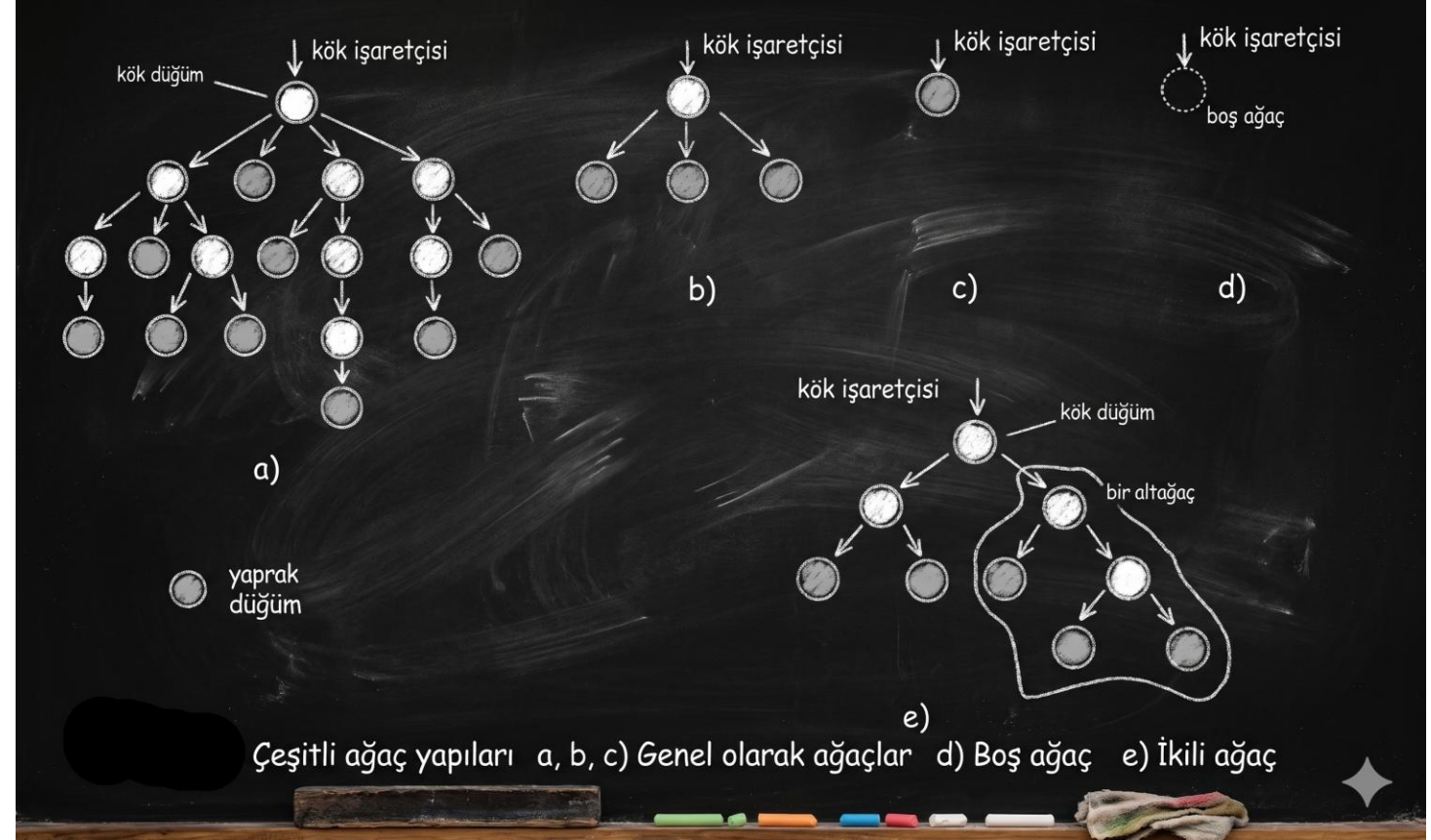


Derece (Degree)

Bir düğümden **alt hiyerarşiye**
yapılan bağlantıların sayısıdır;
yani, **çocuk** veya **alt ağaç**
sayısıdır. Yandaki şekil
kapsamında

a'daki ağacın kök düğüm
derecesi 4,

b'nin 3, c'nin 0 ve e'nin 2'dir.
Yaprak düğümlerin derecesi 0
olur.



Kardeş Düğüm(Sibling): Aynı düğüme bağlı olan düğümlere kardeş düğüm veya kısacası kardeş denilir. Bir düğümün kardeş sayısı bağlı olduğu düğümün derecesine eşittir.

Aile(Parent): Düğümlerin doğrudan bağlı oldukları düğüm aile olarak adlandırılır, diğer bir deyişle aile, kardeşlerin bağlı olduğu düğümdür.

Ata ve Torun(Descendant and Ancestor): Aile düğümünün daha üstünde kalan düğümlere ata denir. Torun, bir düğümün çocuğuna bağlı düğümlere denir.

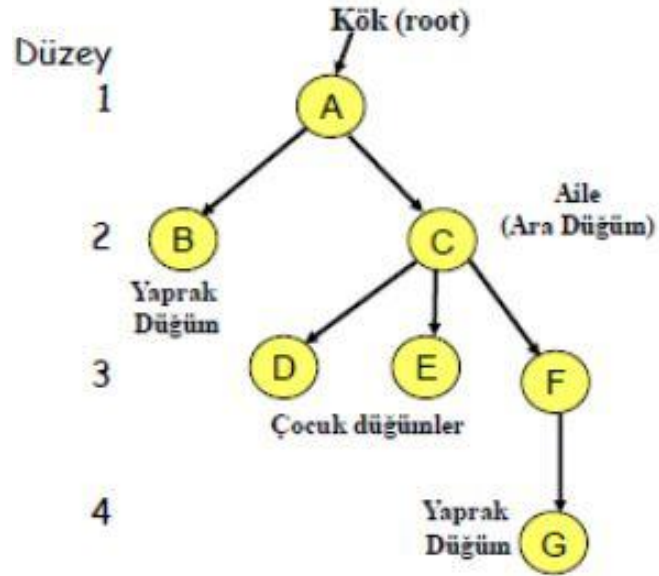
Orman(Forest): Ağaçlar kümesi orman olarak adlandırılır. Örneğin alfabedeki her bir karakter için oluşturulan sözlük ağaçları bir araya gelerek bir sözlük/orman oluşturur

Yol(Path): Bir düğümden aşağı doğru bir başka düğüme gidebilmek için üzerinden geçilmesi gereken düğümlerin listesidir; bir düğümden ancak kendi silsilesinde olan düğümlere yol olabilir. Yol uzunluğu ise başlangıç ve bitiş düğümleri arasındaki dal sayısıdır. Bu, yol üzerindeki düğüm sayısının bir eksiğidir. Ağaç üzerindeki bağlantılar iki yönlü olmadığı sürece yaprakların yol uzunluğu 0'dır. Yani yapraklardan herhangi bir düğüme yol yoktur. Kökten her yaprak düğme birer yol vardır.

Düzey(Level): İki düğüm arasındaki yolun üzerinde bulunan düğümlerin sayısıdır. Kök düğümün düzeyi 1, doğrudan köke bağlı düğümlerin düzeyi 2'dir.

Derinlik(Depth): Bir düğümün köke olan uzaklığıdır; yani düzeyidir. Kök düğümün derinliği 1'dir. Bir ağacın derinliği en uzak mesafede bulunan yaprak düğümün derinliğidir.

Yükseklik(Height): Bir düğümün kendi silsilesinde en uzak mesafedeki yaprak düğüme olan düzey sayısı yükseklik olarak adlandırılır. Ağacın yüksekliği kökün yüksekliğidir.



Şekildeki ağaç, **A düğümü kök olmak üzere 7 düğümden** oluşmaktadır. Sol alt ağaç B kökü ile başlamakta ve sağ alt ağaç da C kökü ile başlamaktadır. A'dan solda B'ye giden ve sağda C'ye giden **iki dal (branch)** çıkmaktadır.

• Düğüm (Node)

- Ağacın her bir elemanına düğüm adı verilir.

• Kök Düğüm (Root)

- Ağacın başlangıç düğümüdür.

• Çocuk (Child)

- Bir düğüme doğrudan bağlı olan düğümlere o çocukları denilir.

• Kardeş Düğüm (Sibling)

- Aynı düğüme bağlı düğümlere kardeş düğüm veya kısaca kardeş denir.

• Aile (Parent)

- Düğümlerin doğrudan bağlı oldukları düğüm aile olarak adlandırılır; diğer bir deyişle aile, kardeşlerin bağlı olduğu düğümdür.

• Ata (Ancestor) ve Torun (Descendant)

- Aile düğümünün daha üstünde kalan düğümlere ata denilir; torun, bir düğümün çocuğuna bağlı olan düğümlere denir.

• Yükseklik (Height)

- Bir düğümün kendi silsilesinden en uzak yaprak düğüme olan uzaklığıdır.

• Yol(Path)

- Bir düğümün aşağıya doğru (çocukları üzerinden) bir başka düğüme gidebilmek için üzerinden geçilmesi gereken düğümlerin listesidir.

Derece (Degree)

- Bir düğümden alt hiyerarşiye yapılan bağlantıların sayısıdır; yani çocuk veya alt ağaç sayısıdır.

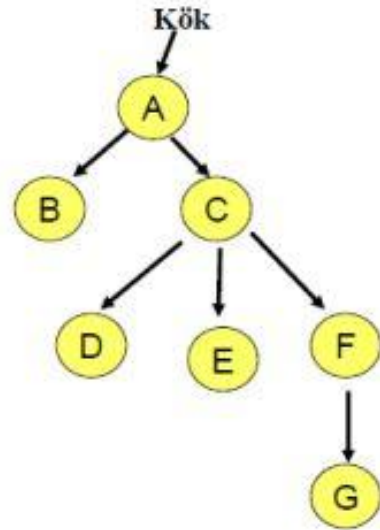
Düzyey (Level) ve Derinlik (Depth)

- Düzyey, iki düğüm arasındaki yolun üzerinde bulunan düğümlerin sayısıdır. Kök düğümün düzyeyi 1, doğrudan köke bağlı düğümlerin düzyeyi 2'dir. Bir düğümün köke olan uzaklığı ise derinliktir. Kök düğümün derinliği 1 dir.

Yaprak (Leaf)

- Ağacın en altında bulunan ve çocukları olmayan düğümlerdir.

Ağaç Veri Modeli Temel Kavramları



Tanım	Kök	B	D
Çocuk	2	0	0
Kardeş	1	2	3
Düzey	1	2	3
Aile	yok	kök	C
Ata	yok	yok	kök
Yol	A	A,B	A,C,D
Derinlik	1	2	3
Yükseklik	3	2	1

Dr.Öğ.Üyesi Ömer ÇETİN

<http://www.omercetin.com.tr/DERS/VY/Konu-10-Agaclar.pdf?i=1>

Ağaç Türleri ve Ağaç Üzerindeki İşlemler

Uygulamaya dönük olarak çok değişik şekillerde ağaç yapıları kurulabilir. Her biri için gerekli **veri yapısı** ve ağaç üzerinde işlem yapacak; arama, düğüm ekleme, düğüm silme gibi **algoritmalar** farklı olur. En çok bilinen ağaç türü, **ikili arama ağacı** olup kodlama ağacı, sözlük ağacı gibi birçok ağaç uygulaması vardır.

Ağaç veri yapısı uygulamaya esneklik getirmesi ve algoritma tasarımını kolaylaştırması açısından **değişik ağaç türleri** oluşturulabilir.

Bilinen ve standart olmuş ağaç türleri üzerine geliştirilmiş **birçok algoritma** vardır. Bunlar genel olarak ağaç türleri üzerinde **dolaşma** yapan, **düğüm ekleyen/silen**, **arama yapan** vb. birçok işlem için hazır algoritmalar/fonksiyonlar vardır. Program tasarımcısı bu gibi hazır algoritmaları kendi tasarımında **doğrudan veya değiştirerek** kullanabilir.

Oluşturduğu çok özel ağaç şekilleri için kendi algoritmasını tasarlamalıdır.

Ağaç veri yapılarının oluşturulmasında kesin bir kural yoktur; uygulamanın ihtiyacına göre farklı ağaç yapıları tasarlanabilir. Bu tasarımda en önemli nokta maliyet dengesidir. Yani bellek kullanımı ile işlem (yürütme) maliyeti kabul edilebilir bir seviyede tutulmalıdır. Gerektiğinde klasik ağaç yapısının dışına çıkılarak, graflarda olduğu gibi çevrim (cycle) içeren yapılar da oluşturulabilir.

Özellikle çok büyük veri kümelerinde ve sık arama yapılan uygulamalarda ağaç veri modeli önemli bir avantaj sağlar. Çünkü belirli bir anahtara göre yapılan aramalarda, ağaç yapıları diğer veri modellerine kıyasla daha düşük maliyetle (daha hızlı) sonuç verir.

Uygulamada Ağaç Üzerinde Gereksinim Duyulan İşlemler

Bir problemi çözmek için ağaç veri modeli seçildiğinde, öncelikle ağaç üzerinde dolaşmayı (traversal) sağlayan algoritmalar gereklidir. Tüm düğümlerin belirli bir sıraya göre gezilmesi sağlanmalıdır. Bunun yanında düğüm ekleme, düğüm silme, düğümleri listeleme ve düğümlerde tutulan verileri disk dosyasına kaydetme ya da dosyadan geri yükleme gibi temel işlemlere de ihtiyaç duyulur.

Ağaç veri modeli üzerinde gereksinim duyulacak temel işlemlerde aşağıdaki gibi özetlenebilir:

- Ağacı ilk oluşturma
- Ağacı dolaşma
- Düğüm ekleme
- Düğüm arama
- Düğüm silme
- Düğümleri listeleme
- Düğümleri saklama veya yükleme

Ağaçlar üzerinde **işlem yapmak** çoğu zaman **düğüm**ler üzerinde **dolaşmayı** gerektirir. Ekleme yapılması, arama yapılması, durum içeriklerini listeleme, silme gibi bütün işlemler ağaç düğümleri üzerinde **dolaşarak** gerçekleştirilir. Üstelik **rekürsif fonksiyon** yapısı kullanılırsa ağaç üzerinde işlem yapan **algoritmaların tasarımı** kolaylaşır.

Ağaçların Bellek Üzerinde Tutulması

Ağaç veri yapısı, **tüm düğümler arasındaki bağlantıları tutabilecek** şekilde tasarlanmalıdır. Bu bağlantıları bellekte temsil etmek için iki temel yaklaşım vardır:

- Düğüm bağlantısı
- İndis bağlantısı

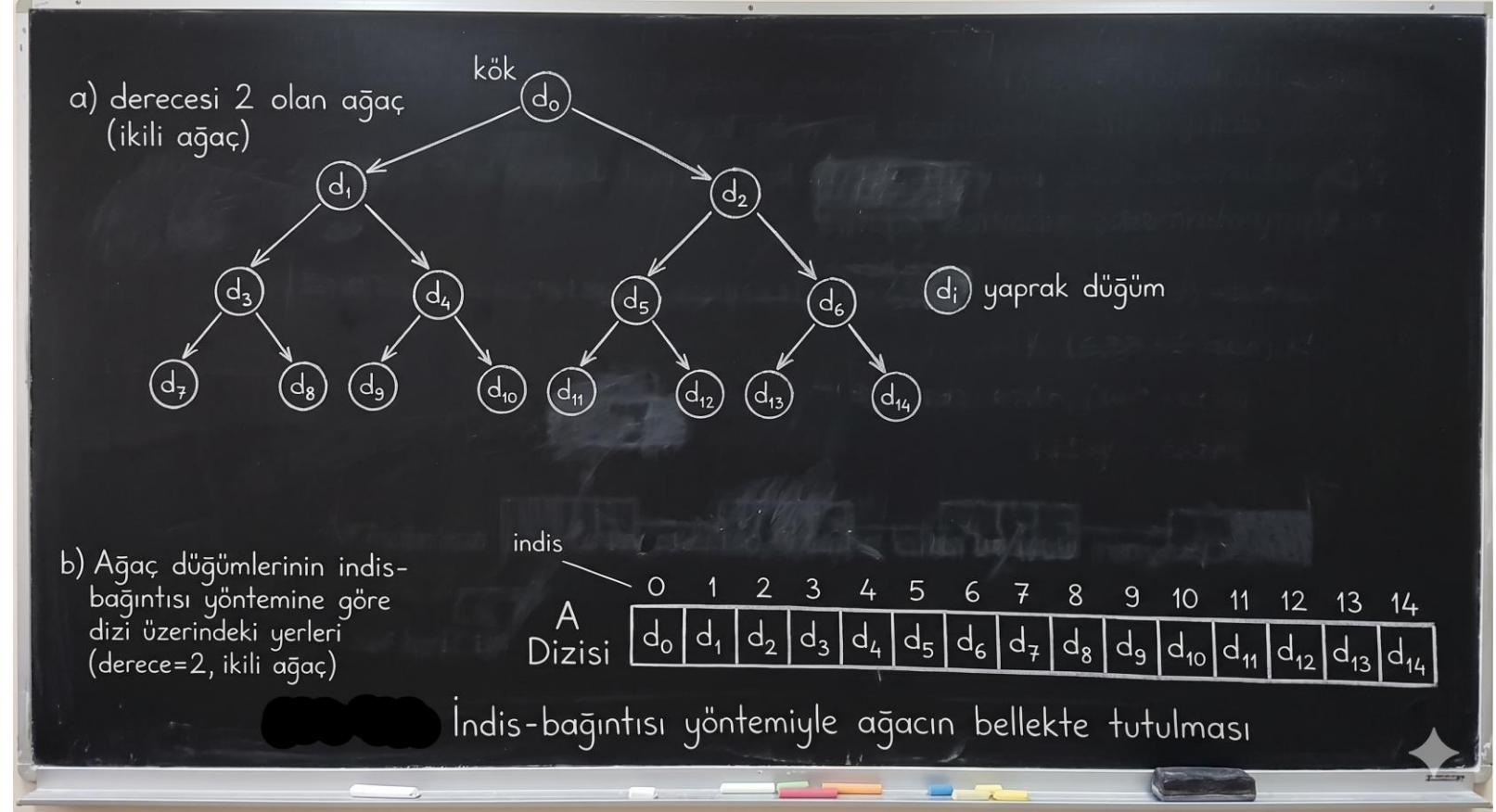
İndis-bağıntısı yöntemi ile Ağacın Kurulması

İndis-bağıntısı yönteminde, ağaçtaki düğümler bir dizi (array) içinde saklanır. Dizinin ilk elemanı kök düğümü temsil eder. Diğer düğümlere ise belirli bir kurala göre, yani indis (index) hesaplanarak ulaşılır.

Bu yöntemde, bir düğümün çocuklarına erişmek için o düğümün dizideki konumu (indisi) kullanılır ve çocuk düğümlerin indisleri bir formül yardımıyla hesaplanır. Böylece ekstra bir bağlantı (pointer) bilgisine ihtiyaç duyulmadan, sadece indis hesaplayarak düğümler arasında geçiş yapılabilir.

İndis Bağıntısıyla Ağaç Kurulması ve Veri Yapısı

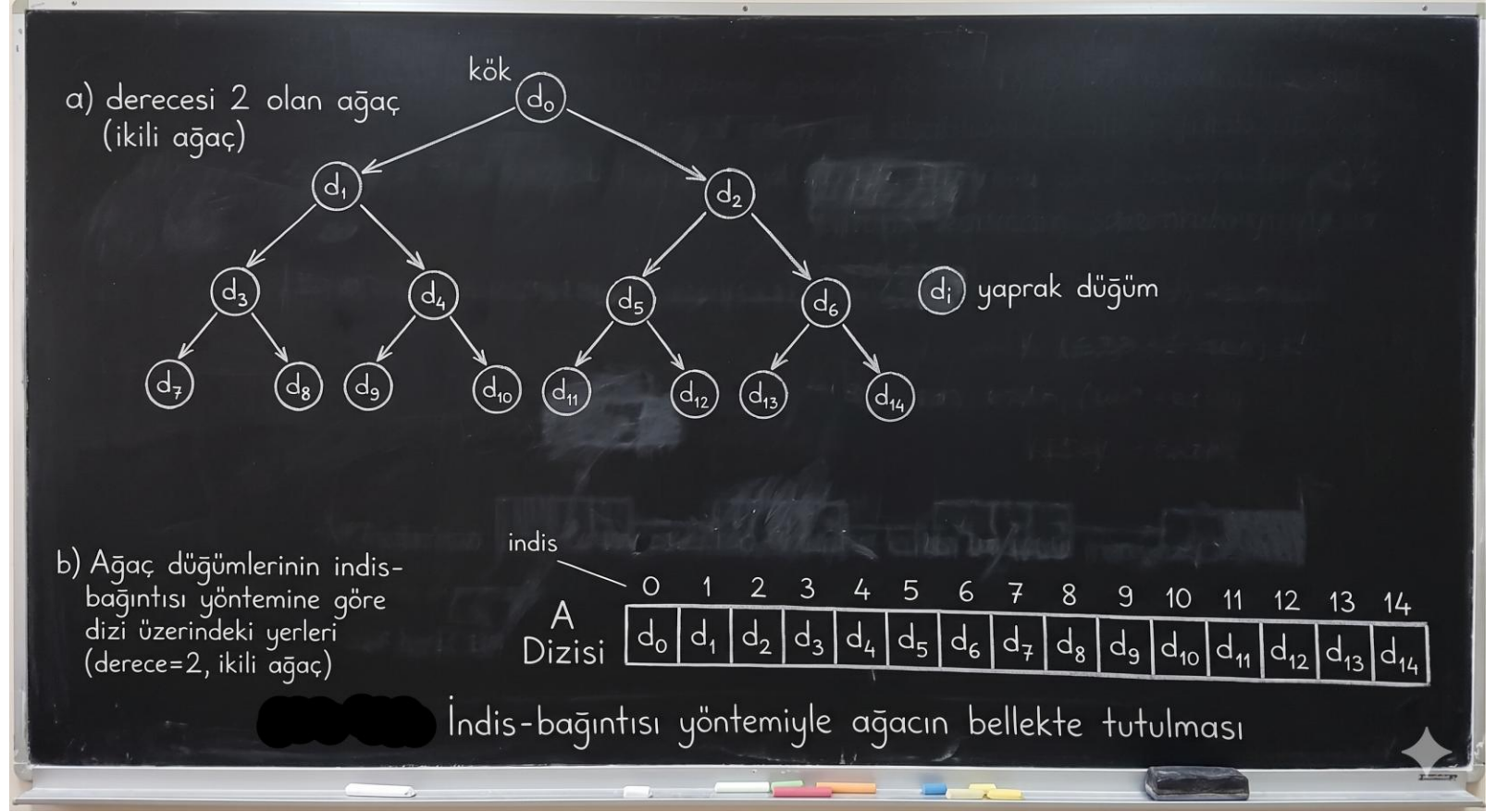
Ağacın **indis bağıntısı** yöntemine göre bellekte tutulması şekilde görüldüğü gibi düğümlerin veri kısmının bir dizi üzerinde tutulması kavramına dayanır.



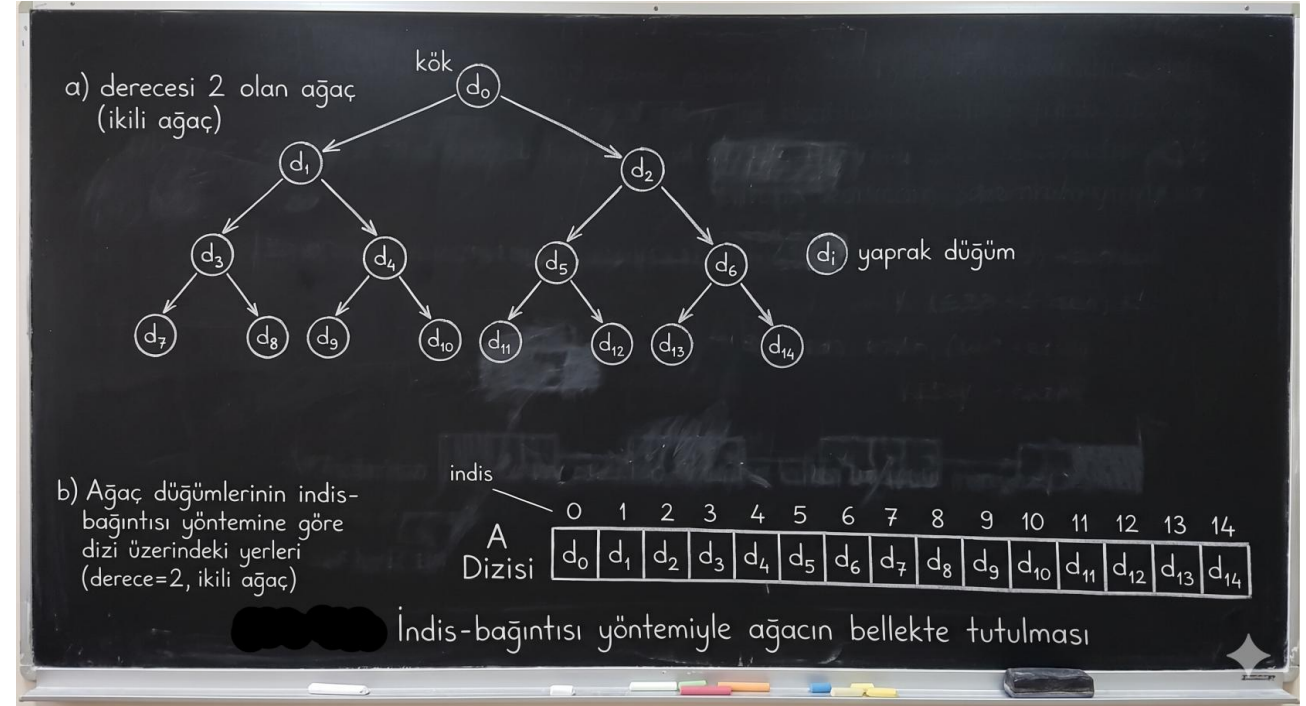
Dizinin ilk elemanı ağacın köküdür; dizinin ikinci ve üçüncü elemanları kök düğümün çocuklarıdır; daha sonraki yani 4.'den sonraki düğümler için ağacın derecesi bilinmelidir. Ağacın derecesine bağlı olarak bir indis bağıntısı bulunur ve düğümler arasındaki akrabalık ilişkileri bu bağıntı uyarınca belirlenir. Bu nedenle bu yöntem indis-bağıntısı olarak adlandırılır; yani **aranan bağlantıya indis değeri üzerinden erişilir.**

Şekilde verilen ağaç yapısında herhangi bir düğümün ağaç yapısına bir bağlantı uyarınca doğrudan erişilebilir.

Kökün indis değeri 0'dır. J indisli düğümün sol alt düğümü $(2j+1)$, sağ alt düğümü de $(2j+2)$ indisli gözlerde bulunur. Örneğin, dizide 3 indisli gözde bulunan d_3 düğümüne bağlı sol alt düğüm indisi $(2j+1)=7$, sağ alt düğümün indisi de 8 olur.



Eğer hesaplanan indis değeri en büyük indis değerinden büyük çıkarsa söz konusu düğümün olmadığı anlaşılır. Örneğin A dizisi eleman sayısı 15 olup dizinin son indisi'14'tür. Buna göre d_7 düğümüne bağlı çocuk olup olmadığı benzer hesaplama ile belirlenebilir; $j = 7$ olduğuna göre sol alt bölümü 15 ve sağ alt düğme 16 indisli düğümlere sahip olmalıdır. Bu değerler 14 olan en büyük indis değerlerinden büyük olduğu için d_7 'nin çocuk düğümleri yoktur denilebilir.

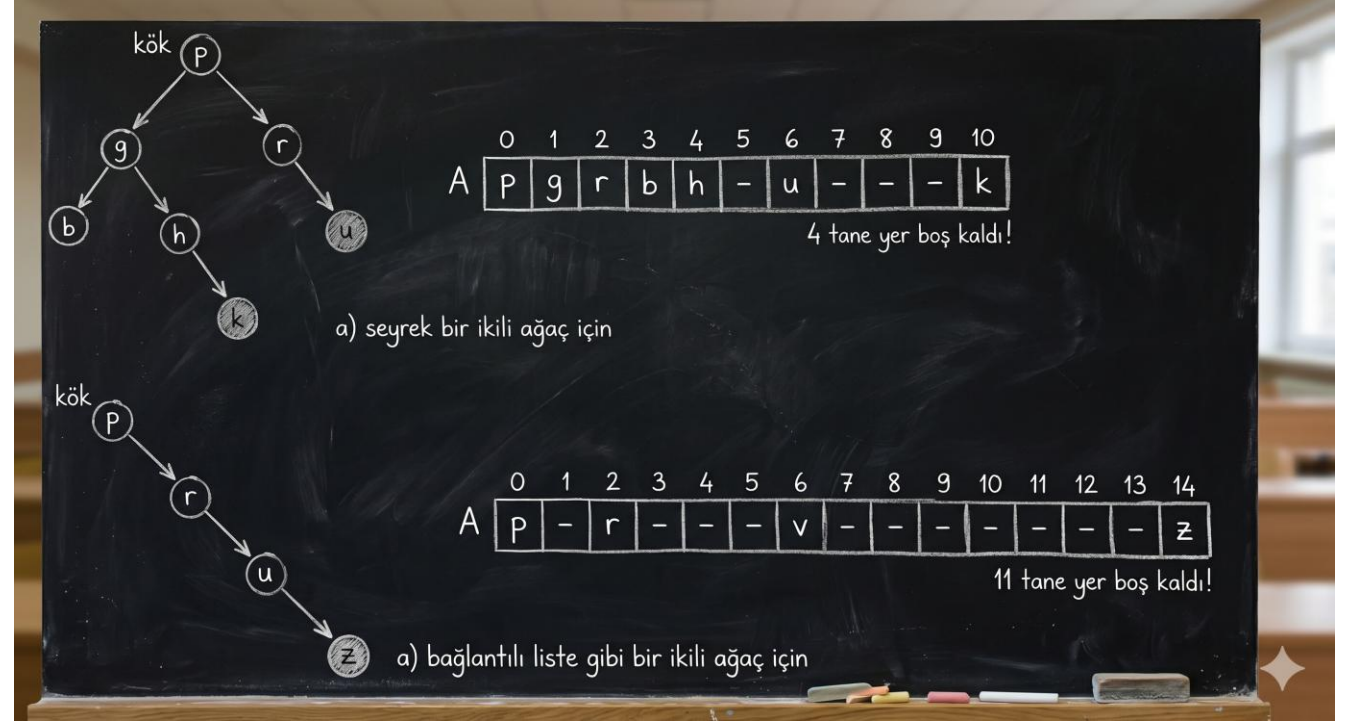


İndis Bağıntısı Yönteminin Olumsuz Yanı

Bu yöntem **seyrek özellikte ağaçlar için fazla bellek alanı** harcar. Çünkü bu yöntemde henüz olmayan düğümler içinde yer ayrılır.

Dolayısıyla ağaç seyrek özellikte ise veya bazı alt ağaçları bağlantılı liste gibi tek bir yönde uzayıp giderse eklemelerde ağacın düğümleri fazla artmadığı halde düzeyi artarsa bellek **sarfiyatı oldukça artar**

indissol(j)=2j+1 ve indissag(j)=2j+2 denklemlerine göre



İndis Bağıntısı Yönteminin Olumlu Yanı

Bu yöntem **dolu ağaçların** veya **dengeli ağaçların** tutulmasında kullanışlıdır. Çünkü **ağaç dengeliyse boşuna bellek kullanılmamış olur** ve ağacın herhangi bir düğümünden onun herhangi bir alt bölümüne bir bağıntı hesabı yapılarak aynı çırpı fonksiyonlarında olduğu gibi doğrudan erişilir. Örneğin ağaç dengeli ve ekleme sol taraftan yapılmış ise **Düğüm**ler **$indissol(j)=2j+1$ ve $indissag(j)=2j+2$ denklemlerine göre arada boş kalmayacak şekilde ağaca yerleştigiinden dolayı dizinin herhangi bir gözü boş kalmaz.**

Düğüm Bağlantısı ile Ağacın Kurulması

Bu yöntemde ilk akla gelen **her bir bağlantı için birer bağlantı bilgisi tutulmasıdır**. Dolayısıyla veri yapısı içerisinde **bilginin tutulacağı değişkenlere** ek olarak **m tane indis/işaretçi** değişken olmalıdır.

Düğüm bağlantısı olan yöntemde **düğümler arası bağlantılar özel olarak veri yapısı içerisine eklenmiş olan işaretçi değişkenler** aracılığı ile sağlanır. Bellek alan maliyeti biraz kabarsa bile çoğu uygulamada bu yöntem seçim olmaktadır denilebilir.

Aşağıda verilen sol örnekte dinamik, sağ örnekte dizi üzerinde ağaç veri yapısını temsil eden struct (yapı) tanımı gösterilmektedir

```
struct isaretcili {  
    int veri;  
    struct isaretcili *cocuk;  
    struct isaretcili *kardes;  
};
```

```
struct indisli {  
    int veri;  
    unsigned int cocuk;  
    unsigned int kardes;  
};
```

İKİLİ AĞAÇLAR

İkili Ağaç (Binary Tree), her düğümün **en fazla iki çocuğu** olan bir ağaç veri yapısıdır. Bu, her düğümün yalnızca **sol** ve **sağ** olmak üzere iki çocuğa sahip olabileceği anlamına gelir. İkili ağaçlar, özellikle arama, sıralama ve veri depolama işlemlerinde yaygın olarak kullanılır.

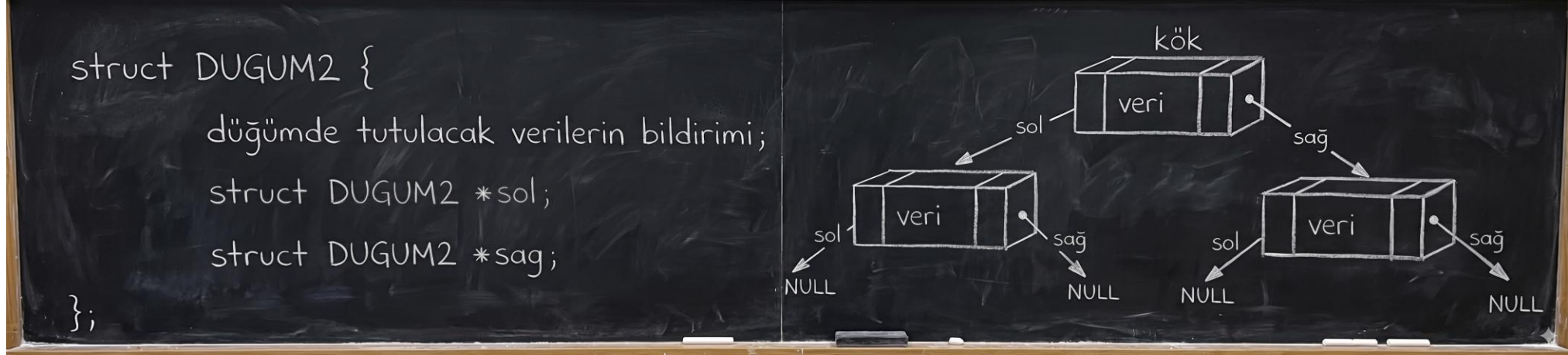
İkili ağaçta **bir düğümün derecesi en fazla 2 olabilir yani bir düğüme en fazla 2 tane alt düğüm bağlanabilir**. Eğer ağacın kurulmasında eklenecek olan düğüm için anahtar veriye bağlı olarak küçük olan değerler sol tarafa büyük olan değerlerde sağ tarafa

veya

tersi yaklaşımla yapılmalıdır. İkili arama ağaçlarında, sıralamada genel yaklaşım küçük olanı sola büyük olanı sağa eklemektir. **(BST kuralı sabittir: küçük sol – büyük sağ)**

İkili ağaçta DÜĞÜM BAĞLANTISI YOLU ile ağaç kurulması için

düğümde ait veri yapısında bilgiye ek olarak iki tane işaretçi gereklidir. Biri sol diğeri sağ olarak adlandırılan bu işaretçiler düğümlerin çocuklarını bağlamak için kullanılır. Böyle bir veri yapısının olası tanımı;



Eğer bir düğümün sol tarafına bir başka düğüm veya bir alt ağaç bağlıysa son adlı işaretçi de bağlı olan düğümün adresi bulunur; aksi durumda boş/bağlantı yok anlamında NULL değeri bulunur. Şekillerde **NULL taralı** olarak gösterilmiştir. Benzer durum sağ içinde geçerlidir. Yaprak düğümlerin sol ve sağ taraflarında NULL değerleri vardır. **Kısacası bir düğümün iki çocuğu sol ve sağ üzerinden bağlanır.**

İkili Arama Ağaçları İçin Algoritmalar

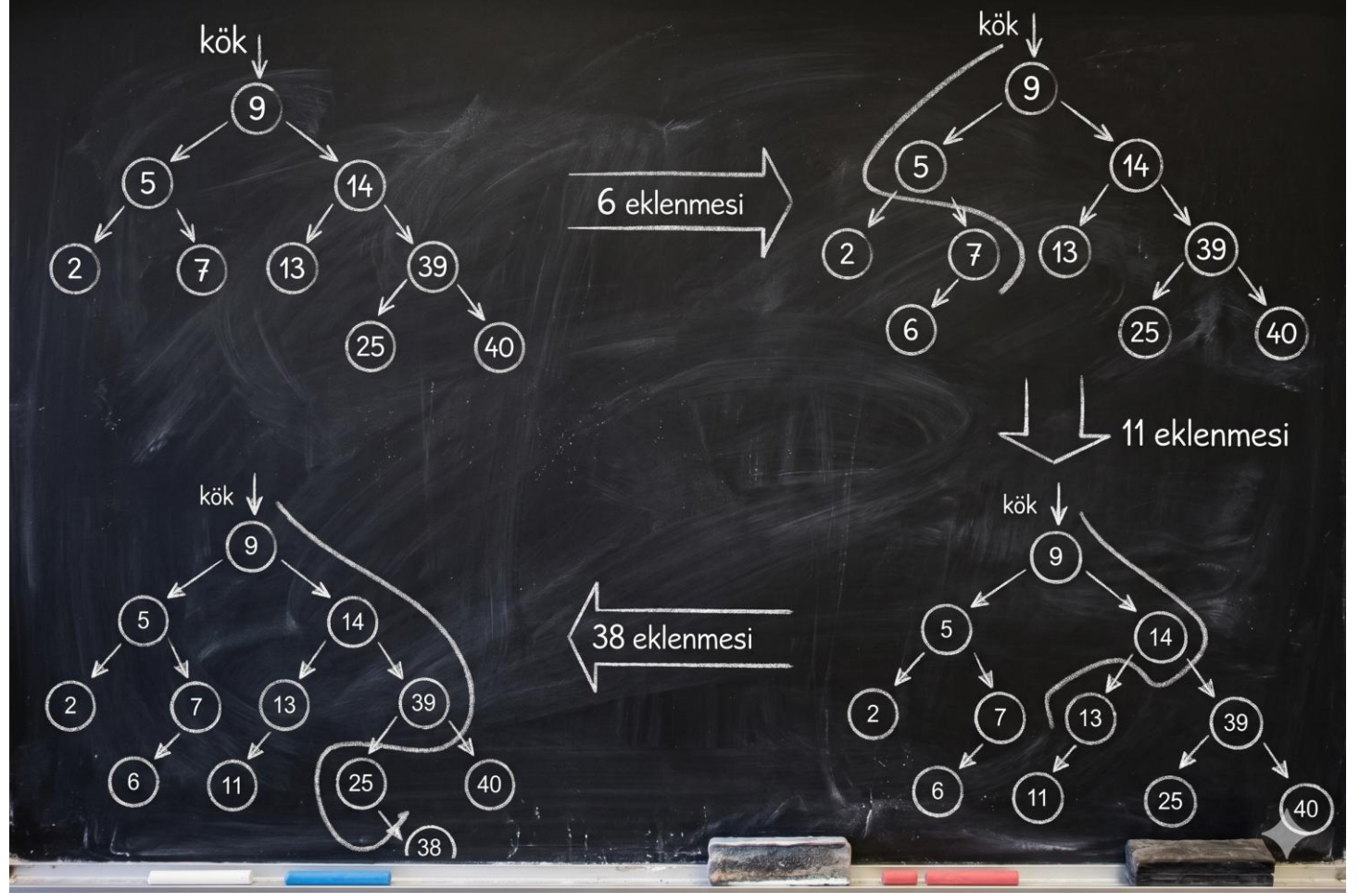
Düğüm Ekleme Algoritması

İkili arama ağacında **ekleme yapılması** için önce yeni gelen verinin anahtar sözcüğüne göre **yerinin bulunması** gerekir. Ekleme algoritmasında ilk bakılması gereken ağacın kökünün boş olup olmadığıdır. Daha ağacın kökünde hiçbir şey yoksa ekleme doğrudan köke yapılır. Ancak kök dolu ise bu durumda eklemenin sol alt ağaca mı yoksa sağ alt ağaca mı olacağı sınılanır. Eğer eklenecek veri, kökteki değerden **küçükse sol, büyükse sağ** alt ağaca yapılacaktır. Dolayısıyla bu alt ağaçlardan birine dallanılır ve gidilen alt ağacın bağımsız bir ağaç olduğu varsayılarak aynı işlemler yinelenir.

İkili arama ağacına düğüm ekleme algoritması

```
if (kök == NULL)
    kök = yeniDüğüm;
else {
    mevcut = kök;

    while (1) {
        if (yeniDüğüm->veri < mevcut->veri) {
            if (mevcut->sol == NULL) {
                mevcut->sol = yeniDüğüm;
                break;
            }
            mevcut = mevcut->sol;
        }
        else {
            if (mevcut->sağ == NULL) {
                mevcut->sağ = yeniDüğüm;
                break;
            }
            mevcut = mevcut->sağ;
        }
    }
}
```



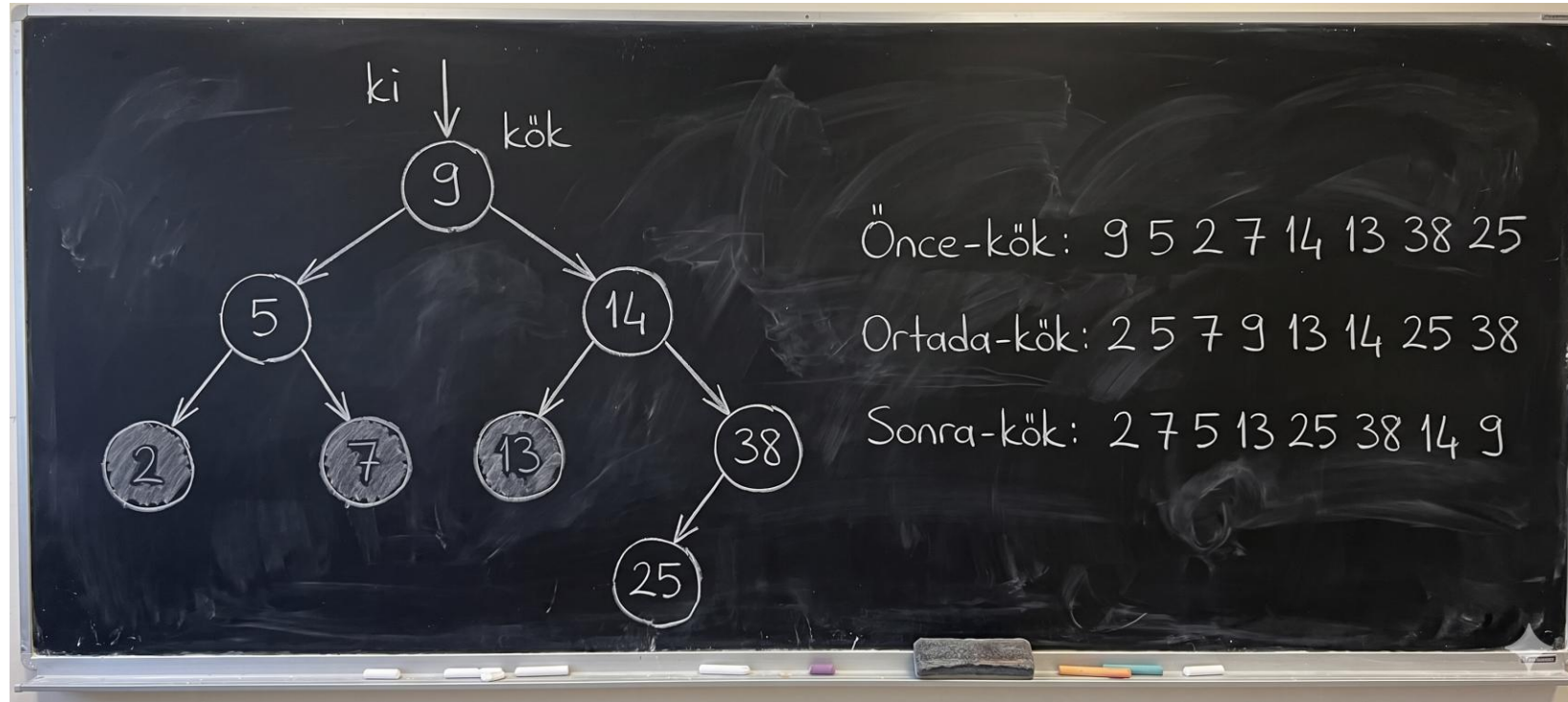
İkili ağaç üzerinde dolaşma ve düğümlere erişim

İkili ağaç üzerinde **dolaşma birçok şekilde yapılabilir**. Ancak rastgele dolaşmak yerine önceden belirlenmiş bir yönteme uyulması algoritmik ifadeyi kolaylaştırır. Önce-kök(preorder), ortada-kök(inorder), sonra-kök(postorder) olarak adlandırılan **3 değişik dolaşma şekli** çeşitli uygulamalara çözüm olmaktadır.

Önce-kök / -> **Kök**, Sol taraf, Sağ taraf

Ortada-kök / Inorder -> Sol taraf, **Kök**, Sağ taraf

Sonra-kök / Postorder -> Sol taraf, Sağ taraf, **Kök**



Önce-kök / -> **Kök**, Sol taraf, Sağ taraf

Ortada-kök / Inorder -> Sol taraf, **Kök**, Sağ taraf

Sonra-kök / Postorder -> Sol taraf, Sağ taraf, **Kök**

Ağaç üzerinde dolaşma kök dahil **tüm düğümlerin ziyaret edilmesi** işlemidir. Düğüme ulaşıldığında uygulama gereksinimine göre düğüm bilgisini yazdırma, başka bir birime aktarma, karşılaştırma vs. gibi işlemler yapılabilir. **Dolaşma işlemi daha önce açıklandığı gibi önce-kök, ortada-kök, sonra-kök şeklinde yapılabilir.** Aşağıda **önce-kök** şeklinde rekürsif yapıya uygun bir dolaşma algoritmasının kaba kodu verilmiştir.

ikili arama ağacı üzerinde dolaşma algoritması

```
if ( kök boş değilse; düğüm varsa ) {  
    düğüm üzerindeki bilgiyi kullan;  
    fonksiyonu kökün sol altağacı için rekürsif olarak tekrar çağır;  
    fonksiyonu kökün sağ altağacı için rekürsif olarak tekrar çağır;  
}
```

Ağaç üzerinde dolaşma daha çok tüm düğümlerin veya büyük bir kısmının listelenmesi veya başka nedenlerle kullanılması durumunda gereksinim duyulan bir işlemdir. Tüm düğümler ziyaret edileceği için karmaşıklığı $O(N)$ olur yani doğrusal liste gibi..

İkili Arama Ağacında Arama Algoritması

ikili arama ağacı üzerinde **arama**, onun en önemli yanıdır. Dengeli denilebilecek bir ikili arama ağacında arama işlemi $O(\log N)$ karmaşıklığında gerçekleştirilir. Dolayısıyla ikili ağaçların bu kadar yaygın kullanılmasının nedeni bu özelliktir.

İkili arama ağacı üzerinde arama algoritması ü

```
while (kök != NULL ve kök->veri != aranan) {  
    if ( aranan kökteki bilgiden küçükse )  
        kök = kökün sol çocuğu adresi;  
    else  
        kök = kökün sağ çocuğu adresi;  
}  
return kök;
```

İkili arama ağaçları, arama işlemlerinin yoğun olduğu uygulamalarda etkin bir veri yapısı olarak kullanılmaktadır. Uygun şekilde dengelenmiş bir ikili arama ağacında arama işlemi, anahtar değerler üzerinden karşılaştırma yapılarak ortalama $O(\log N)$ zaman karmaşıklığında gerçekleştirilir.

Arama algoritması dolaşma algoritmasından farklılık gösterir. Çünkü dolaşma algoritmasında tüm düğümlerin ziyaret edilmesi gerekir ancak arama algoritmasında arananın bulunduğu yere doğru ilerlenmesi yeterlidir. Yani kendi silsilesinde yoksa ağaçta da yoktur denilebilir.

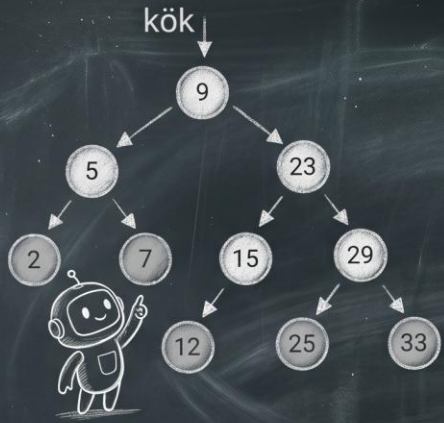
İkili arama ağacında (BST) arama işleminin **en kötü durum zaman karmaşıklığı: $O(N)$**

Aranan ağaçta birden çok varsa ilk bulduğu yerin adresini gönderir

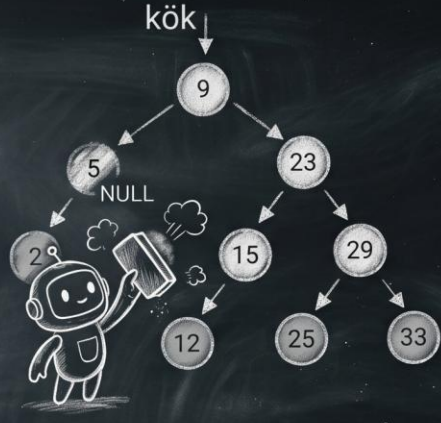
Düğüm Silme Algoritması

İkili arama ağaçlarında düğüm silme işlemi, yapının temel özelliklerinin korunması açısından dikkatle ele alınması gereken bir süreçtir. Bu işlem sırasında hem ikili arama ağacı özelliğinin sürdürülmesi hem de mümkün olduğunca yapısal dengenin bozulmaması hedeflenir. Genel olarak, bir düğümün silinmesi sürecinde üç temel durum söz konusudur. Silinecek düğüm:

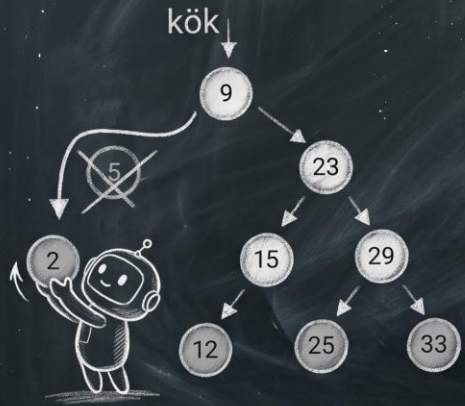
- Yaprak düğüm olabilir (çocuğu yoktur),
 - Tek çocuklu bir düğüm olabilir,
 - İki çocuklu bir düğüm olabilir.



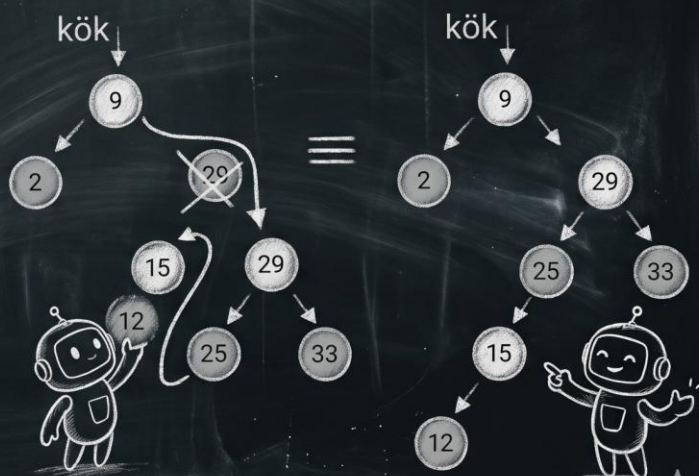
a) Tipik olarak ağaç



b) 7'nin silinmesi; yaprak



c) 5'nin silinmesi; bir çocuk var



d) 23'ün silinmesi; iki çocuk var

Silinecek eleman b'de görüldüğü gibi **yaprak düğüm** olabilir; bu durumda onun ailesindeki işaretçi alanına **NULL** yerleştirilir ya da;

c'de görüldüğü gibi yalnızca bir tarafında alt ağaç olabilir; bu durumda **o alt ağacın bağlantısı çıkarılacak düğümün ailesindeki işaretçi alanına yerleştirilir** ya da;

d'de görüldüğü gibi çıkarılacak düğümün iki tarafında da alt ağaç olabilir. Yani iki tane çocuğu da vardır. **Bu durumda sol alt ağaç sağ alt ağacın en küçük düğümünün soluna eklenir**

b'de verilen durum ağacın dengesinin bozulduğu bir yaklaşımdır. Çünkü silinmek istenen düğümün sol alt ağacı tümüyle sağ alt ağacının en küçük değere sahip düğümünün sonuna eklenmektedir. Dolayısıyla b'de görüldüğü gibi tek yönde bir büyüme olmaktadır. Ağaçtan düğüm silme c'de gösterildiği gibi yapılırsa ağacın dengesi fazla bozulmadan çıkartma işlemi gerçekleştirilmiş olur. Görüldüğü gibi c'de önce silinecek düğünün **sağ alt ağacında ki en küçük değere sahip düğümü bulunmakta**; Bu bir yaprak düğüm olacaktır. Ardından silinecek düğüm ile bu yaprak düğüm yer değiştirmektedir. Yer değiştirme işleminden sonra silinecek düğüm, kendi silsilesinde sağ alt ağacının en **sondaki yaprak düğünündedir**. Artık **silme işlemi kolayca yapılır**; ailesiyle bağlantısı kesilir ve işgal ettiği bellek alanı serbest bırakılır.

Sonuçta ağaç, silme
işleminde sonra YANDAKİ
gibi olur
**ikili arama ağacından düğüm silme
algoritması kaba-kodu**

Silinecek düğüm **q** bulunur, ebeveyni **qa** belirlenir

Eğer **q bulunamazsa** → işlem sonlandırılır

Eğer q yaprak ise (çocuğu yoksa):

q, ebeveyninin ilgili bağlantısından kaldırılır

Eğer q'nun tek çocuğu varsa:

q'nun tek çocuğu belirlenir

Bu çocuk, q'nun ebeveynine bağlanır

q bağlantıdan çıkarılır

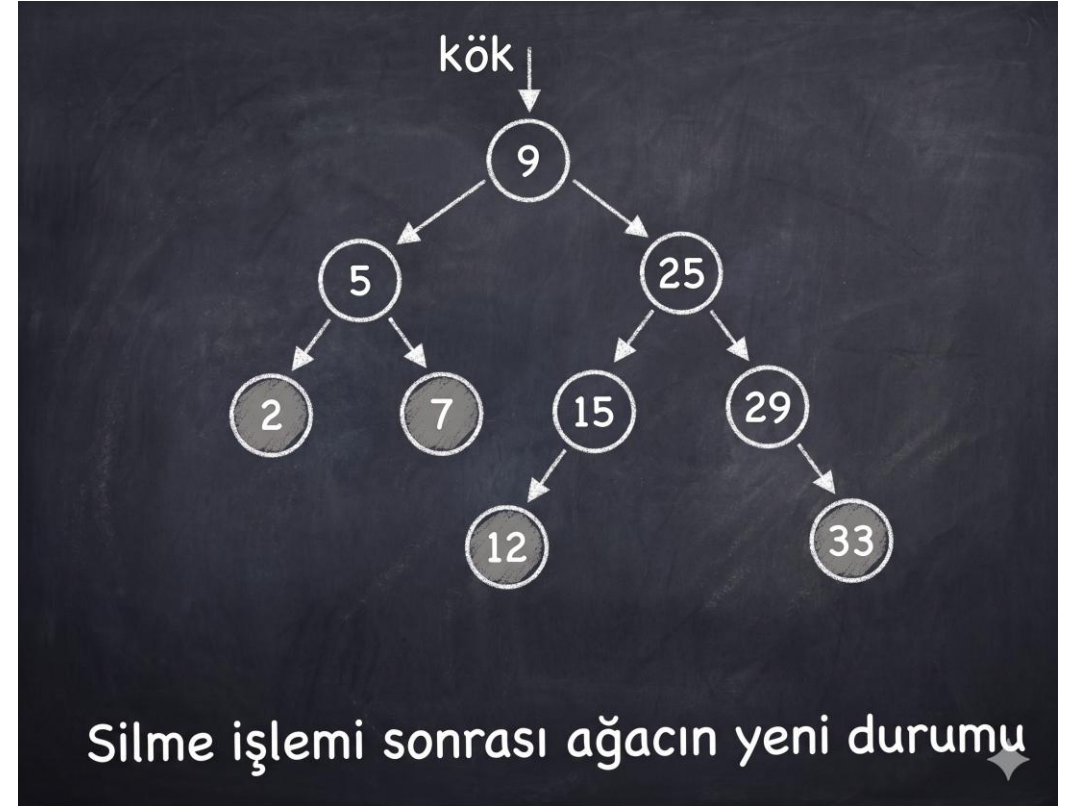
Eğer q'nun iki çocuğu varsa:

q'nun yerine geçecek düğüm seçilir:

- sol alt ağacın en büyük düğümü **veya**

- sağ alt ağacın en küçük düğümü

Bu düğümün değeri q'ya aktarılır



Bu kod ikili arama ağaçlarında (Binary Search Tree - BST) DOLAŞMA (traversal) ve ağaç oluşturma işlemidir.

```
#include <iostream>
using namespace std;

struct node{
    int data;
    struct node * left;
    struct node * right;
};

struct node * kokDugum(int key){
    struct node * root=new node();
    root->right=NULL;
    root->left=NULL;
    root->data=key;
}

struct node * dugumEkle(struct node * root, int key){
    if(root==NULL){
        root=kokDugum(key);
    }
    else{
        if(key<root->data){
            root->left=dugumEkle(root->left,key);
        }
        else{
            root->right=dugumEkle(root->right,key);
        }
    }
    return root;
}
```

2. Kendi Kendini Çağırma
İş bir alt kademeye devredildi.

```
void preOrder(struct node * root){
    if(root!=NULL){
        cout<<root->data<<" ";
        preOrder(root->left);
        preOrder(root->right);
    }
}

void inOrder(struct node * root){
    if(root!=NULL){
        inOrder(root->left);
        cout<<root->data<<" ";
        inOrder(root->right);
    }
}

void postOrder(struct node * root){
    if(root!=NULL){
        postOrder(root->left);
        postOrder(root->right);
        cout<<root->data<<" ";
    }
}
```

```

int main() {
    struct node * root=NULL;
    int sayi;
    int i=0;
    while(i<7){
        cout<<"Sayi girin"<<endl; cin>>sayi;
        root=dugumEkle(root,sayi);
        i++;
    }
    cout<<"preorder :"; preOrder(root);
    //preOrder(root);
    cout<<endl;
    cout<<"inorder :"; inOrder(root);
    //inOrder(root);
    cout<<endl;
    cout<<"postOrder :"; postOrder(root);
    //postOrder(root);
    cout<<endl;

    return 0;
}

```

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\agaclar>g++ -o agacyapisi agacyapisi.cpp

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\agaclar>agacyapisi

```

Sayi girin
5
Sayi girin
1
Sayi girin
4
Sayi girin
8
Sayi girin
9
Sayi girin
7
Sayi girin
3
preorder :5 1 4 3 8 7 9
inorder :1 3 4 5 7 8 9
postOrder :3 4 1 7 9 8 5

```

Faktöriyel(Factorial) Fonksiyon:

- $4! = 4 \cdot 3 \cdot 2 \cdot 1 = 24$
- $4! = 4 \cdot 3!$
- $3! = 3 \cdot 2!$
- $2! = 2 \cdot 1!$
- $1! = 1 \cdot 0!$
- $0! = 1$ (Base case)

$$n! = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot (n-1)! & \text{eğer } n > 0 \end{cases}$$

```
#include <iostream>
using namespace std;
```

```
// Rekürsif Faktöriyel Fonksiyonu
```

```
int faktoriyel(int n) {
    // 1. DURMA NOKTASI (Base Case):
    // n sıfır olduğunda artık kendini çağırmayı bırakır ve 1 döner.
    if (n == 0) {
        return 1;
    }
    // 2. REKÜRSİF ADIM:
    // Fonksiyon, sonucu bulmak için kendini n-1 ile tekrar çağırır.
    else {
        return n * faktoriyel(n - 1);
    }
}

int main() {
    int sayi;
    cout << "Bir sayi girin: ";
    cin >> sayi;
    // Fonksiyonu çağırıyoruz ve sonucu ekrana basıyoruz
    cout << sayi << "! = " << faktoriyel(sayi) << endl;
    return 0;
}
```

İkili arama ağaçlarında (Binary Search Tree - BST)

EKLEME İşlemi

```
#include <iostream>
using namespace std;

// Düğüm Yapısı
struct node {
    int data;
    node* left;
    node* right;
};

// Yeni bir düğüm oluşturma fonksiyonu
node* yeniDugum(int veri) {
    node* root = new node();
    root->data = veri;
    root->left = NULL;
    root->right = NULL;
    return root;
}

int main() {
    node* root = NULL;

    // Sayıları sırayla ekleyelim
    root = dugumEkle(root, 50);
    root = dugumEkle(root, 30);
    root = dugumEkle(root, 70);
    root = dugumEkle(root, 20);

    cout << "Dugumler basariyla eklendi!" << endl;
    return 0;
}
```

```
// Rekürsif Düğüm Ekleme Fonksiyonu
node* dugumEkle(node* root, int veri) {
    // 1. DURMA NOKTASI: Eğer boş bir yer bulursak düğümü ekle
    if (root == NULL) {
        return yeniDugum(veri);
    }

    // 2. REKÜRSİF ADIMLAR:
    if (veri < root->data) {
        // Veri küçükse sol dala git ve sonucu sol kola bağla
        root->left = dugumEkle(root->left, veri);
    }
    else {
        // Veri büyük veya eşitse sağ dala git ve sonucu sağ kola bağla
        root->right = dugumEkle(root->right, veri);
    }

    // Güncellenmiş düğümü geri döndür (Bağlantının kopmaması için şart)
    return root;
}
```

İkili arama ağaçlarında (Binary Search Tree - BST) SİLME işlemi

```
// Bir ağaçtaki en küçük değerli düğümü bulan yardımcı fonksiyon
node* enKucukDeger(node* root) {
    node* current = root;
    while (current && current->left != NULL)
        current = current->left;
    return current;
}
```

```
node* dugumSil(node* root, int key) {
    if (root == NULL) return root; // Ağaç boşsa veya bulunamadıysa
    // 1. Silinecek düğümü arıyoruz
    if (key < root->data)
        root->left = dugumSil(root->left, key);
    else if (key > root->data)
        root->right = dugumSil(root->right, key);

    // 2. Düğümü bulduk (key == root->data)
    else {
        // Durum A: Tek çocuklu veya çocuksuz
        if (root->left == NULL) {
            node* temp = root->right;
            delete root;
            return temp; // Sağdakini yukarı (üst düğüme) gönder
        }
        else if (root->right == NULL) {
            node* temp = root->left;
            delete root;
            return temp; // Soldakini yukarı gönder
        }
        // Durum B: İki çocuklu düğüm
        // Sağ alt ağacın en küçüğünü (inorder successor) bul
        node* temp = enKucukDeger(root->right);

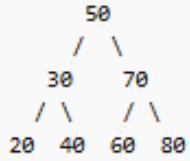
        // En küçüğün verisini sildiğimiz yere kopyala
        root->data = temp->data;

        // Kopyaladığımız o en küçük düğümü aşağıdan sil
        root->right = dugumSil(root->right, temp->data);
    }
    return root;
}
```

önce silinecek düğümün **bir kez sağına** gidiliyor
Ardından enKucukDeger fonksiyonu çağrılıyor. Bu fonksiyon kendi içinde bir while döngüsüyle **sürekli sola** gidiyor.

İkili arama ağaçlarında (Binary Search Tree - BST) SİLME İşlemi

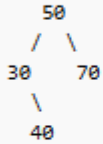
ÖRNEK AĞAÇ



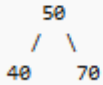
2. DURUM: Tek çocuk (30'u sil)

30'un çocukları: (20 ve 40 yok artık varsayalım)

Diyelim 30'un sadece sağ çocuğu var:



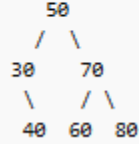
Silince:



✓ 40 yukarı çıkar

1. DURUM: Yaprak silme (20'yi sil)

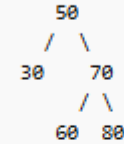
20'nin çocuğu yok:



3. DURUM: İKİ ÇOCUK (50'yi sil)

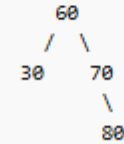
50'nin iki çocuğu var → 30 ve 70

Adım 1: Sağ alt ağacın EN KÜÇÜĞÜNÜ bul (60)



→ 60 seçilir

Adım 2: 50 yerine 60 yaz



Adım 3: Eski 60 düğümünü sil

Dengeli Ağaç ve AVL Ağaç Yapısı

Dengeli ağaç aynı doğada olduğu gibi serpilmesini tüm dallarına **homojen bir biçimde** yansıtan ağaç şeklidir; tanım olarak herhangi bir düğüme bağlı **alt ağaçların yükseklikleri arasındaki fark en fazla 1** olmalıdır. Bir dalın fazla büyümesi ağacın dengesini bozar ve ağaç üzerine hesaplanmış karmaşıklık hesapları ve yürütme zamanı bağıntılarından sapılır. Dolayısıyla ağaç veri modelinin en önemli getirisi kaybolmaya başlar. Örneğin bir ikili ağaç uygulamasında ağaca eklenecek veriler hali hazırda sıralı olarak gelirse; **sonuç ağaç değil de bağlantılı liste şeklinde olur. Bu, ağaç veri modelinde istemeyen bir durumdur. İstenen özellikle arama ağaçlarında ağacın dengeli genişlemesidir**

AVL ağacı, dengeli ikili arama ağacı (binary search tree, BST) türlerinden biridir. Adını, bu ağacın icatçıları olan **Adelson-Velsky** ve **Landis**'ten alır. AVL ağacının temel amacı, her zaman dengede olmasını sağlamaktır. Yani, her düğümün altındaki alt ağaçların yükseklik farkının, **en fazla 1** olmasını sağlar. Bu, arama, ekleme ve silme işlemlerinin her zaman **$O(\log n)$** zaman karmaşıklığında gerçekleşmesini sağlar.

AVL Ağacının Tanımı ve Temel Özellikleri:

- **Dengeli Ağaç:** Bir AVL ağacında, her düğüm için sol ve sağ alt ağaçlarının yükseklik farkı (daha doğru bir ifadeyle, sol alt ağacın yüksekliği ile sağ alt ağacın yüksekliği arasındaki fark), **-1, 0, veya 1** olmalıdır. Eğer bu fark **-1** ile **1** arasında değilse, ağacın dengesi bozulur ve yeniden dengeleme (rebalancing) işlemi yapılır.
- **Yükseklik Hesaplama:** Bir düğümün yüksekliği, o düğümden aşağıya doğru en uzun yol boyunca giden düğüm sayısıdır. Yükseklik farkı hesaplanırken, her düğümün sol ve sağ alt ağaçlarının yükseklik farkı kontrol edilir.
- **Dengeli Yapı:** AVL ağacının temel özelliği, her ekleme veya silme işleminden sonra ağacın dengesini korumaktır. Bu dengeleme, bir dizi dönüşüm (rotasyon) ile yapılır.

AVL Ağacında Denge Faktörü

AVL ağacında her düğüm için denge faktörü şu şekilde tanımlanır:

$$BF = h_{sol} - h_{sağ}$$

Burada:

h_{sol} : düğümün **sol alt ağacının yüksekliği**

$h_{sağ}$: düğümün **sağ alt ağacının yüksekliği**

BF: denge faktörü

Bir AVL ağacında her düğüm için denge faktörü yalnızca şu değerlerden biri olmalıdır:

-1 → sağ alt ağaç bir seviye daha yüksek

0 → iki alt ağacın yüksekliği eşit

1 → sol alt ağaç bir seviye daha yüksek

Eğer denge faktörü **-1, 0, 1 dışına çıkarsa**, ağaç dengesiz hale gelir.

Örneğin:

sol yükseklik = 3

sağ yükseklik = 2

ise

$BF=3-2=1 \rightarrow$ Bu durumda düğüm dengededir.

Fakat

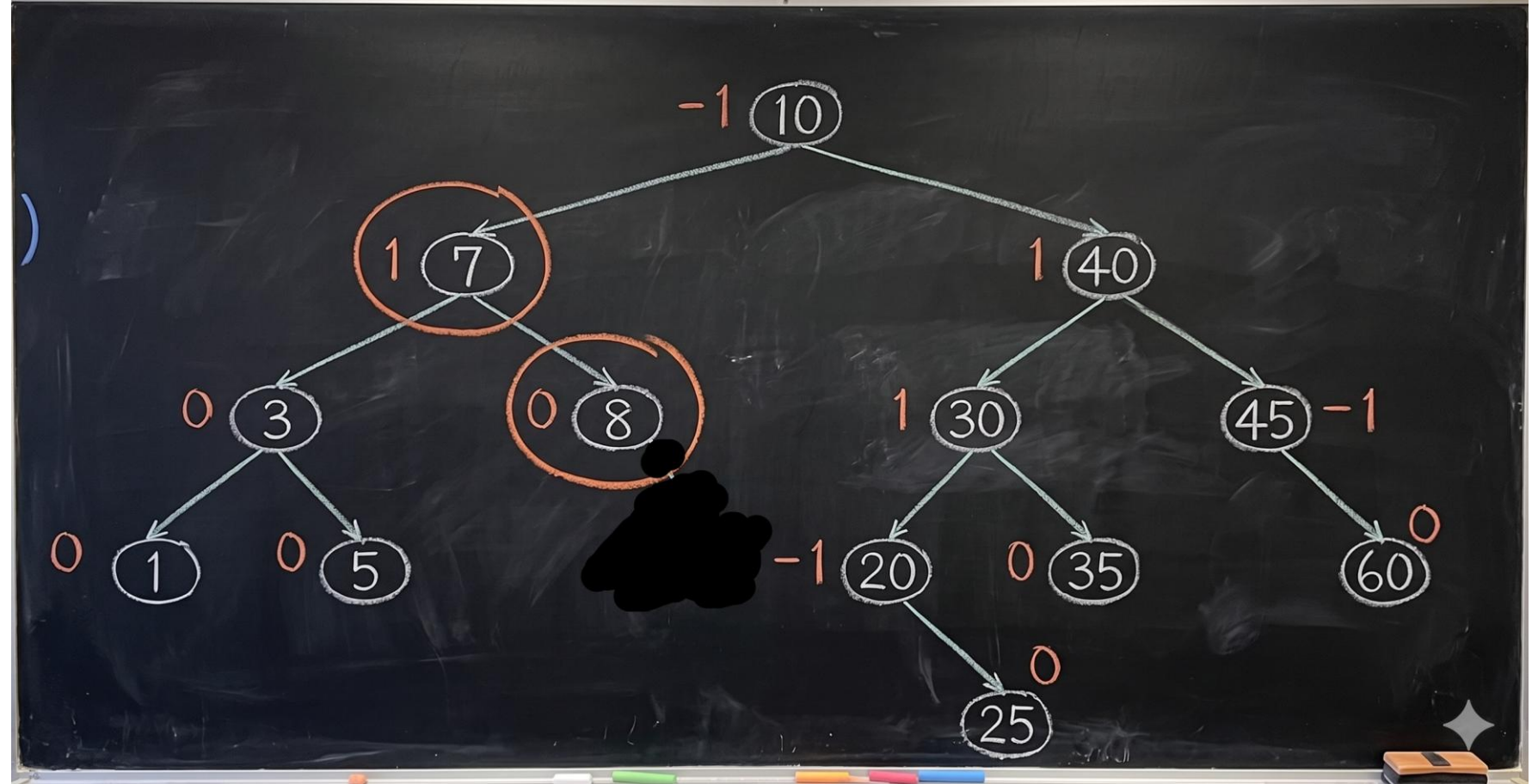
sol = 4

sağ = 2 ise

$BF=4-2=2$ olur ve AVL özelliği bozulur.

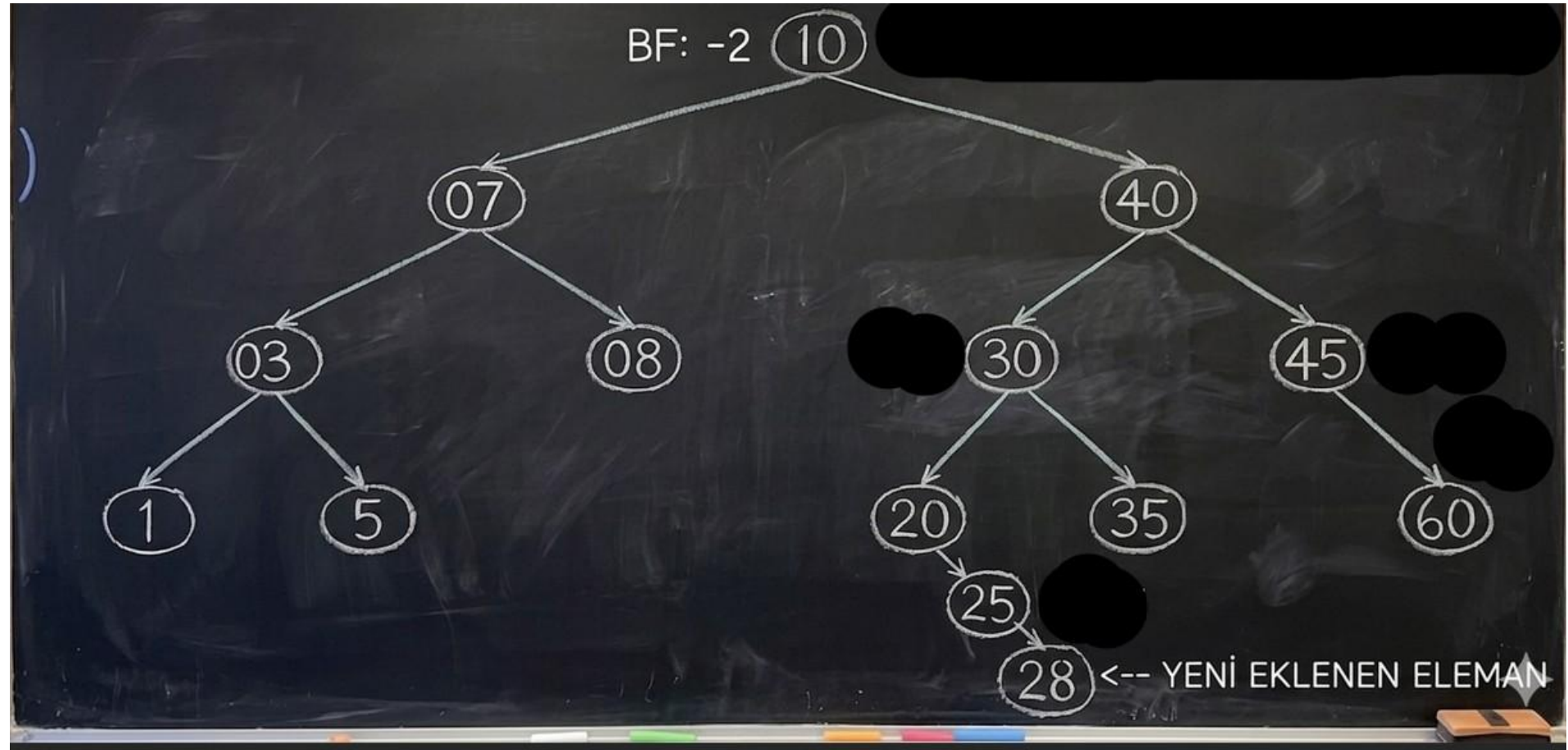
Bu durumda **rotasyon (sağa/sola dönüş)** uygulanarak denge yeniden sağlanır.

Tüm düğümlerin denge faktörleri -1, 0 veya 1 değerlerini koruyor. Hiçbir yerde 2 veya -2 gibi bir dengesizlik oluşmamış.



AVL Ağacına Eleman Ekleme

28
Elemanını
Ekledik ve
denge
bozuldu



AVL Ağaçlarında Denge ve Döndürme (Rotation)

Bir AVL ağacında denge, her düğüm için tanımlanan **denge faktörü (BF)** ile kontrol edilir. Denge faktörü, bir düğümün sol ve sağ alt ağaçlarının yükseklik farkıdır.

Bir düğüm için:

$|BF| \leq 1 \rightarrow$ ağaç dengelidir

$|BF| > 1 (\pm 2) \rightarrow$ ağaç dengesizdir ve **döndürme (rotation)** gerekir

Temel Döndürme Türleri

AVL ağaçlarında dengeyi sağlamak için 4 temel durum vardır

1. Sol-Sol (LL) Durumu

Koşul: Sol çocuğun soluna ekleme yapılır

Sonuç: Düğümün $BF = +2$ olur

Çözüm: Sağa döndürme (Right Rotation)

2. Sağ-Sağ (RR) Durumu

Koşul: Sağ çocuğun sağına ekleme yapılır

Sonuç: Düğümün $BF = -2$ olur

Çözüm: Sola döndürme (Left Rotation)

3. Sol-Sağ (LR) Durumu

Koşul: Sol çocuğun sağına ekleme yapılır

Çözüm: Çift döndürme

- Sol çocukta sola döndürme
- Ana düğümde sağa döndürme

4. Sağ-Sol (RL) Durumu

Koşul: Sağ çocuğun soluna ekleme yapılır

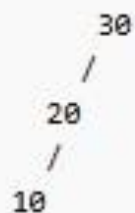
Çözüm: Çift döndürme

- Sağ çocukta sağa döndürme
- Ana düğümde sola döndürme

1) LL (Left-Left) – Sağa tek rotasyon

✗ Dengesiz durum

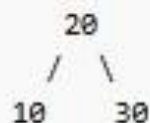
("solun solu ağır")



30 düğümünde $BF = +2$ (sol ağır)

20'nin soluna ekleme yapılmış → LL durumu

Sağ rotasyon (right rotation at 30)

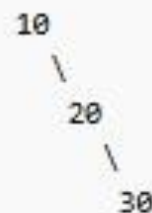


✓ Denge sağlandı

2) RR (Right-Right) – Sola tek rotasyon

✗ Dengesiz durum

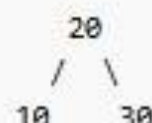
("sağın sağı ağır")



10 düğümünde $BF = -2$

sağın sağına ekleme → RR durumu

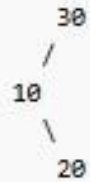
Sol rotasyon (left rotation at 10)



✓ Denge sağlandı

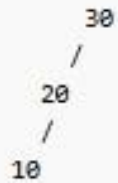
3) LR (Left-Right) – Çift rotasyon

✗ Dengesiz durum

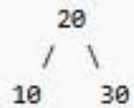


30 sol ağır (BF +2)
ama sol çocuğun SAĞINA ekleme yapılmış → LR

Adım 1: Sol rotasyon (10 üzerinde)



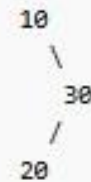
Adım 2: Sağ rotasyon (30 üzerinde)



✓ Denge sağlandı

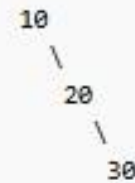
4) RL (Right-Left) – Çift rotasyon

✗ Dengesiz durum

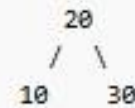


10 sağ ağır (BF -2)
sağ çocuğun SOLUNA ekleme → RL

Adım 1: Sağ rotasyon (30 üzerinde)

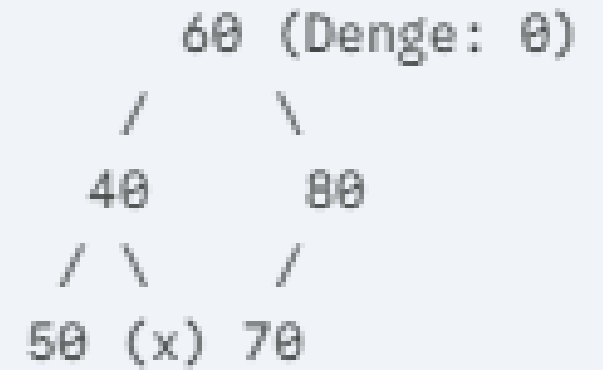
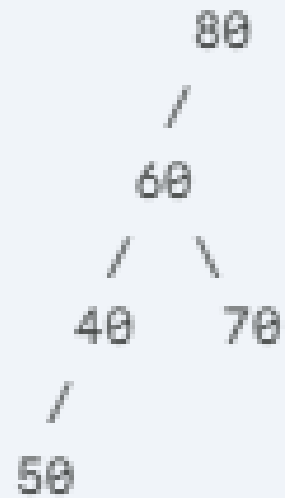
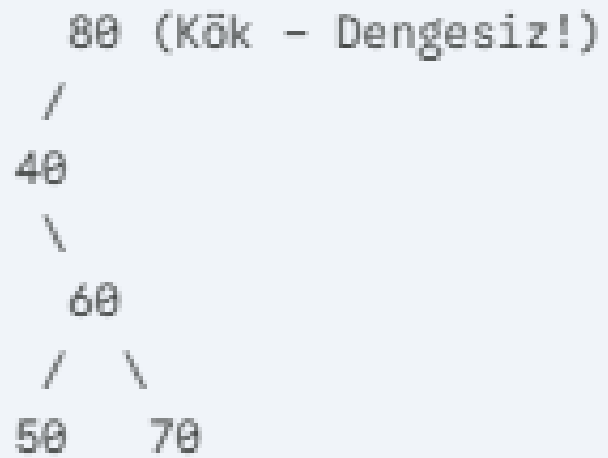


Adım 2: Sol rotasyon (10 üzerinde)



✓ Denge sağlandı

LEFT-RIGHT



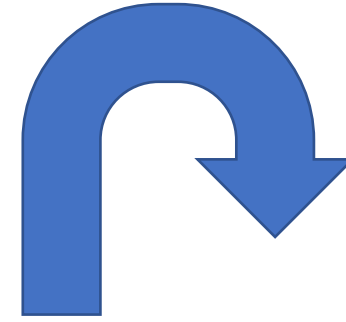
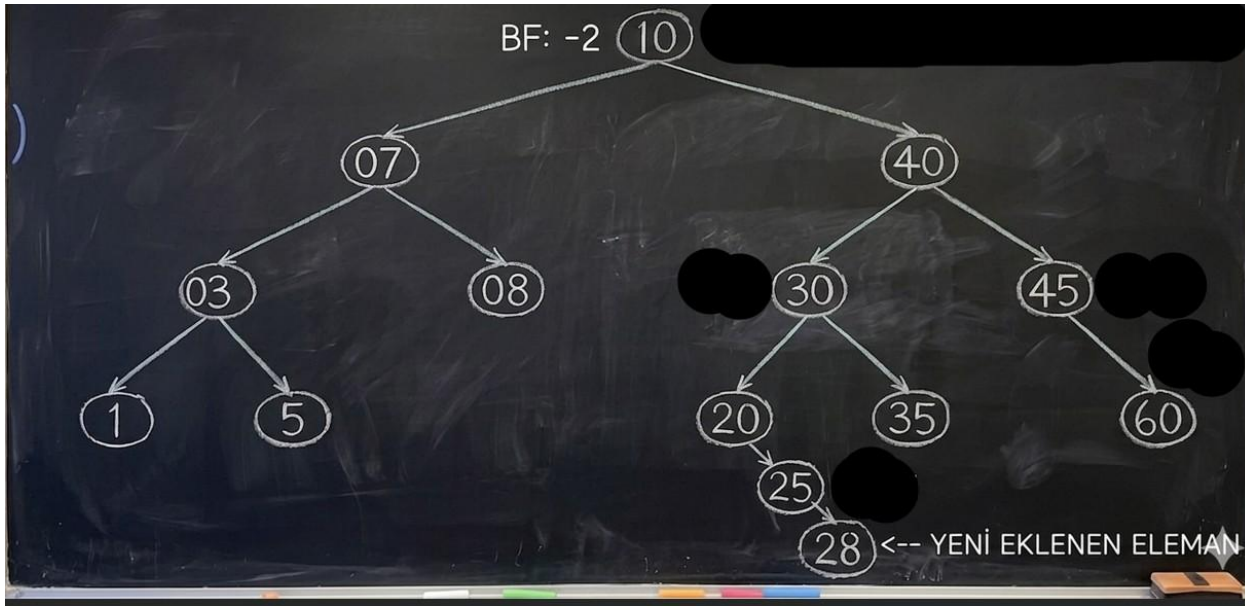
Durum	Tip	İşlem
Sol-sol	LL	Tek sağ rotasyon
Sağ-sağ	RR	Tek sol rotasyon
Sol-sağ	LR	Sol + Sağ
Sağ-sol	RL	Sağ + Sol

UYARI

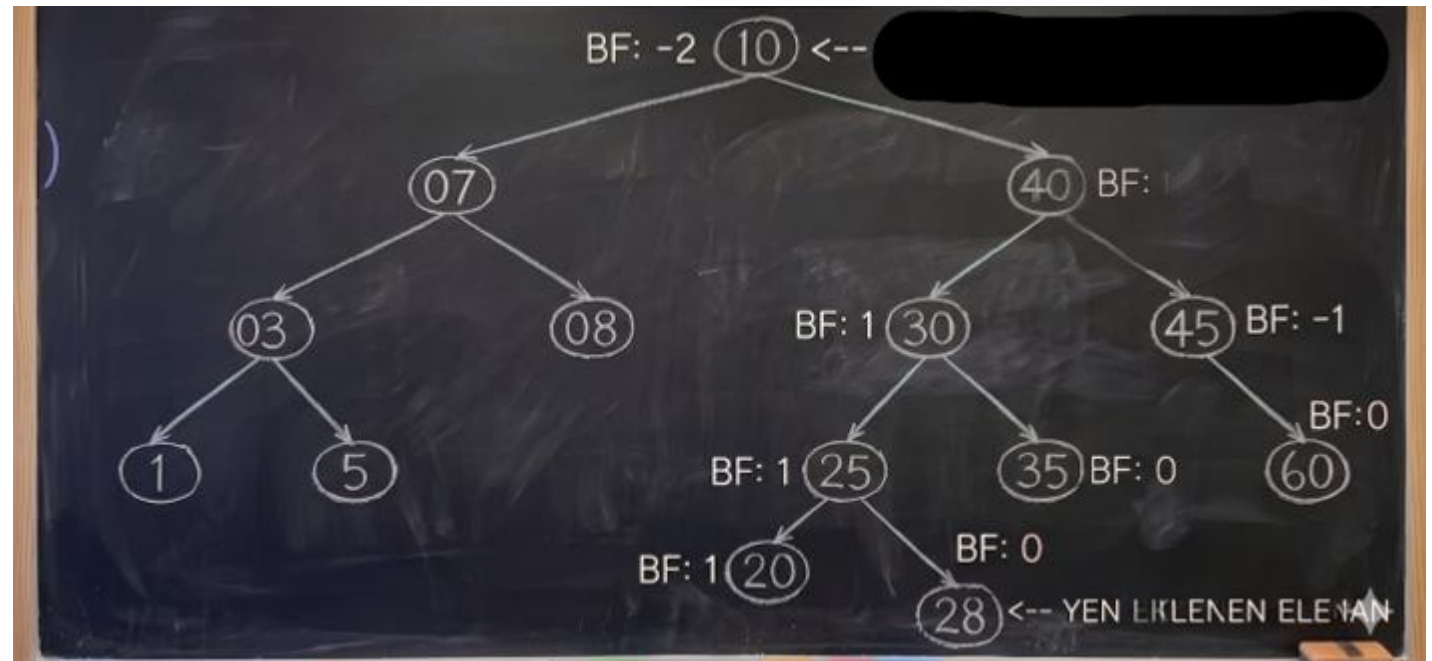
Aynı yerde tekrar LR/RL yapılmaz

Her işlem sonrası ağaç tekrar kontrol edilir

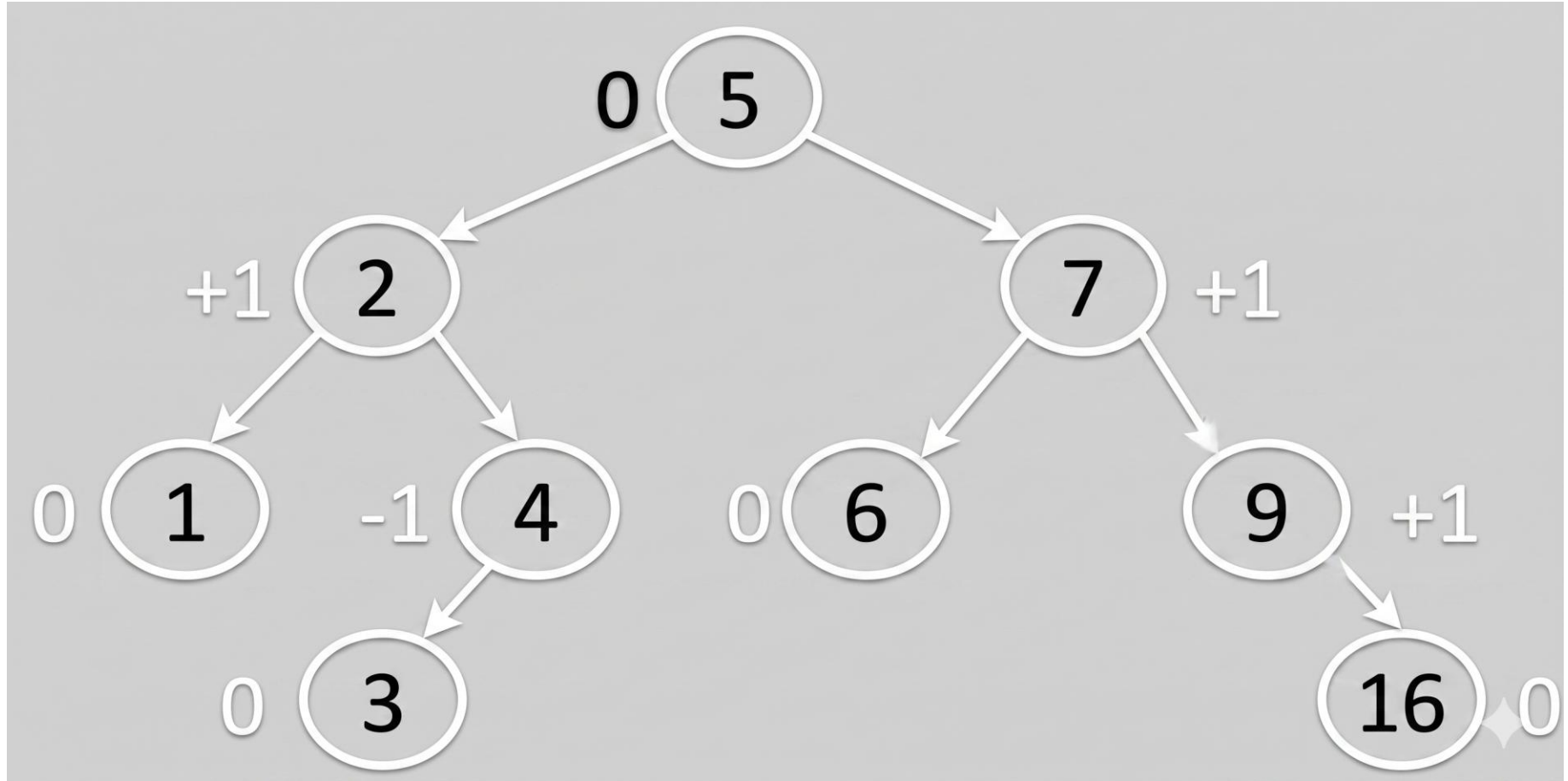
Farklı düğümlerde yeniden LR/RL olabilir

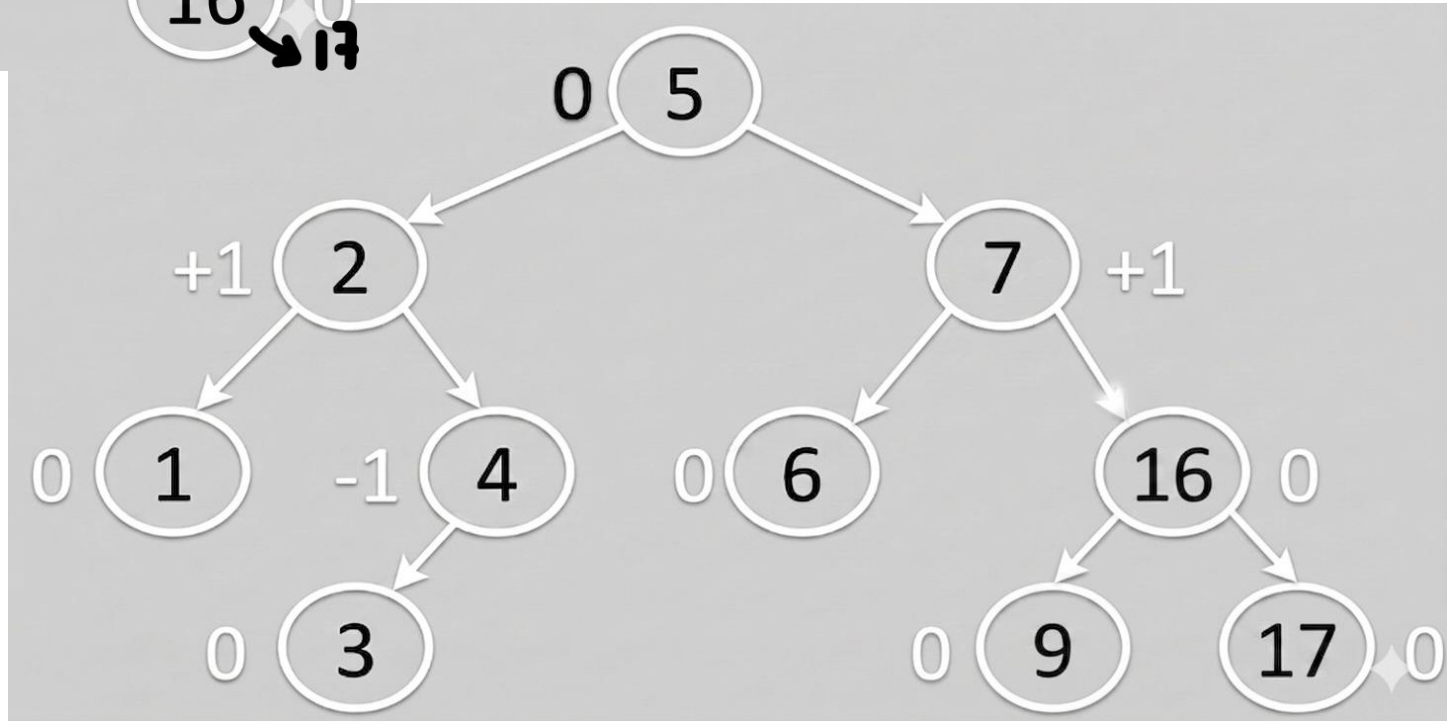
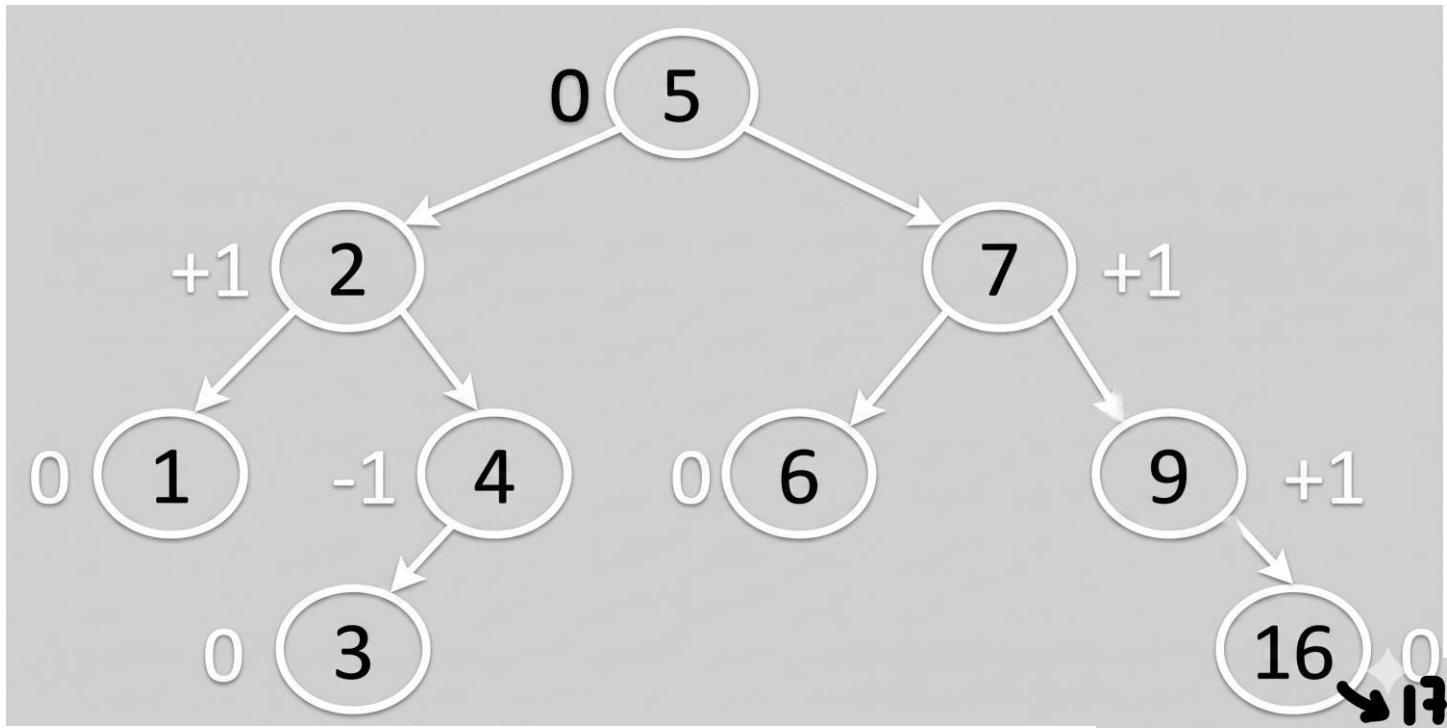


**Ekleme yapılan
sorumuza istinaden
Tekli Sola Döndürme
(Left Rotation) işlemi
uyguladık ve ağaç
dengelendi**



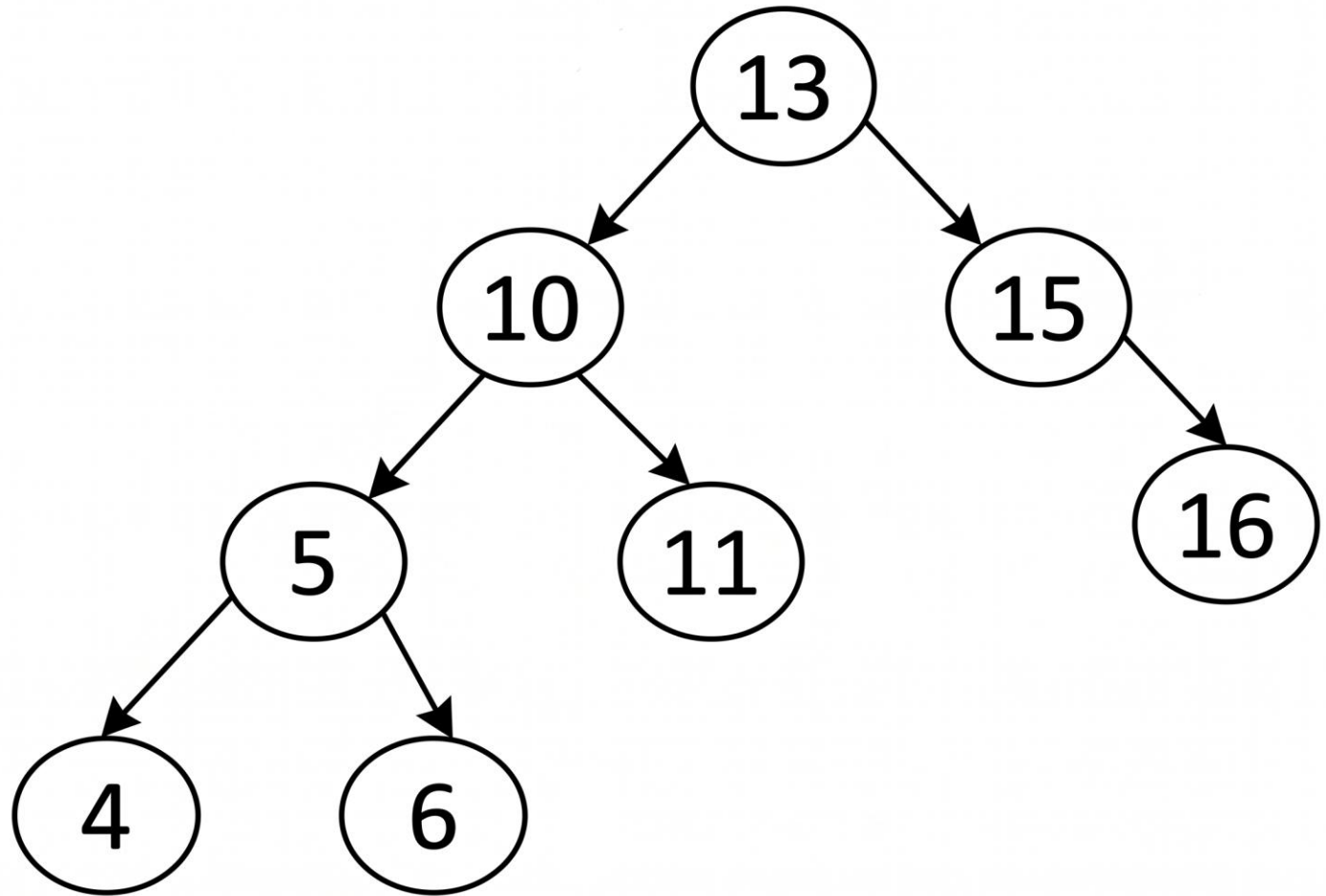
Yandaki ağaca
17 anahtarı
eklenmek
isteniyor.
Ekleme
işleminde
sonra ağacın
yapısı nasıl
olur?

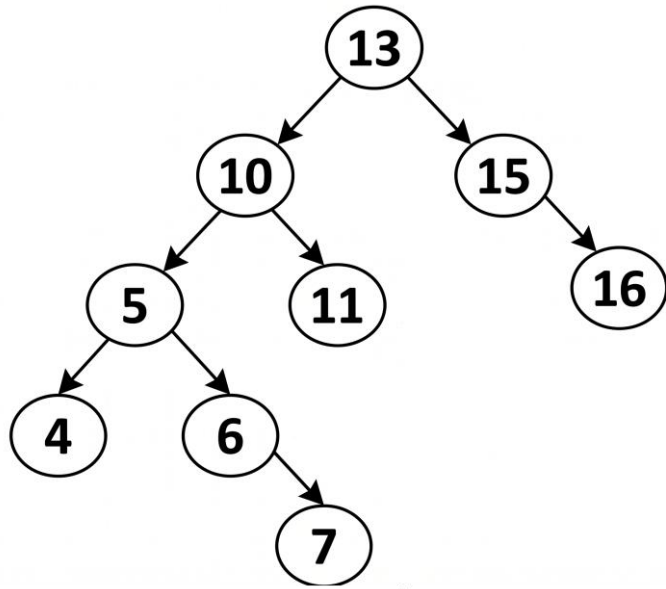




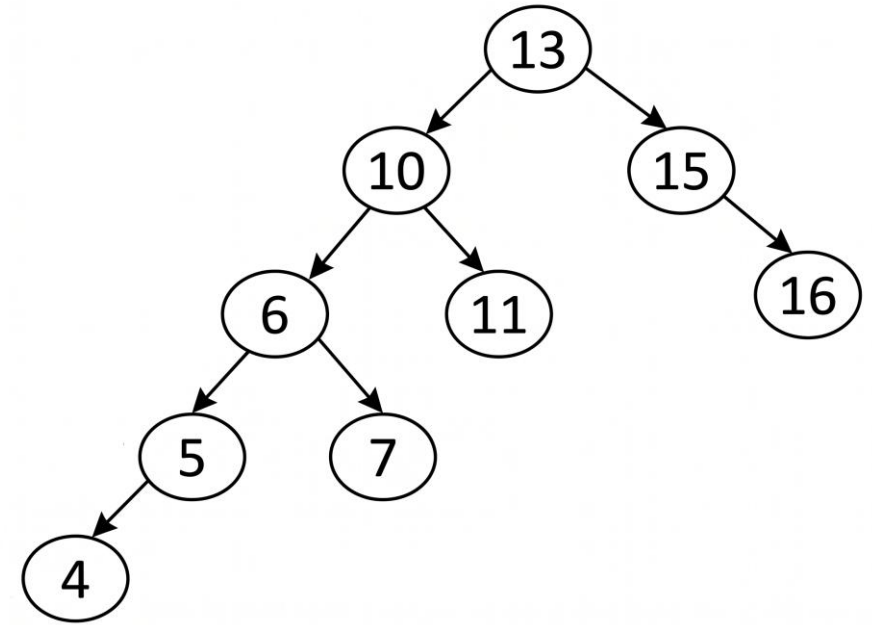
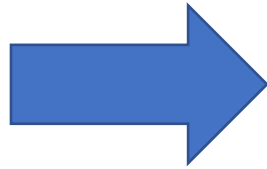
Döndürme sağlandı----->>>>>>>

Yandaki ağaca 7 anahtarı eklenmek isteniyor. Ekleme işleminden sonra ağacın yapısı nasıl olur?

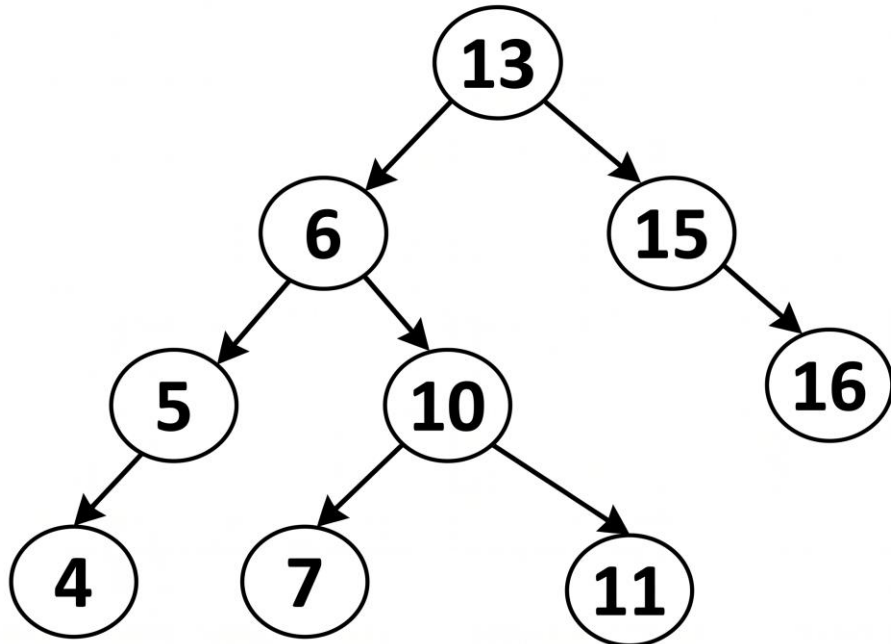
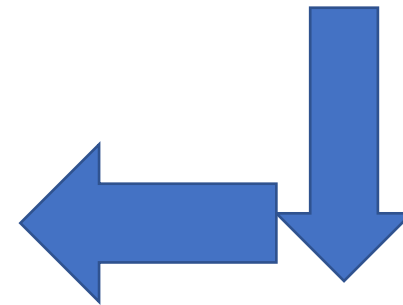




5 düğümü pivot
olmak üzere sola
döndürme



10 düğümünü pivot noktası olarak sağa döndürme



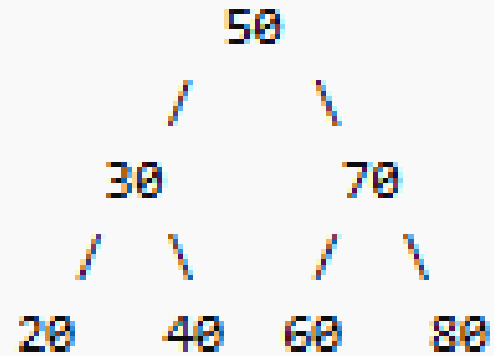
AVL Ağacından Eleman Silme

AVL ağaçlarında silme işlemi sonrasında ağaçta dengesizlik oluşabilir. Bu durumda, ağacın AVL özelliğini koruyabilmesi için gerekli dengeleme işlemleri uygulanır. Silme işlemi sonrasında üç farklı durum ortaya çıkabilir:

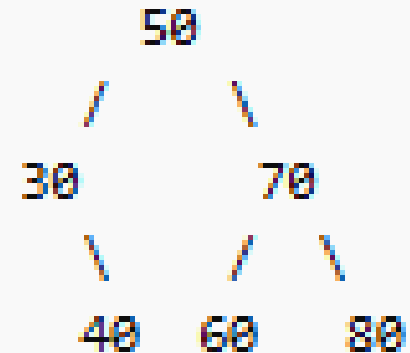
- (1) silme işlemi ağacın dengesini bozmaz,
- (2) dengesizlik tek bir döndürme ile giderilebilir,
- (3) dengesizlik çift döndürme gerektirir.

1) Dengesizliğe neden olmayan silme

Silme öncesi:

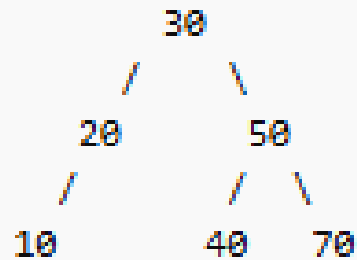


👉 20 silinsin:

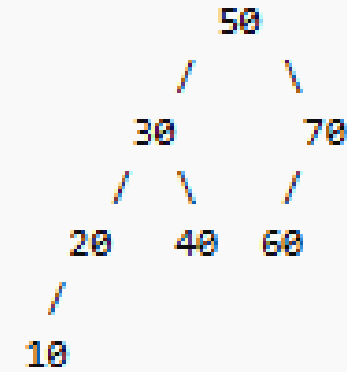


2) Tek döndürme ile dengelenen durum

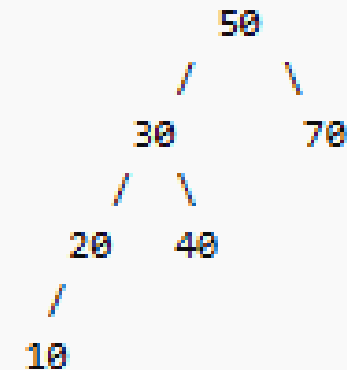
Çözüm: SAĞ ROTASYON (50 üzerinde)



Başlangıç (dengeli AVL)



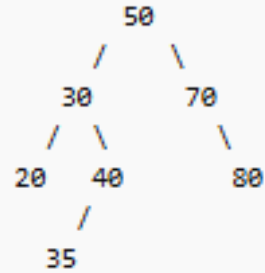
60 silinsin



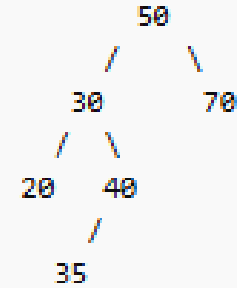
3) Çift döndürme ile dengelenen durum

ÇİFT DÖNDÜRME (LR)

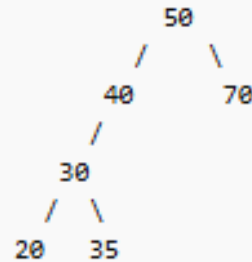
Silme öncesi



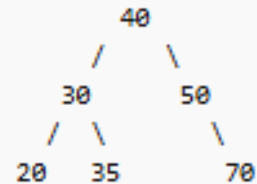
80 silinsin



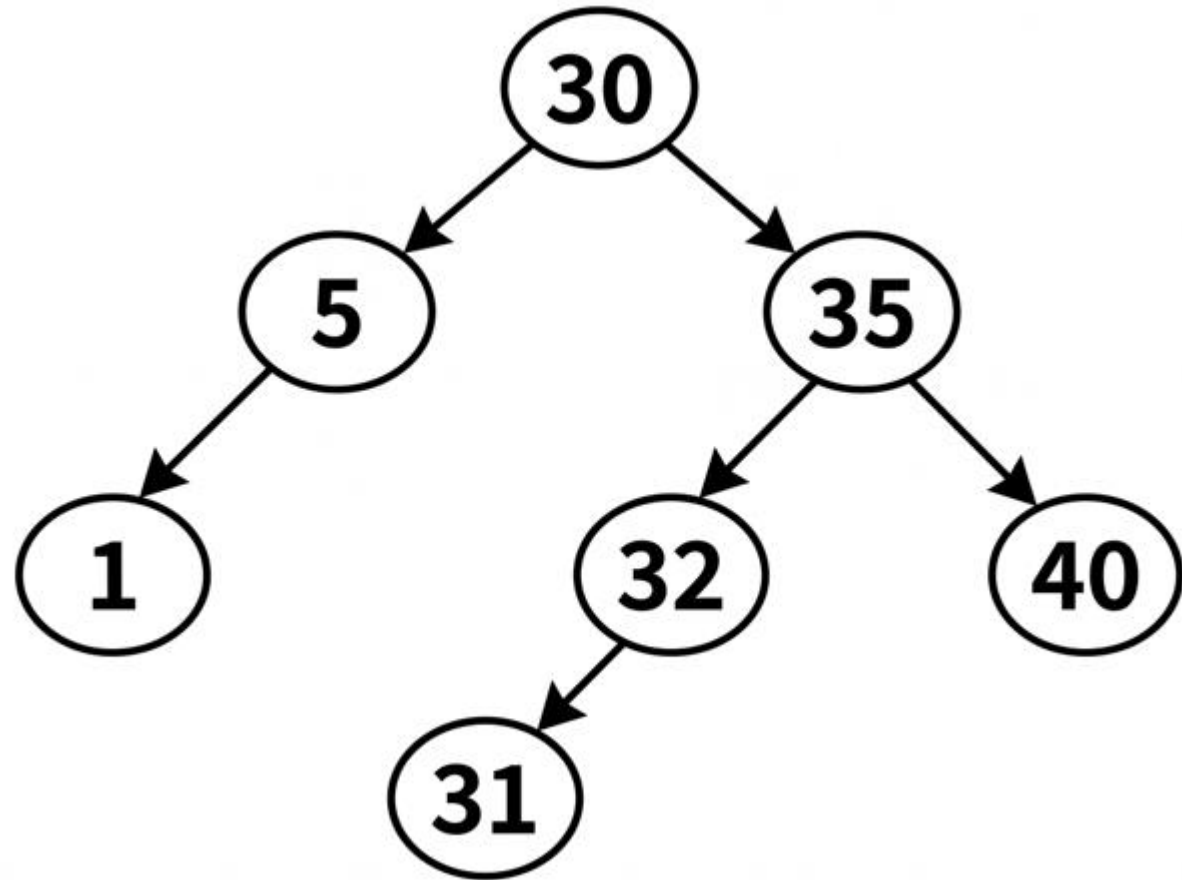
ADIM 1: 30 üzerinde SOL ROTASYON

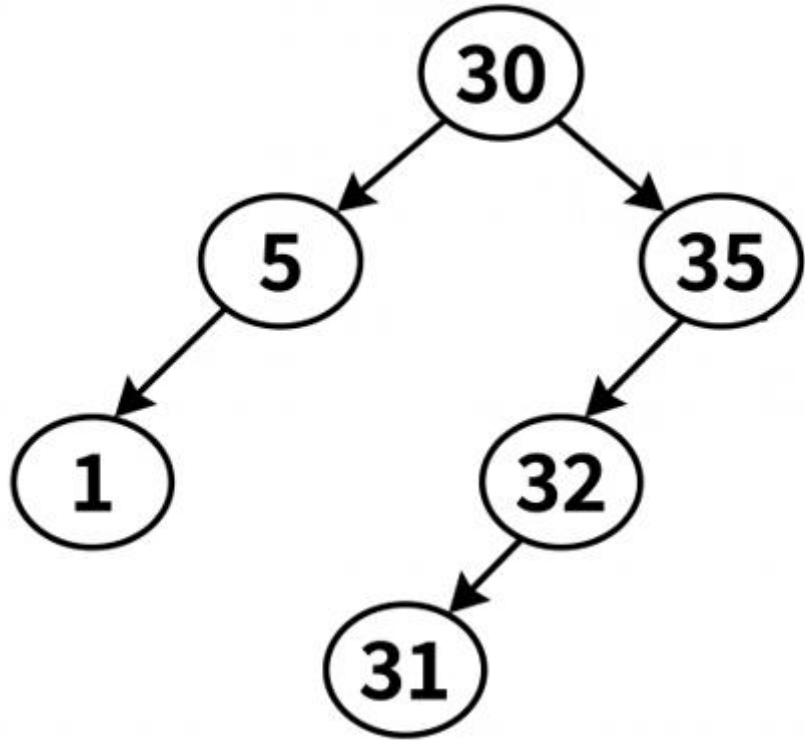


ADIM 2: 50 üzerinde SAĞ ROTASYON

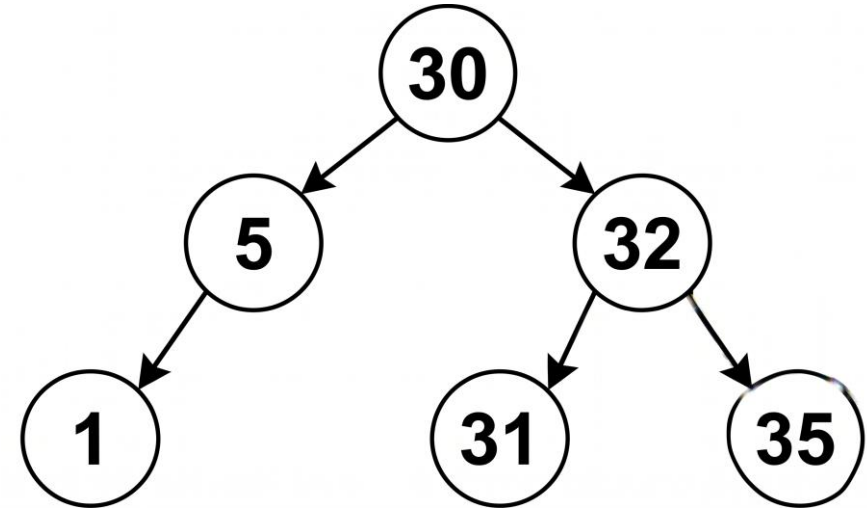


Yandaki ağaca 40 anahtarı
silinmek isteniyor. Silme
işleminin sonra ağacın
yapısı nasıl olur?





40 anahtarı silindi



35 düğümü pivot olmak üzere sağa döndürüldü

Özellik	Normal BST (Senin Kodun)	AVL Ağacı
Dolaşma	Aynı	Aynı
Ekleme	Sadece uygun boşluğa koyar.	Koyar + Dengeyi kontrol eder + Gerekirse döndürür.
Silme	Sadece düğümü çıkarır.	Çıkarır + Dengeyi kontrol eder + Gerekirse döndürür.
Performans	Şansına bağlı (Kötü dizilirse yavaşlar).	Her zaman çok hızlıdır ($O(\log n)$).

Kümeleme Ağacı

Kümeleme ağacı (Heap Tree), ağaç oluşturulurken düğümlerin belirli bir öncelik kuralına göre yerleştirildiği, genellikle tamamlanmış (complete) ikili ağaç yapısına sahip bir veri yapısıdır.

Bu yapıda, eğer “maksimum kümeleme” (max-heap) kullanılıyorsa kök düğümde en büyük değer bulunur ve aşağıya doğru inildikçe düğümlerin değerleri küçülür. “Minimum kümeleme” (min-heap) durumunda ise kökte en küçük değer yer alır ve aşağıya doğru değerler büyür. Böylece veriler ya büyükten küçüğe ya da küçükten büyüğe doğru hiyerarşik olarak düzenlenmiş olur. Aksi belirtilmediği sürece kümeleme ağaçları sayısal verilerde öncelik sırasına göre (max-heap veya min-heap), sözcüklerde ise genellikle alfabetik sıraya göre düzenlenir. Ayrıca, kümeleme ağacı denildiğinde aksi belirtilmedikçe ikili (binary) heap yapısı anlaşılır.

Heap Nedir?

Özetle heap, aslında **Tam İkili Ağaç (Complete Binary Tree)** yapısında olan özel bir veri yapısıdır. İki temel kuralı vardır:

1.Şekil Özelliği: Ağaç, son seviye hariç tamamen doludur ve son seviyedeki düğümler mümkün olduğunca soldadır.

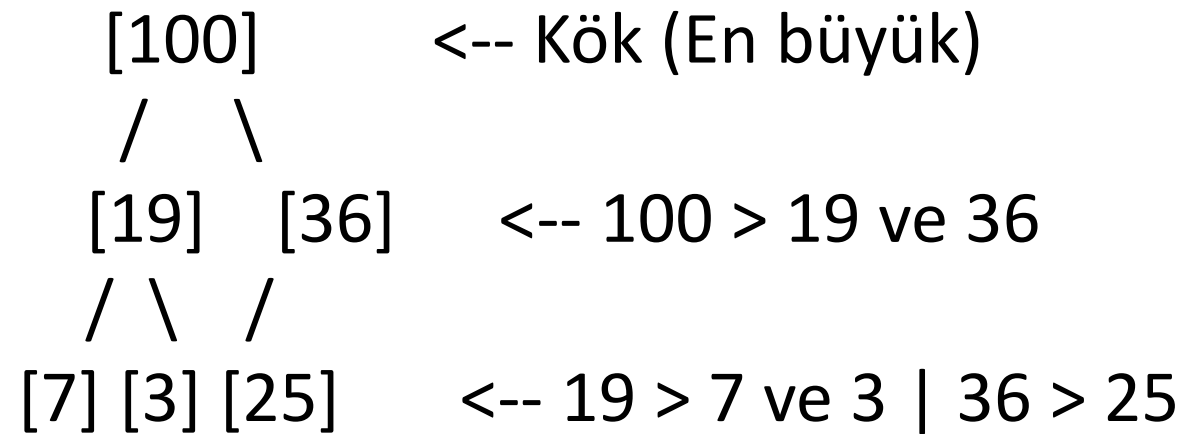
2.Yığın Özelliği (Heap Property): *

Max-heap: Her ebeveyn $\geq$ çocukları $\rightarrow$ kökte en büyük değer

Min-heap: Her ebeveyn $\leq$ çocukları $\rightarrow$ kökte en küçük değer

1. Heap Özelliği Gösteren Ağaç

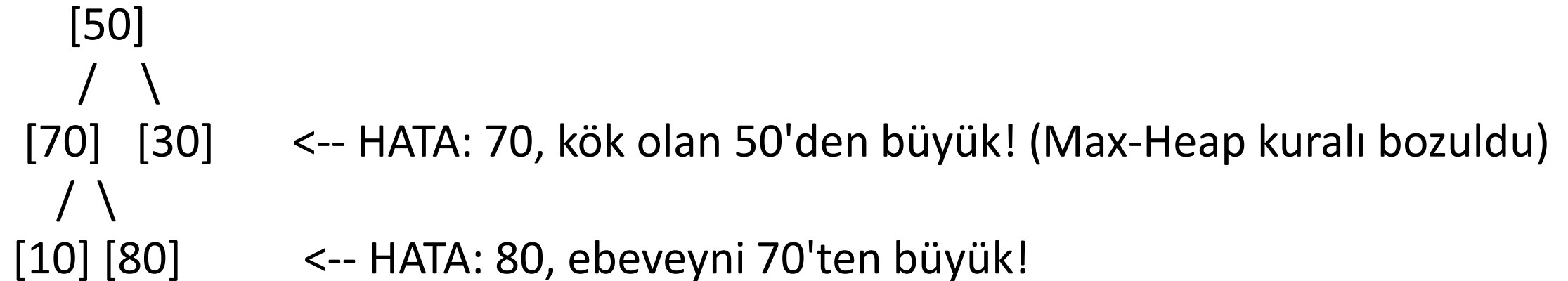
Bu ağaçta her ebeveyn, çocuklarından büyüktür ve "tam ikili ağaç" kuralına uyar.



Neden Heap? Hem tam bir ağaç hem de her ebeveyn çocuklarından daha büyük.

2. Heap Özelliği Göstermeyen Ağaç

Bu ağaçta kurallar ihlal edilmiştir.



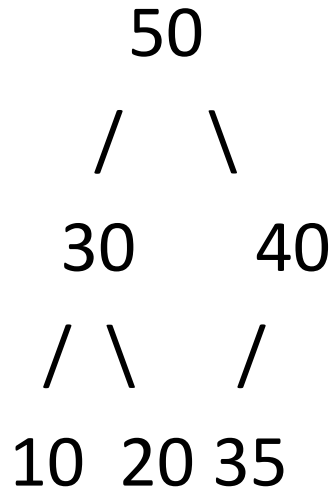
Neden Heap Değil? Ebeveynlerin çocuklarından büyük olma zorunluluğu (Max-Heap için) birden fazla noktada bozulmuştur.

Heap Örneği

Dizi gösterimi (heap'in array hali):

[50, 30, 40, 10, 20, 35]

Ağaç gösterimi:



❶ Yapısal Kontrol (Tam İkili Ağaç mı?)

En alt seviye hariç tüm seviyeler dolu

En alt seviye soldan sağa doldurulmuş

Arada boş düğüm yok

Tam ikili ağaç şartı sağlanıyor

Bu yapı:

Tam ikili ağaçtır

Her ebeveyn $\geq$ çocuk

Heap özelliği sağlanır

❷ Heap Kuralı Kontrolü (Max-Heap)

Her ebeveyn $\geq$ çocuklar mı, tek tek bakalım:

$50 \geq 30$ ve 40

$30 \geq 10$ ve 20

$40 \geq 35$

Tüm düğümler heap kuralını sağlıyor

İşlevsel Ayırıştırma: Kim, Ne Yapar?

Heapify() (Yığınlaştırma)

Sistemin “Bakım Uzmanı”

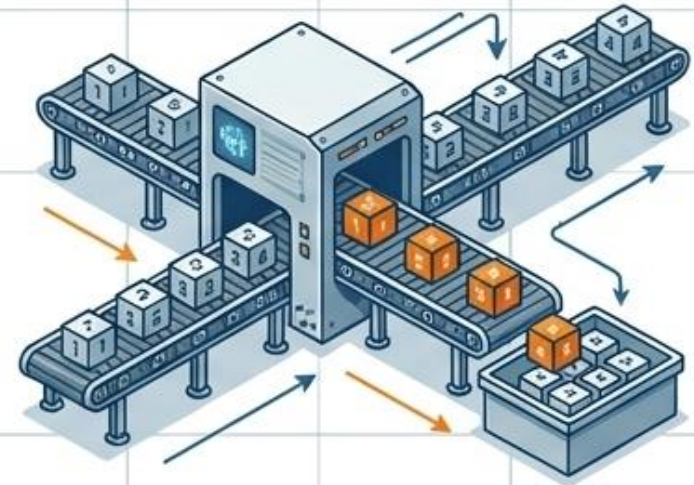
Belirli bir düğümden başlayarak, o düğümün ve altındaki ağacın Max-Heap kuralına uymasını sağlar. Bozulan kuralı onarır, sistemi dengeye oturtur.



Heapsort (Yığın Sıralaması)

Sistemin “Fabrika Yöneticisi”

Diziyi sıralamak için **Heapify** mekanizmasını defalarca kullanan ana üretim hattıdır. Kökten en büyük elemanı alır, dizinin sonuna kilitler ve sistemi tekrar dengeye girmesi için **Heapify**'a gönderir.



HEAPLEŞTİRME (BUILD MAX-HEAP)

Heap Yapılanması: Diziye Tek Tek Eleman Ekleme Örneği

ÖRNEK 1:

Varsayalım ki verilerimizin sırası yandaki gibi verilsin: 15, 10, 20, 8, 25

Başlangıçta heap boş.

1 15 elemanını ekle

Dizi:

[15]

Ağaç:

15

2 10 elemanını ekle

Diziye sona eklenir:

[15, 10]

Ağaç:

15

/

10

$15 \geq 10 \rightarrow$ heap özelliği bozulmaz

3 20 elemanını ekle

Diziye sona eklenir:

[15, 10, 20]

Ağaç:

15

/ \

10 20

$20 > 15 \rightarrow$ heap bozuldu

Yukarı taşıma (heapify-up) yapılır:

Yer değiştir:

[20, 10, 15]

Yeni ağaç $\rightarrow$

20

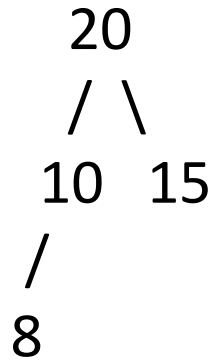
/ \

10 15

4 8 elemanını ekle

Dizi:[20, 10, 15, 8]

Ağaç:



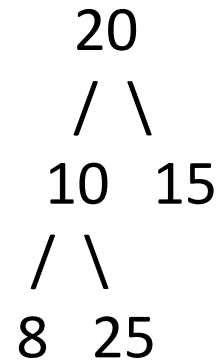
$8 < 10 \rightarrow$ sorun yok

Değişiklik yok

5 25 elemanını ekle

Dizi:[20, 10, 15, 8, 25]

Ağaç:



Yukarı Taşıma Adımları

1. adım:

$25 > 10 \rightarrow$ yer değiştir

[20, 25, 15, 8, 10]

2. adım:

$25 > 20 \rightarrow$ tekrar yer değiştir

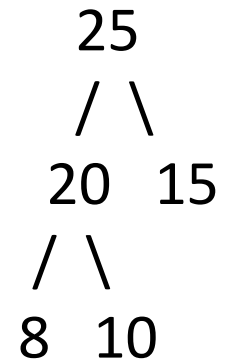
[25, 20, 15, 8, 10]

Son Heap (Max-Heap)

Dizi gösterimi:

[25, 20, 15, 8, 10]

Ağaç gösterimi:



Tüm düğümler için:

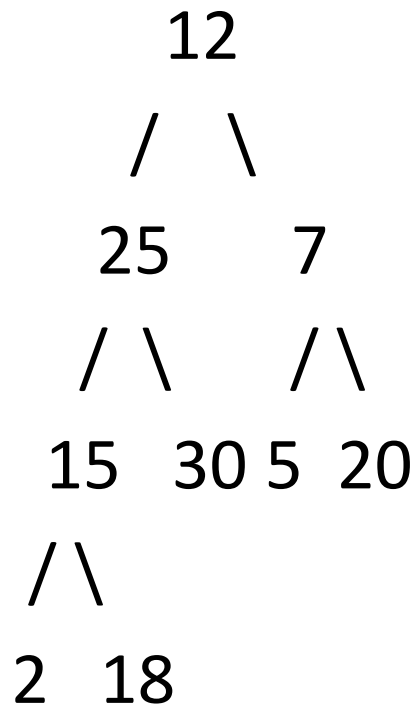
Ebeveyn $\geq$ çocuk

Ağaç tam ikili

Heap yapılanması

tamamlandı

BAŞLANGIÇ: Heap Özelliği Olmayan Kompleks Ağaç



Dizi Gösterimi (seviye sıralı – level order)

[12, 25, 7, 15, 30, 5, 20, 2, 18]

Bu dizi **heap değildir**, çünkü:

$12 < 25$

$25 < 30$

$7 < 20$

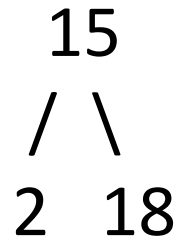
Yani **heap özelliği bozuk**.

HEAPLEŞTİRME (BUILD MAX-HEAP)

Heap Sort'ta kullanılan yöntem:
Bottom-up heapify (aşağıdan yukarı)

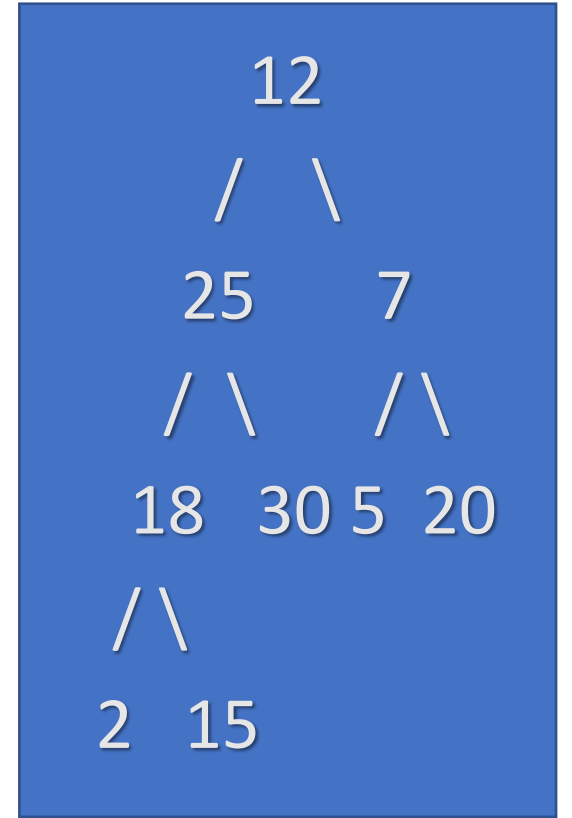
Son iç düğümden başlarız.

Adım 1 – 15 düğümü (çocukları: 2, 18)

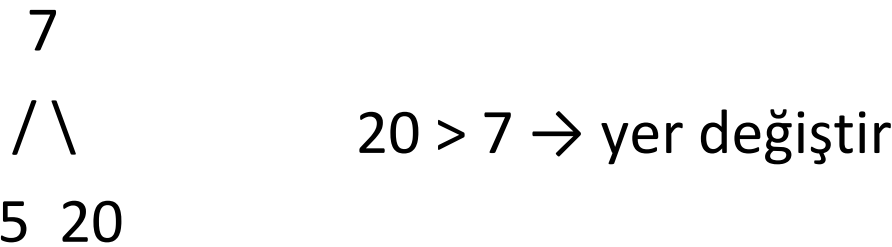


$18 > 15 \rightarrow$ yer değiştir

[12, 25, 7, 18, 30, 5, 20, 2, 15]



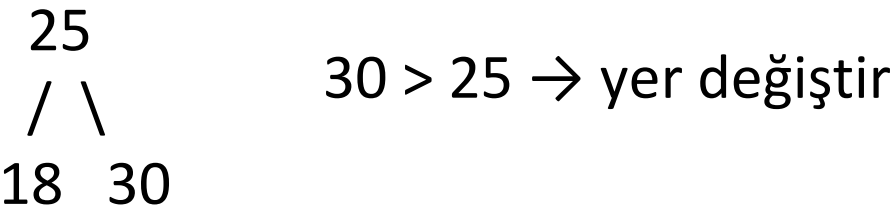
Adım 2 – 7 düğümü (çocukları: 5, 20)



Dizi:

[12, 25, 20, 18, 30, 5, 7, 2, 15]

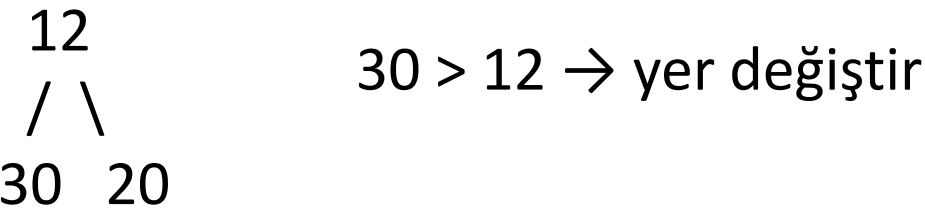
Adım 3 – 25 düğümü (çocukları: 18, 30)



Dizi:

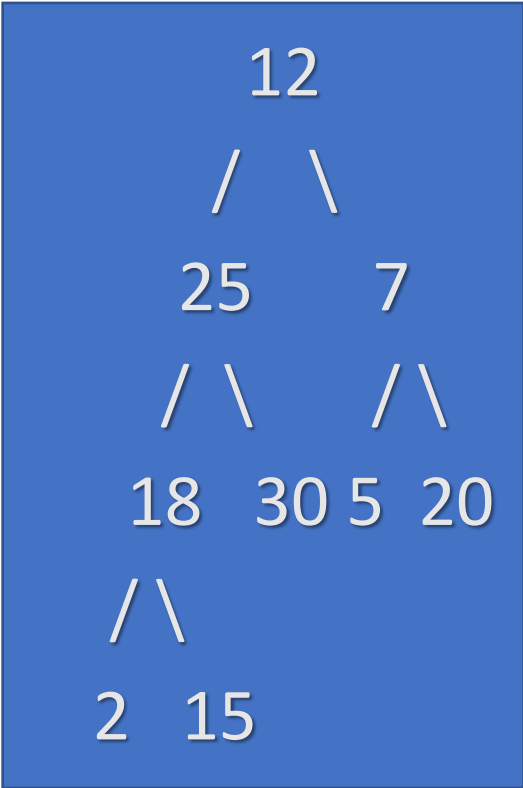
[12, 30, 20, 18, 25, 5, 7, 2, 15]

Adım 4 – 12 düğümü (çocukları: 30, 20)



Dizi:

[30, 12, 20, 18, 25, 5, 7, 2, 15]



Bir önceki sayfadaki adımların son hali

Aşağı doğru kontrol devam eder:

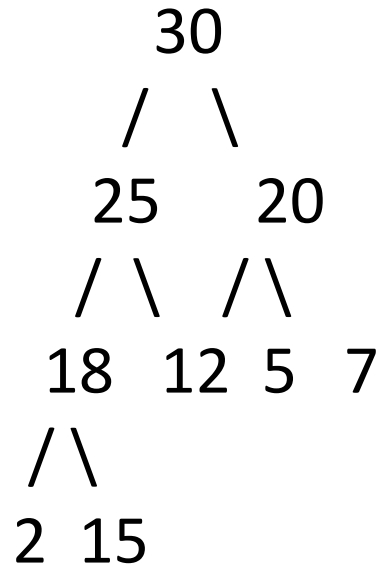
30, 12, 20, 18, 25, 5, 7, 2, 15

$12 < 25 \rightarrow$ yer değiştir.

[30, 25, 20, 18, 12, 5, 7, 2, 15]

SONUÇ: MAX-HEAP OLUŞMUŞTUR

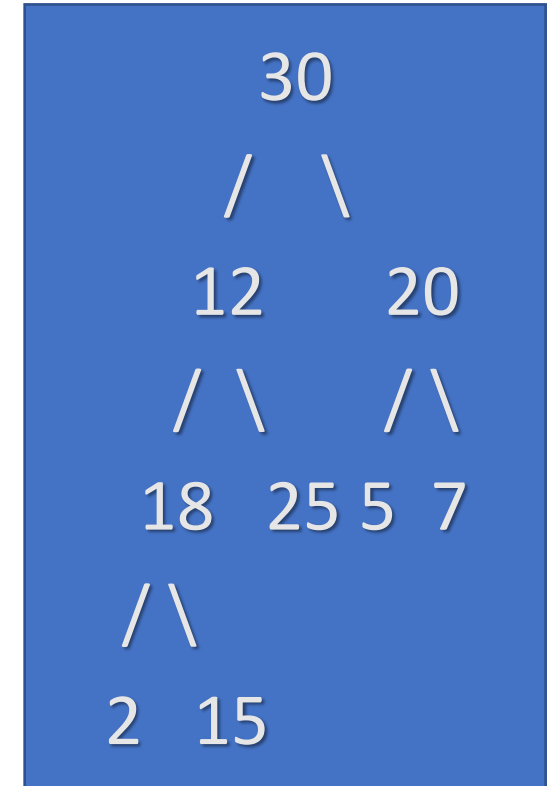
[30, 25, 20, 18, 12, 5, 7, 2, 15]



Tam ikili ağaç

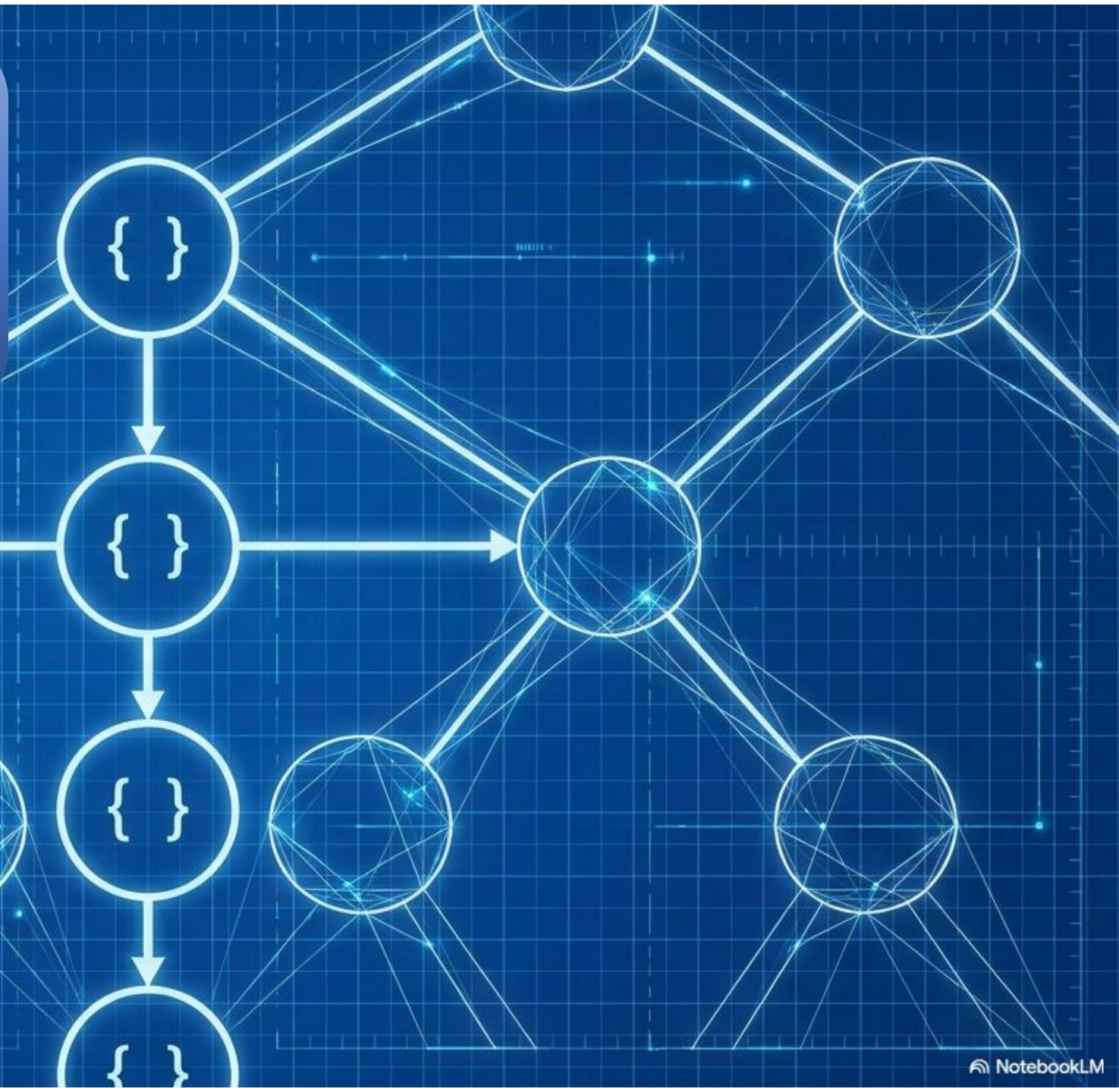
Her ebeveyn $\geq$ çocuk

MAX-HEAP elde edildi



Bir önceki sayfadaki adımların son hali

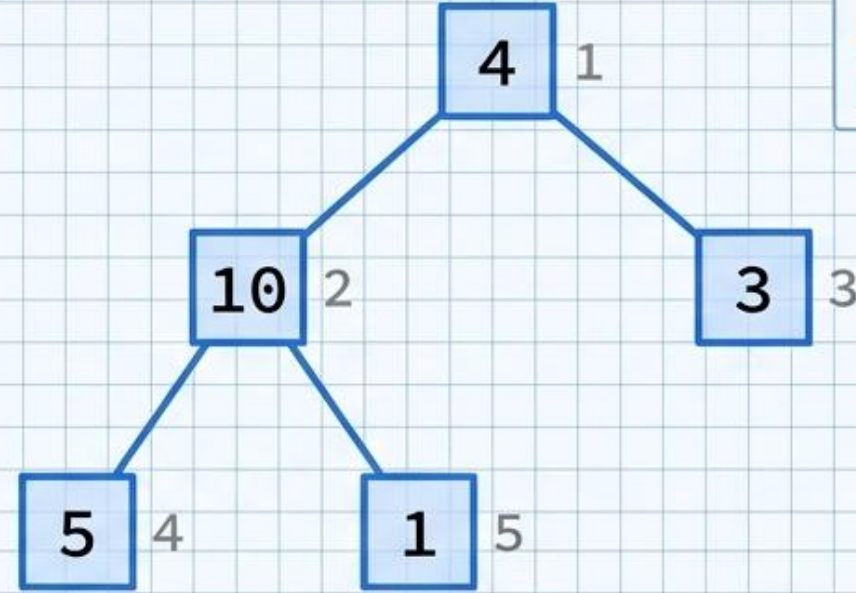
Baştan Sona Bir Örnek Yapalım



Dinamik Süreç: Başlangıç Durumu

Verilen rastgele dizi: $A = [4, 10, 3, 5, 1]$

Amaç: Önce bu kaosu bir Max-Heap'e dönüştürecek (Build-Heap), ardından Heapsort ile küçükten büyüğe sıralayacağız.



- İşlem Bekliyor
- Odak/Değişim
- Kilitli/Sıralanmış



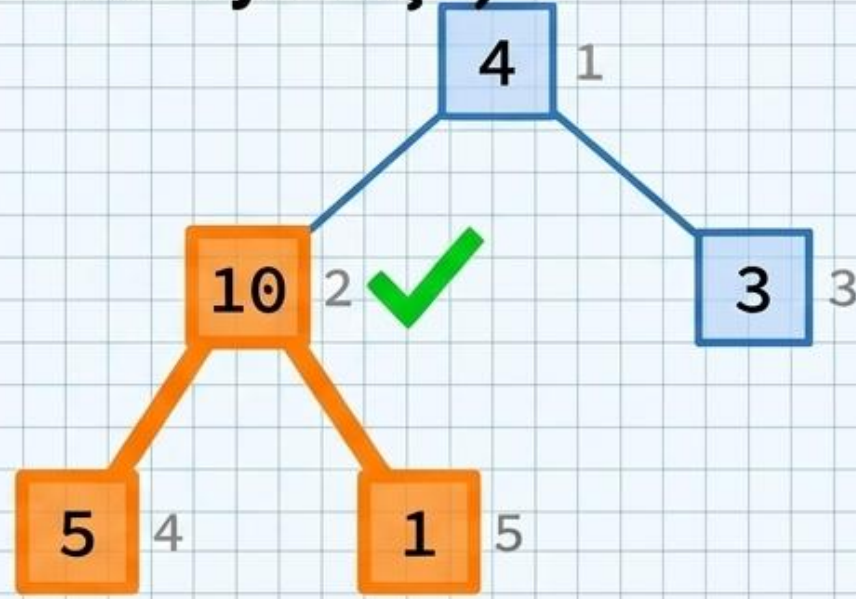
Aşama 1: Build-Heap (Aşağıdan Yukarıya İnşa)

İşlem, yaprak (leaf) olmayan ilk düğümden, yani $n/2$ indeksinden başlar. ($5/2 = 2$. indeks).

Odak noktası: $A[2] = 10$.

Çocukları: 5 ve 1.

Durum: 10, zaten 5 ve 1'den büyük olduğu için Max-Heap kuralı ihlal edilmiyor.

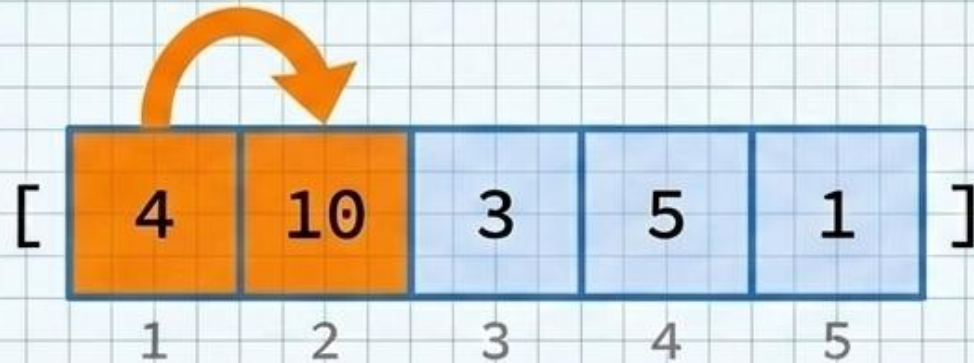
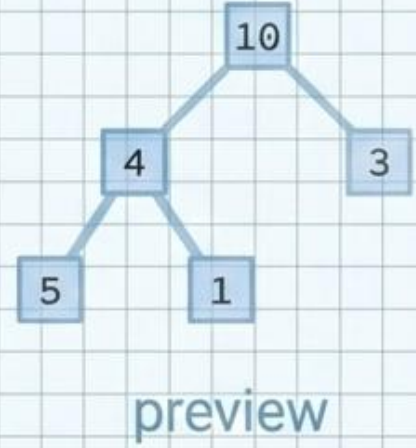
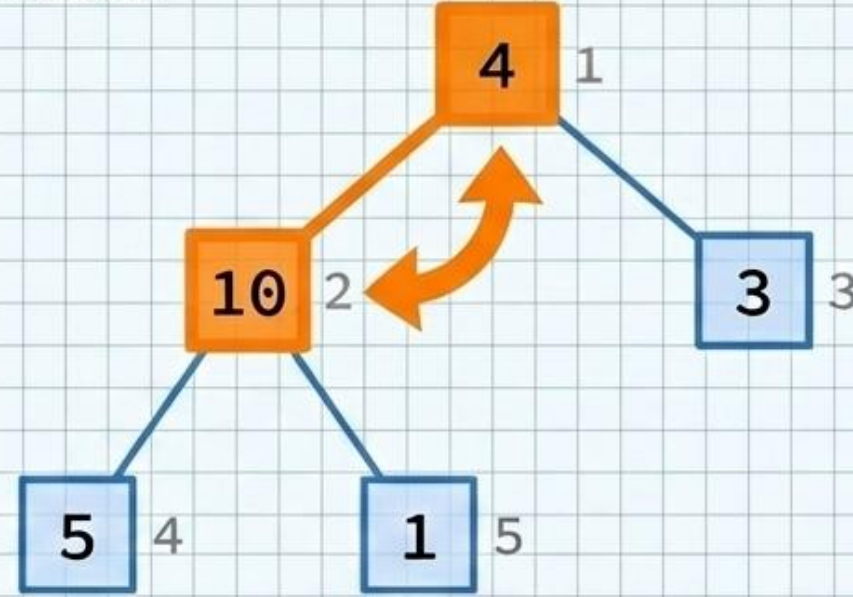


Aşama 2: Kök Düğümün İncelenmesi

İndeks 1'e ($A[1] = 4$) geçiyoruz.
Çocukları: 10 ve 3.

Durum: 4, 10'dan küçük! Max-Heap kuralı bozuldu.

Eylem: 4 ile en büyük çocuk olan 10 yer değiştirir (Swap).

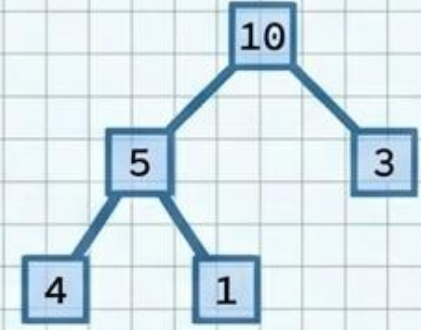
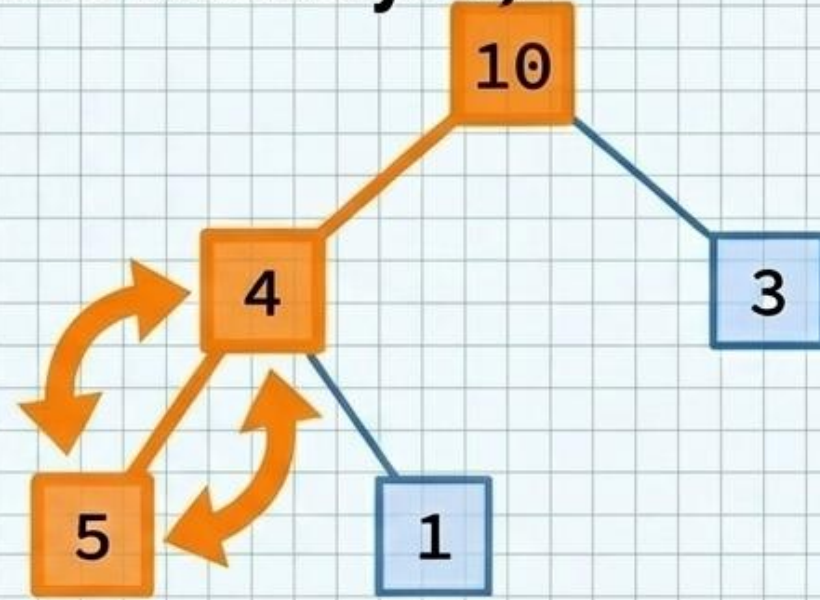


Aşama 3: Max-Heapify (Zincirleme Reaksiyon)

4 aşağı indi ancak altındaki yeni çocuklarıyla (5 ve 1) kuralı hala ihlal ediyor mu? Evet, 4, 5'ten küçük!

Heapify devreye girer ve hatayı aşağıya kadar izler: 4 ile 5 yer değiştirir.

Sonuç: Kusursuz bir Max-Heap elde edildi!

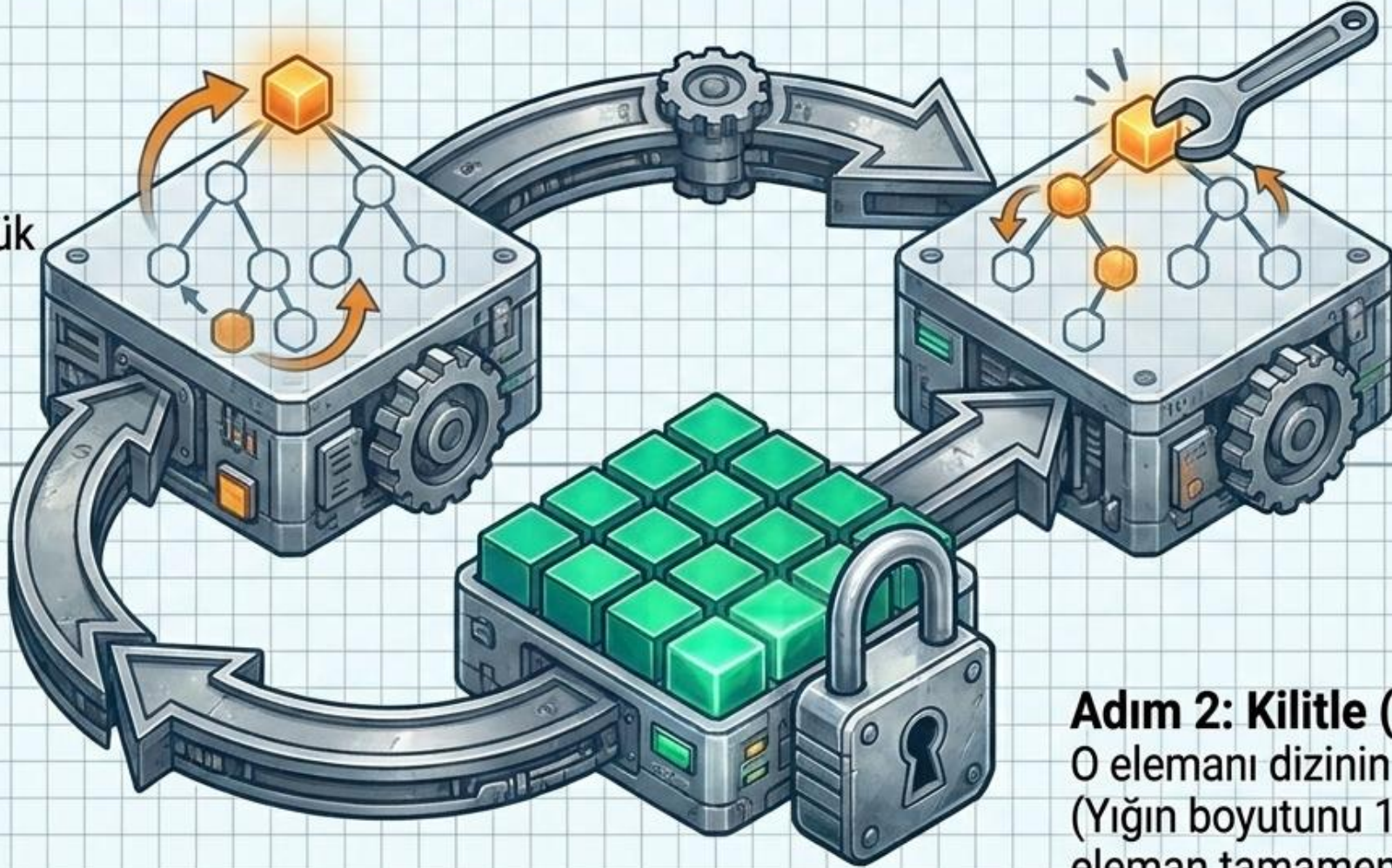


Heapsort'un Anatomisi: İşlem Döngüsü

Max-Heap hazır olduğuna göre Heapsort işlemine başlayabiliriz. Heapsort basit bir 3 adımlı motor gibi çalışır:

Adım 1: Çıkar (Extract)

En üstteki kök düğümü (en büyük sayıyı) al, dizinin en son elemanı ile yer değiştir.



Adım 3: Onar (Heapify)

Yukarıya yeni taşıdığı zayıf düğüm kuralı bozacağı için kök düğümüne tekrar Heapify uygula.

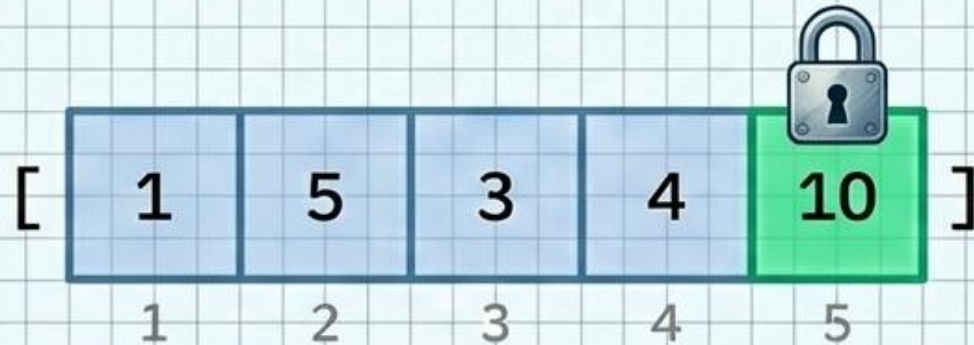
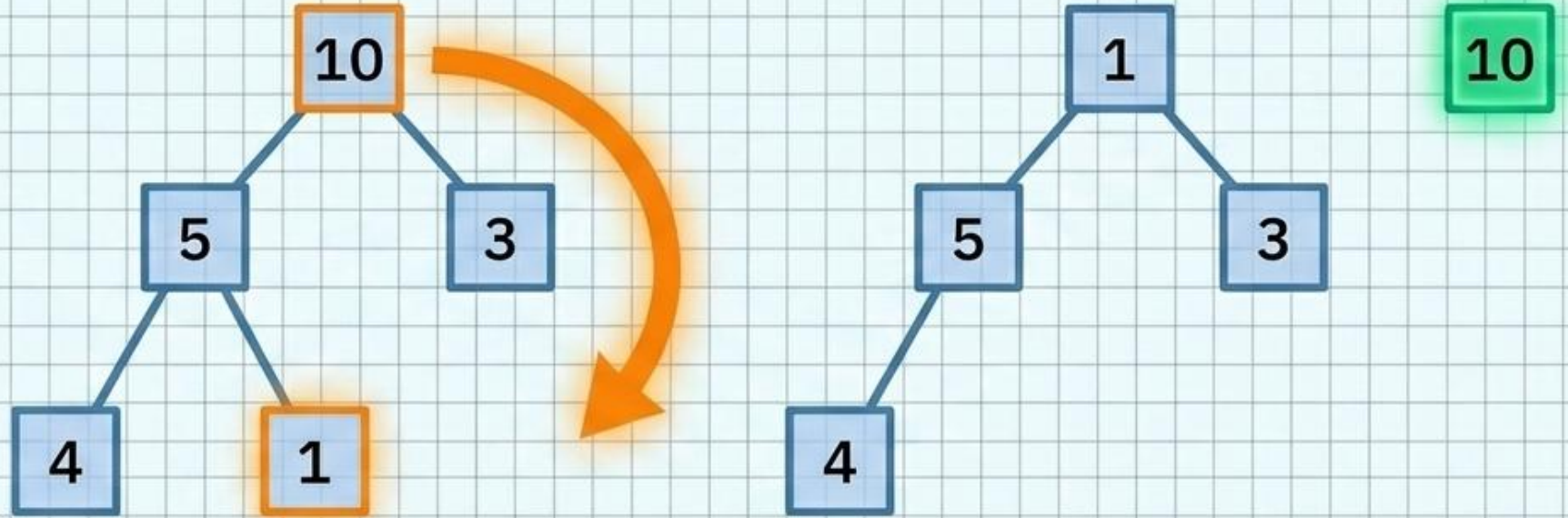
Adım 2: Kilitle (Lock)

0 elemanı dizinin sonuna sabitle (Yığın boyutunu 1 azalt). Artık o eleman tamamen sıralanmıştır.

Heapsort: İlk Çıkarma ve Kitleme

Kök düğüm (10) ile en sondaki eleman (1) yer değiştirir.

10, artık dizinin en sonundadır ve "Kilitlenmiştir". Ağaç yapısından tamamen koparılır.

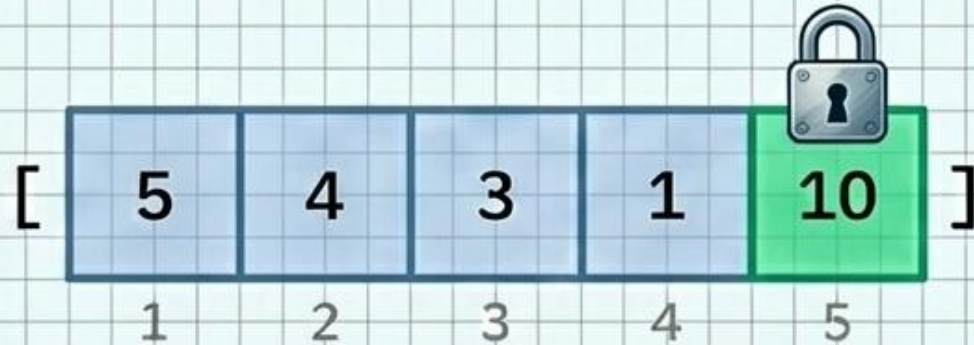
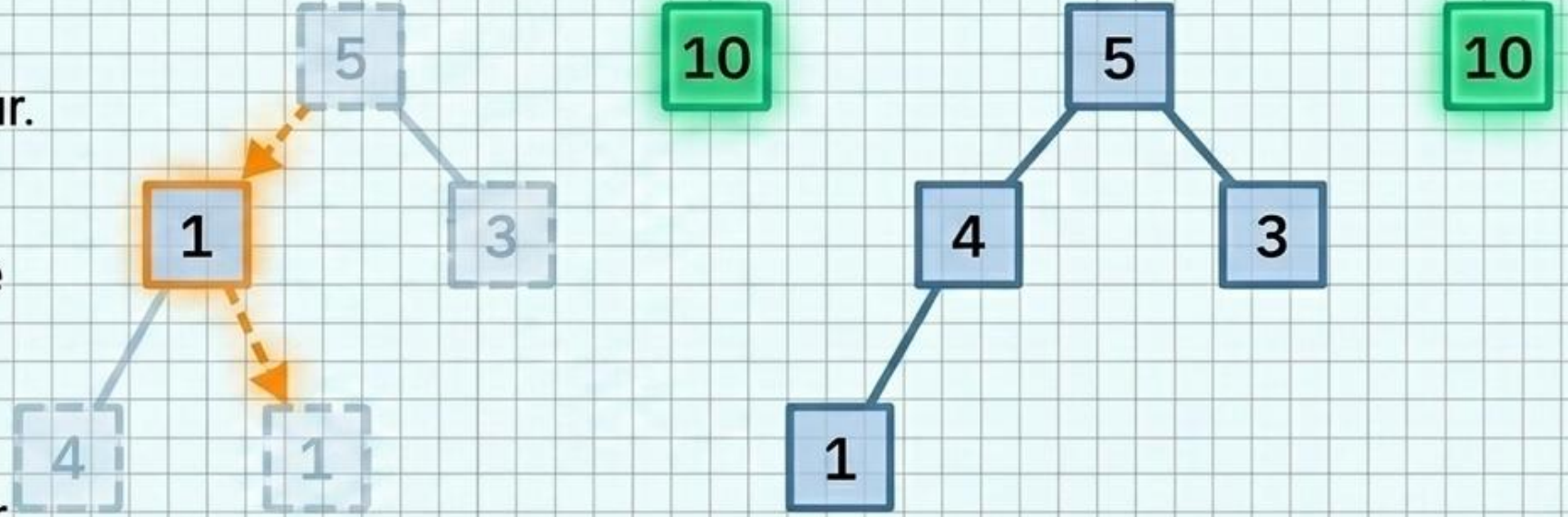


Heapsort: Dengenin Yeniden Sağlanması (Heapify)

Tepeye oturan '1' kuralı bozuyor. Heapify(1) çağrılır.

1, çocukları (5 ve 3) arasından büyük olan 5 ile yer değiştirir.

Aşağı inen 1, bu kez 4 ile yer değiştirir. Denge tekrar kuruldu.

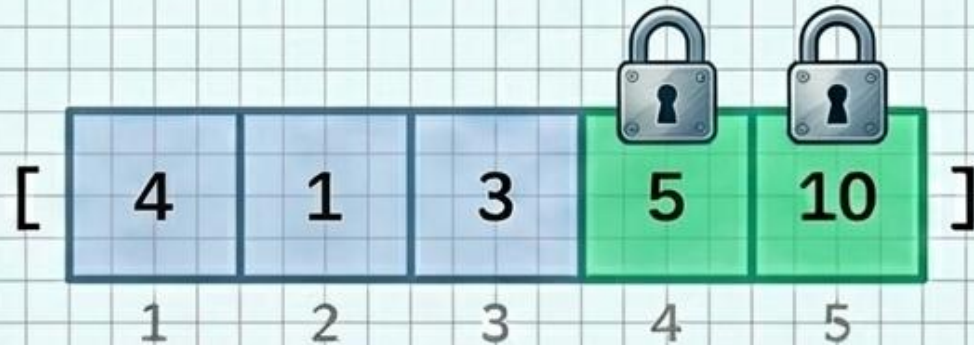
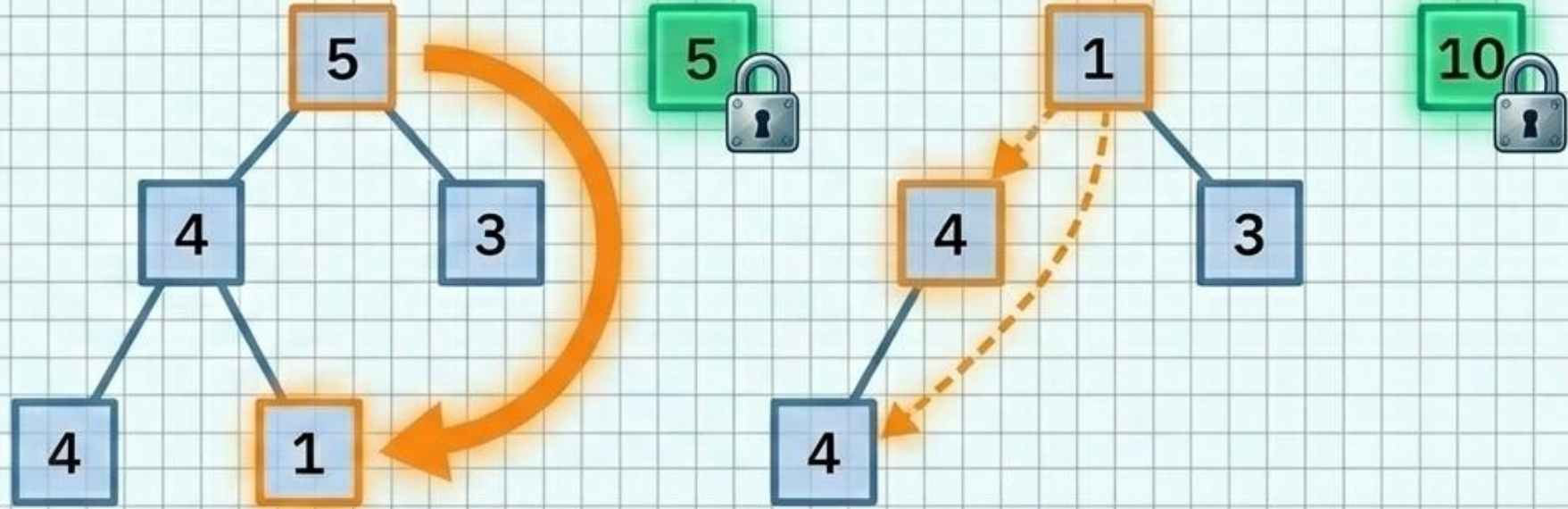


Heapsort: İkinci En Büyük Eleman (Döngü Tekrarı)

Yeni kök (5) ile aktif yığının en son elemanı en son elemanı (1) yer değiştirir.

5 kilitlenir.

Yukarı çıkan 1, yeni kök olarak Heapify işleminden geçer ve 4 ile yer değiştirir.

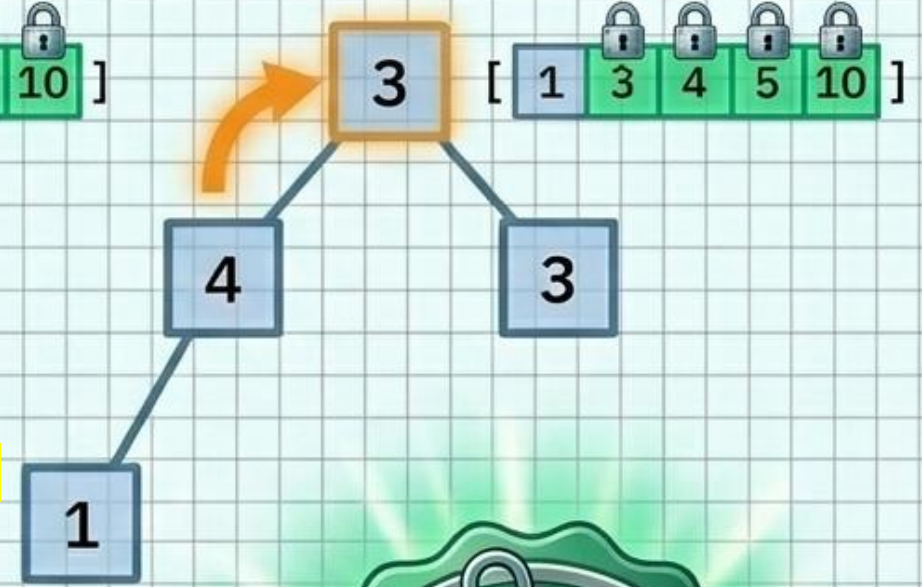
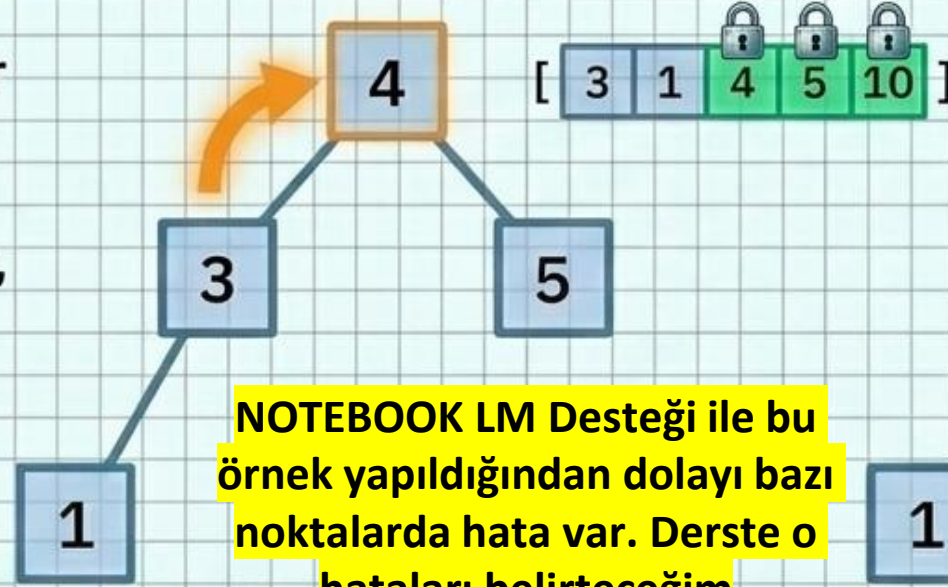


Final Fazı: Dizinin Tamamlanması

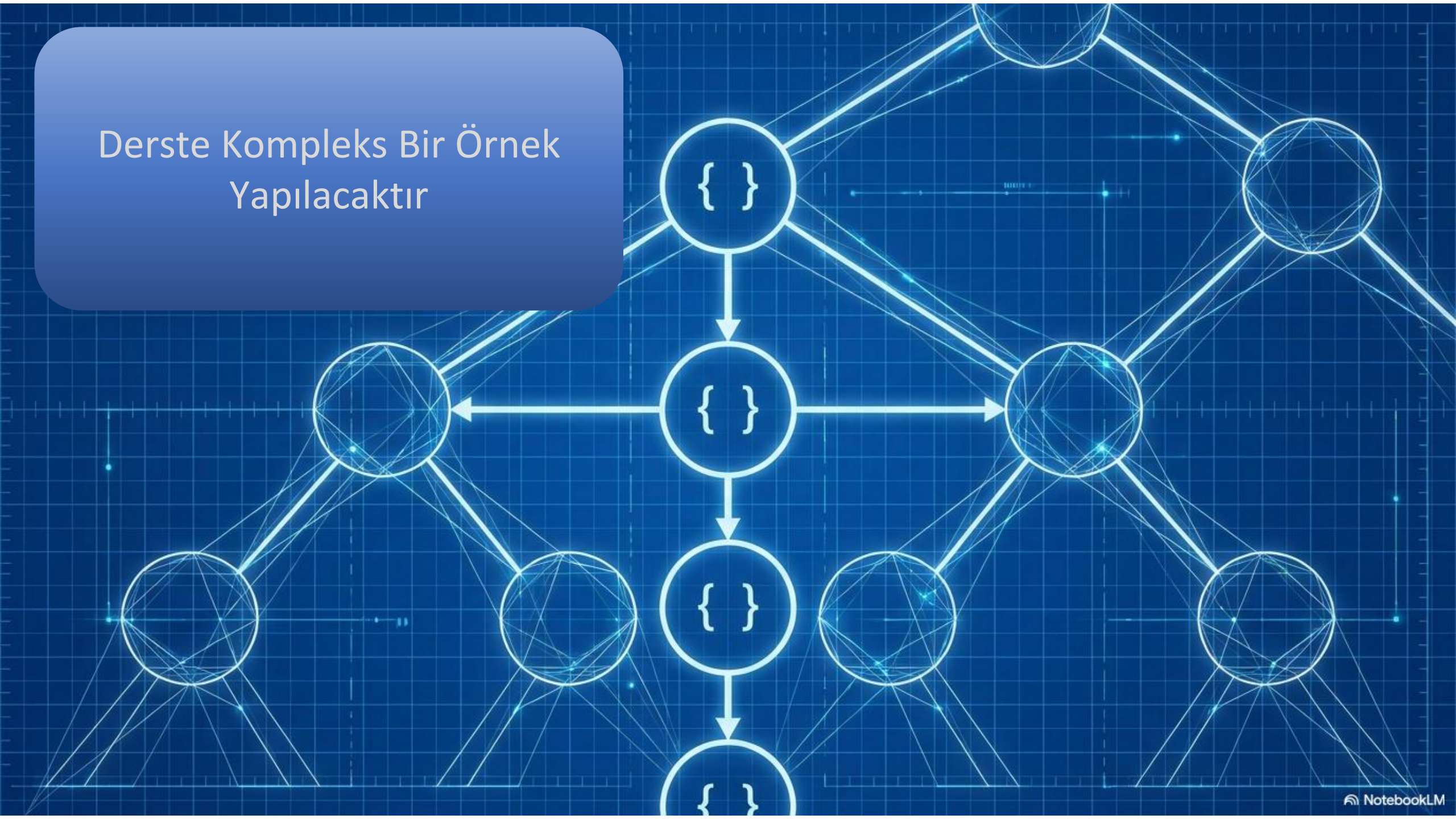
Aynı işlemler kalan sayılar (4, 3, 1) için uygulanır.

Kökler sırasıyla sona atılır, kalanlar yığınlştırılır.

Yığın boyutu 1'e düştüğünde işlem biter.



Derste Kompleks Bir Örnek
Yapılacaktır



Heap ağacı bilgisayar biliminde bazı problemlerin çözümü için çok uygundur. Hatta birebir uyuşmaktadır denilebilir. Üstelik hem bellek alanı hem de yürütme zamanı açısından getirisi olur. Çünkü ağacın tamamlanmış olması ve şekilde görüldüğü gibi boş olabilecek düğümlerin sağ taraflarda olması ağacın dizi üzerinde **indis bağıntısı** yaklaşımına göre tutulmasının avantajlarını öne çıkarır. **Bellek alanı kazancı ve düğümleri hızlı erişim sağlanır. Örnek iki ağacın veri yapıları aşağıdaki gibidir.**

	0	1	2	3	4	5	6	7		A	0	2	3	4	5	
A	77	55	66	33	22	60	50			A	z	r	t	c	f	u

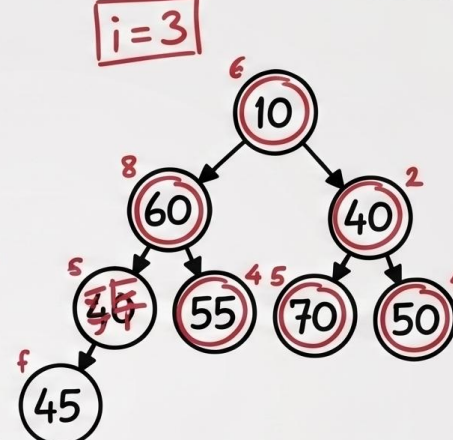
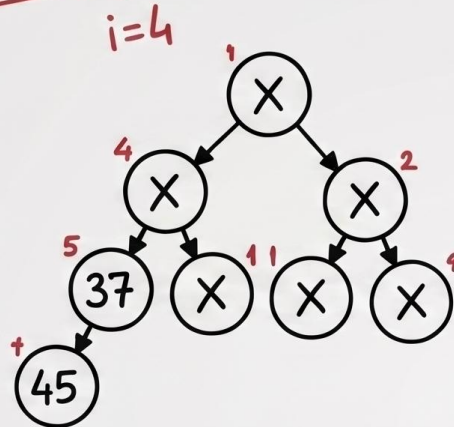
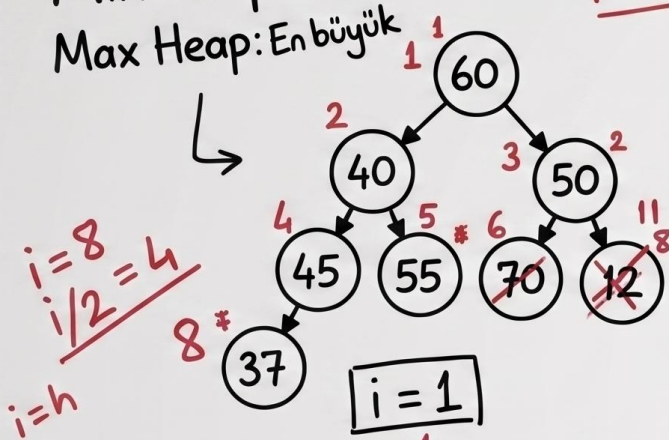
Heap - Sort

Min Heap: En küçük = Root

Max Heap: En büyük

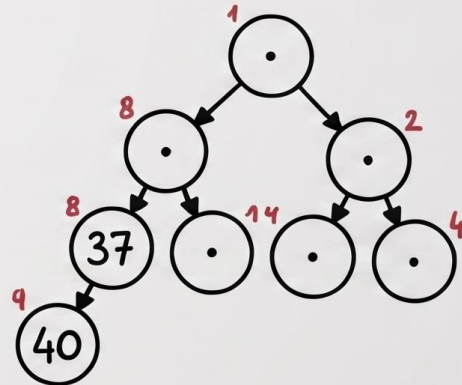
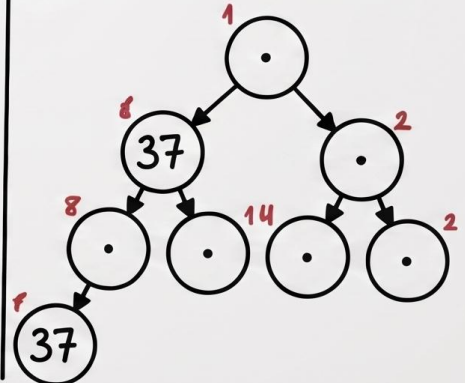
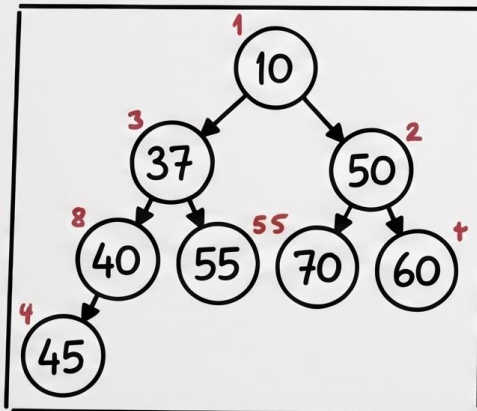
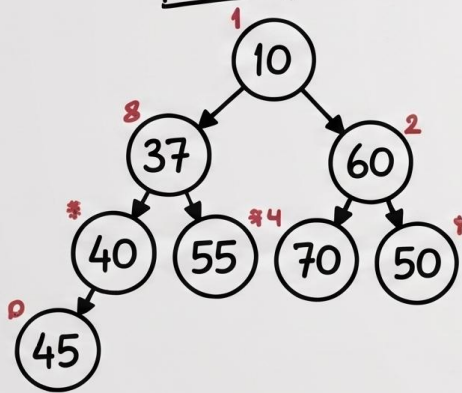
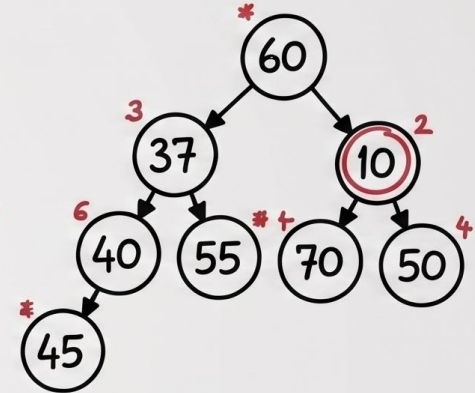
Min Heapiy

60	40	50	45	55	70	10	37
----	----	----	----	----	----	----	----

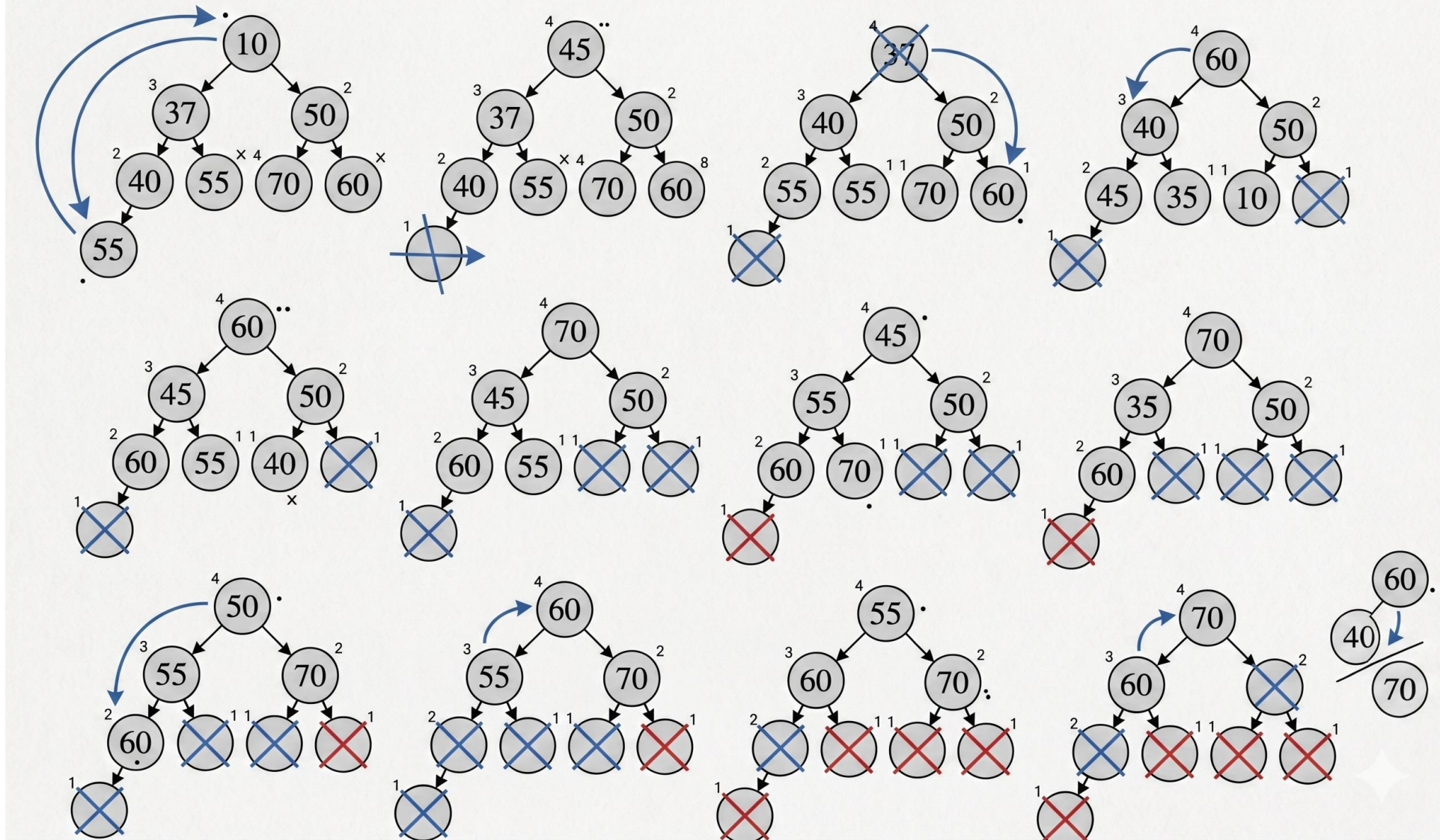


Algoritma Uzmanı

$i=2$



30	37	40	45	50	55	60	70
60	40	50	45	55	70	10	37



10	37	40.	15	50	55	60	70
----	----	-----	----	----	----	----	----

60	40	50	45	55	70	10 _{5.}	37 ₈
----	----	----	----	----	----	-----------------------------	-----------------

1) İkili Ağaç (Binary Tree)

İkili ağaç **mantıksal bir yapıdır**, fiziksel olarak iki şekilde tutulabilir:

1. Bağlı liste (pointer) yapısı

Her düğüm:

sol çocuk pointer

sağ çocuk pointer

En yaygın kullanım budur.

2. Dizi (indis) ile temsil

Özellikle **complete tree** ise:

kök $\rightarrow 1$ (veya 0)

sol çocuk $\rightarrow 2i$

sağ çocuk $\rightarrow 2i+1$

Bu temsil genelde **heap için kullanılır**

2) Heap (Min/Max Heap)

Gerçek durum:

Heap neredeyse her zaman:

DİZİ (array / indeks) yapısı ile implement edilir

Neden?

Complete binary tree olduğu için boşluk yok

Pointer yerine indeks çok daha hızlı ve hafif

Heap'te yapı

index: 0 1 2 3 4 5

50 30 40 10 20 35

İlişkiler:

$\text{parent}(i) = i // 2$

$\text{left}(i) = 2i$

$\text{right}(i) = 2i + 1$

Yani heap = **array + indeks mantığı**

3) AVL Tree

Gerçek durum:

AVL Tree her zaman:

Bağlı liste (pointer) yapısı ile tutulur

AVL'de dizi kullanılır mı?

Teorik olarak evet ama

Pratikte kullanılmaz

Neden?

AVL sürekli **rotate (döndürme)** yapar

Dizi yapısı bunu çok zorlaştırır

dynamic tree structure gerekir

Yapı	Liste (pointer)	Dizi (indeks)
Binary Tree	Evet	Bazen
AVL Tree	Evet (zorunlu)	Neredeyse hiç
Heap	Genelde yok	Evet (standart)

Sözlük Ağacı(Dictionary Tree)

Sözlük ağaçları (Trie yapıları), bir dildeki sözcüklerin karakter bazlı olarak organize edilmesini sağlayan ağaç veri yapılarıdır. Bu yapıların temel amacı, arama işlemlerini yüksek performansla gerçekleştirmek ve belleği mümkün olan en verimli biçimde kullanmaktır.

Sözlük ağaçlarının bir araya gelmesiyle oluşan yapı **sözlük ormanı (forest)** olarak adlandırılabilir. Bu ormanda, alfabenin karakter sayısı kadar kök düğüm bulunur ve her bir kök, ilgili karakter ile başlayan sözcükleri temsil eden alt ağaçların başlangıç noktasıdır.

Örneğin, “baba” sözcüğü “b” harfi ile başladığı için, “b” kök düğümüne sahip sözlük ağacında aranır ya da bu ağaca eklenir. Bu yapı sayesinde, ortak ön ek (prefix) paylaşan sözcükler aynı dalları kullanarak depolanır ve böylece hem arama süresi optimize edilir hem de bellek kullanımı azaltılır.

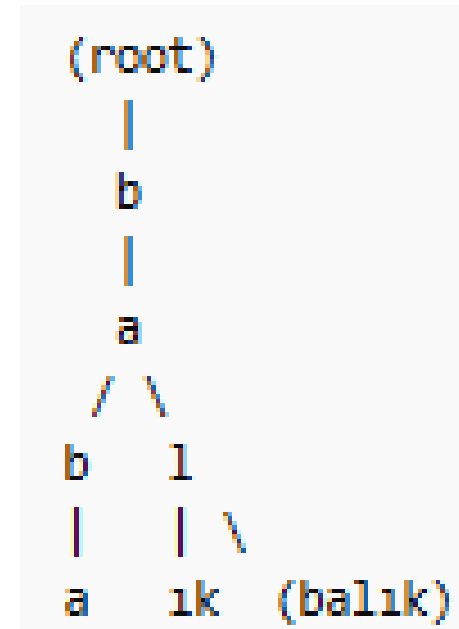
Tüm kelimeler **b** → **a** ile
başlıyor → ortak yol
kullanılır

Sonra dallanma olur:

b → **a** → **b** → **a** → *baba*

b → **a** → **l** → *bal*

b → **a** → **l** → **ı** → **k** → *balık*



Kodlama Ağaçları

Kodlama ağacı, bir alfabe'deki (ya da herhangi bir sembol kümesindeki) her bir karaktere **ikili (binary) kodlar atamak** için kullanılan ağaç veri yapısıdır.

Bu ağaçta:

Kökten başlanır

Yapraklara doğru ilerlenir

Her kenar bir değer taşır (genelde **0 ve 1**)

Yaprağa ulaşıncaya elde edilen 0-1 dizisi o karakterin kodudur

Kodlama ağacı **bir alfabe'deki veya daha genel olarak bir kümedeki karakterlere kod atanması** için kurulan ağaç şeklindedir. Bu tür ağaçlarda kökten başlanıp yapraklara kadar olan yol üzerindeki **bağlantı değerleri** kodu verir. Kodlama ağacı denildiğinde ilk akla gelen **Huffman** kodlaması olmasına karşın çok çeşitli kodlama ağaçları kurulabilir. (Huffman Kodlama Ağacı, Shannon Fano Kodlama Ağacı)

Neden kullanılır?

- ✓ Veri sıkıştırma (compression)
- ✓ Daha az bit ile veri saklama
- ✓ İletimde verimlilik

Veri Sıkıştırma Nedir?

Veri sıkıştırma, bir dosyanın ya da verinin **daha az yer kaplayacak** şekilde yeniden temsil edilmesidir.

Ama önemli nokta şu

Bilgi kaybolmadan ya da kontrollü olarak kaybolarak yapılır.

Yani amaç:

Aynı bilgiyi **daha az bit ile** saklamak veya iletmek

Neden Veri Sıkıştırma Yapılır?

Veri sıkıştırmanın temel amaçları:

Depolama

Daha az disk alanı kullanılır

Örn: .zip, .rar

İletim (Network)

Daha hızlı veri gönderimi

Daha az bant genişliği kullanımı

Zaman ve Maliyet

Daha kısa yükleme/indirme süresi

Daha az enerji ve maliyet

Veri Sıkıştırma Temel Mantığı

Veri sıkıştırma **tekrarlanan veya gereksiz bilgiyi** azaltır.

Basit örnek:

AAAAABBBCC

Bunu şu şekilde saklayabilirsin:

5A3B2C

Aynı bilgi, daha kısa temsil.

Veri Sıkıştırma Türleri

Örnek

1. Kayıpsız (Lossless) Sıkıştırma

Orijinal veri %100 geri elde edilir

Metin, kod, tıbbi veriler için kullanılır

Örnekler:

ZIP

PNG

Huffman Coding

LZW

Orijinal: ABCABCABC

Sıkıştırılmış: (ABC)x3

2. Kayıplı (Lossy) Sıkıştırma

Bir miktar bilgi kaybolur

Ama insan gözü/kulağı fark etmez

Örnekler:

JPEG (resim)

MP3 (ses)

MP4 (video)

Örnek

İnsan kulağının duyamadığı frekanslar silinir

Sıkıştırma Sistemi Nasıl Çalışır?

Genel olarak **3 adım vardır:**

Analiz

Veride tekrar eden desenler bulunur

Kodlama

Sık geçen veriler daha kısa kodlarla temsil edilir

Az geçen veriler daha uzun kodlarla

Depolama / İletim

Kodlanmış veri saklanır veya gönderilir

Veri Sıkıştırma Sisteminin Bileşenleri

Bir sıkıştırma sistemi genelde şunları içerir:

1.Kaynak Modeli

1.Hangi semboller sık geçiyor?

2.Kodlayıcı (Encoder)

1.Veriyi sıkıştırır

3.Kod Çözücü (Decoder)

1.Veriyi eski haline getirir

Orijinal Veri → Encoder → Sıkıştırılmış Veri
Sıkıştırılmış Veri → Decoder → Orijinal Veri

Huffman Kodlama Nedir?

Huffman kodlama, sık kullanılan sembollere **kısa**, az kullanılanlara **uzun** bit kodları vererek yapılan **kayıpsız veri sıkıştırma** yöntemidir.

ÖRNEK ÜZERİNDEN GİDELİM

ABBCCCDDDDDD

ABBCCDDDDD

1 Adım: Frekans Tablosu
Oluşturma

Karakter	Frekans
A	1
B	2
C	3
D	5

En sık geçen: D
En az geçen: A

2 Adım: Huffman Ağacı
Oluşturma

En küçük frekanslıları
birleştiririz:

$$A(1) + B(2) \rightarrow 3$$

$$C(3) + (AB)(3) \rightarrow 6$$

$$D(5) + (CAB)(6) \rightarrow 11 \text{ (kök)}$$

Yanlış birleştirmek ne olur?

O zaman:

D ve C **derine inerdi**

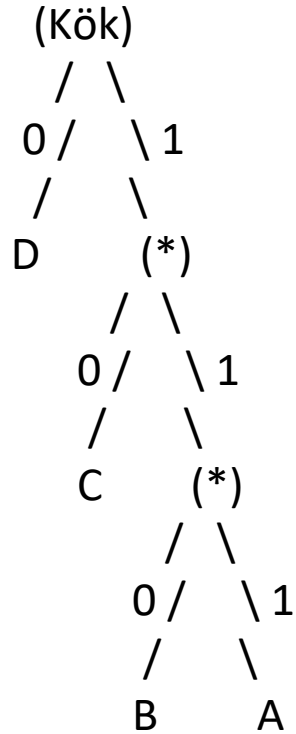
A ve B yukarı çıkardı

Toplam bit maliyeti artardı

Huffman olmazdı

3 Adım: Huffman Kodlarını Atama

En sık geçen → en kısa kod alır
En az geçen → en uzun kod alır



SOL VE SAĞ DAL NE DEMEK?

Sola gidiyorsan → 0

Sağa gidiyorsan → 1

D

Kök → sol **Kod: 0**

C

Kök → sağ (1) → sol (0) Kod: **10**

B

Kök → sağ (1) → sağ (1) → sol (0) Kod: **110**

A

Kök → sağ (1) → sağ (1) → sağ (1) Kod: **111**

En sık geçen: D

En az geçen: A

Daha sık geçen → daha kısa YA DA en fazla eşit uzunlukta kod alır

Daha az geçen → daha uzun YA DA en fazla eşit uzunlukta kod alır

SIKISTIRMA (ENCODING)

Orijinal

ABBCCDDDDD

Bit dizisi

A	B	B	C	C	C	D	D	D	D	D
111	110	110	10	10	10	00	00	00	00	00

Kodlama

A → 111

B → 110

C → 10

D → 0

Sıkıştırılmış veri:

11111011010101000000

Sayısal olarak - Sıkıştırma Kazancı

Frekanslar

A = 1
B = 2
C = 3
D = 5

Kod uzunlukları

A = 3
B = 3
C = 2
D = 1

Toplam maliyet

A $\rightarrow 1 \times 3 = 3$
B $\rightarrow 2 \times 3 = 6$
C $\rightarrow 3 \times 2 = 6$
D $\rightarrow 5 \times 1 = 5$
TOPLAM = 20

Normal Şartlar Altında,

Karakter	Frekans
----------	---------

A	1
B	2
C	3
D	5

Sıkıştırma Kazancı

Önce:

11 karakter $\times$ 8 bit = **88 bit**

Sonra:

20 bit

Yaklaşık **%75 sıkıştırma oranı**

ÇÖZME (DECODING)

Şimdi bu bit dizisini geri açalım:

11111011010101000000

Çözülen metin

ABBCCDDDDD

Original veri %100 geri elde edildi → KAYIPSIZ

Huffman ağacına göre oku:

111 → A

110 → B

110 → B

10 → C

10 → C

10 → C

0 → D

0 → D

0 → D

0 → D

0 → D

HUFFMAN ÖRNEĞİ 2

FREKANS TABLOSU

Karakter	Frekans
A	9
B	3
C	1
D	1
E	1
F	1
G	1
H	1

Orijinal Metin : BACADAEAFABBAAGAH

HUFFMAN AĞACINI OLUŞTURALIM

En küçükleri birleştirerek gideriz:

$C(1) + D(1) \rightarrow 2$

$E(1) + F(1) \rightarrow 2$

$G(1) + H(1) \rightarrow 2$

$(CD)(2) + (EF)(2) \rightarrow 4$

$(GH)(2) + B(3) \rightarrow 5$

$(CDEF)(4) + (GHB)(5) \rightarrow 9$

$A(9) + (\text{diğerleri})(9) \rightarrow 18 \leftarrow \text{kök}$



BU AĞAÇTA NEYİ GÖRÜYORSUN?

En sık geçen

A(9) → köke en yakın → **1 bit**

En az geçenler

C, D, E, F, G, H (1) → en derin → **4 bit**

Orta frekans

B(3) → orta derinlik → **3 bit**

Yani **frekans–derinlik ilişkisi kusursuz.**

YZ desteği ile soldaki görüntü oluşturulduğu için son adımda ufak kayma durumları oluşmuştur.

ENCODING (HUFFMAN KODLAMA / SIKIŞTIRMA)

Orijinal Metin : BACADAEAFABBAAAGAH

Huffman Kod Tablosu (Önceki üretilen ağaçtan okunmuştur)

Karakter	Frekans	Huffman Kodu	Bit
A	9	0	1
B	3	111	3
C	1	1000	4
D	1	1001	4
E	1	1010	4
F	1	1011	4
G	1	1100	4
H	1	1101	4

Orijinal Metin
BACADAEAFABBAAAGAH

Karakter
→
Bit dönüşümü

Sıkıştırılmış bit dizisi (ENCODED DATA)
111010000100101010101101111111000110001101

B → 111
A → 0
C → 1000
A → 0
D → 1001
A → 0
E → 1010
A → 0
F → 1011
A → 0
B → 111
B → 111
A → 0
A → 0
A → 0
G → 1100
A → 0
H → 1101

SIKIŞTIRMA KAZANCI

Orijinal veri

18 karakter

1 karakter = 8 bit

$18 \times 8 = 144$ bit

Karakter

A

B

C–H

Huffman sonrası

Hesap

$9 \times 1 = 9$

$3 \times 3 = 9$

$6 \times 4 = 24$

Toplam = 42 bit

Sıkıştırma Oranı

$$\text{Kazanç} = \frac{144 - 42}{144} \times 100$$

≈ %70.8 sıkıştırma kazancı

Çok yüksek ve **optimal** bir sonuç

DECODING (HUFFMAN ÇÖZME)

Girdi (sıkıştırılmış veri)

111010000100101010
101101111111000110
001101

Huffman ağacında bitleri oku ->>>

Çözülmüş metin (DECODED OUTPUT)

BACADAEAFABBAAGAH

Orijinal veri birebir geri elde edildi → KAYIPSIZ

Bitler	Karakter
111	B
0	A
1000	C
0	A
1001	D
0	A
1010	E
0	A
1011	F
0	A
111	B
111	B
0	A
0	A
0	A
1100	G
0	A
1101	H

Shannon–Fano Algoritması Nedir?

Shannon–Fano, sembollerin frekanslarına göre **üstten-aşağı (top-down)** çalışan, **kayıpsız** bir veri sıkıştırma algoritmasıdır.

Amaç:

Sık geçen sembolere kısa, az geçenlere uzun kodlar vermek

- Huffman'a benzer
- Ama **her zaman optimal değildir**

Özellik	Huffman	Shannon–Fano
Yaklaşım	Bottom-up	Top-down
Optimal mi?	Evet	Her zaman değil
Kullanım	Daha yaygın	Daha basit
Ağaç	En küçükler birleşir	Liste bölünür

SHANNON–FANO ALGORİTMASININ ADIMLARI

Sembolleri frekansa göre sırala (azalan)

Toplam frekansı **ikiye en yakın olacak şekilde böl**

Sol gruba 0, sağ gruba 1 ata

Her grup için **aynı işlemi tekrar et**

Tek sembol kalınca dur

ÖRNEK

Orijinal semboller ve frekanslar

Karakter	Frekans
A	9
B	3
C	3
D	2
E	2
F	1

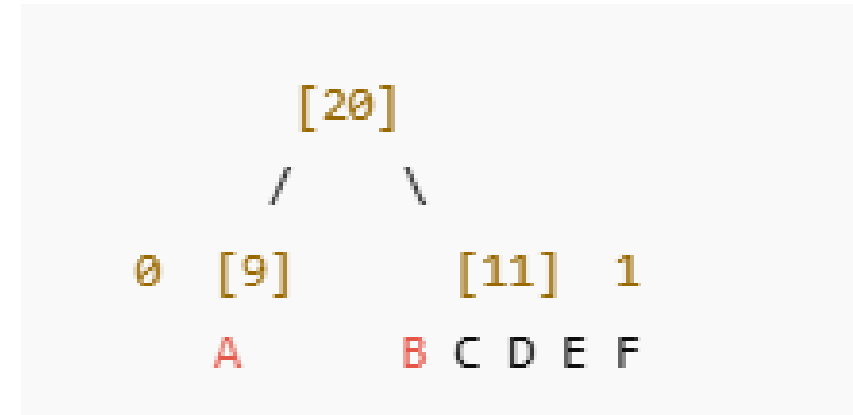
SIRALAMA (AZALAN)

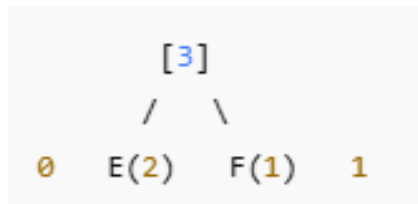
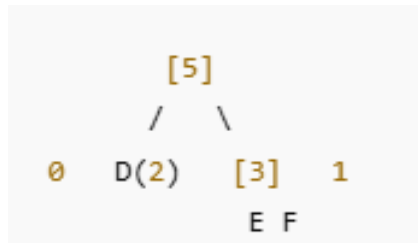
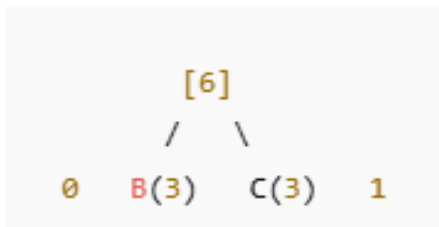
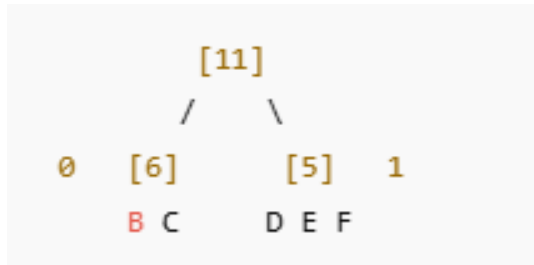
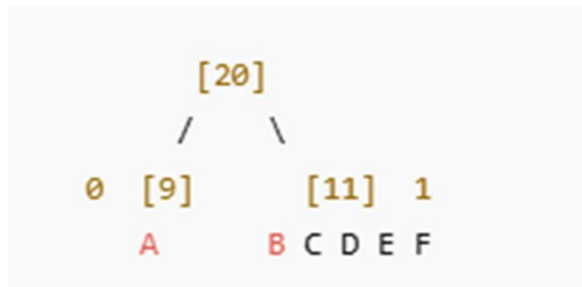
A(9), B(3), C(3), D(2), E(2), F(1)

İLK BÖLME (TOPLAM ≈ 10)

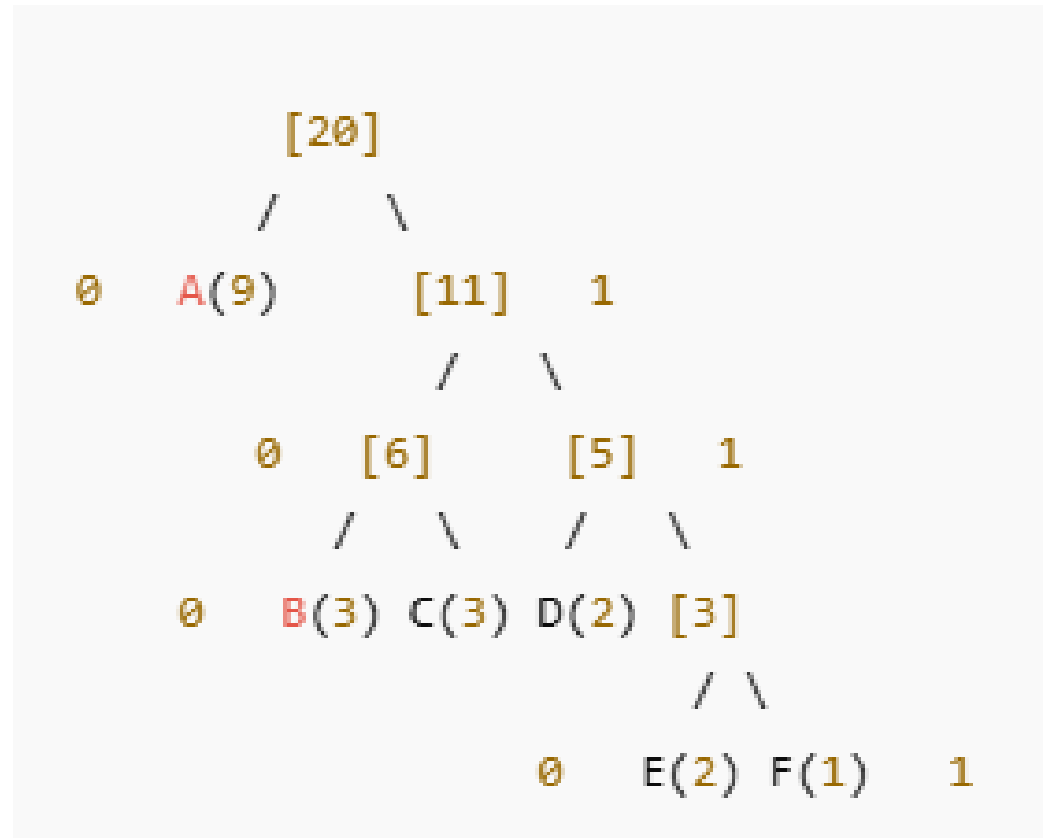
Sol grup: A(9) $\rightarrow 0$

Sağ grup: B(3) C(3) D(2) E(2) F(1) $\rightarrow 1$





TAM AĞAÇ (TEK PARÇA)



KOD TABLOSU (AĞAÇTAN OKUNUŞ)

Karakter	Yol	Kod
A	sol	0
B	sağ → sol → sol	100
C	sağ → sol → sağ	101
D	sağ → sağ → sol	110
E	sağ → sağ → sağ → sol	1110
F	sağ → sağ → sağ → sağ	1111

Huffman'da:
“Ağaç → kod”
Shannon–Fano'da:
“Bölme → kod → ağaç”

KAZANÇ

Kazanç, sıkıştırmadan önceki bit sayısı ile sıkıştırmadan sonraki bit sayısı arasındaki farktır.

SIKIŞTIRMADAN ÖNCE (Sabit uzunluklu kod)

Karakter sayısı = 6

Gerekli bit sayısı: $\lceil \log_2(6) \rceil = 3$ bit (Yaklaşık)

Her karakter 3 bit ile temsil edilir

Toplam bit

Toplam karakter = 20

Önceki bit sayısı = $20 \times 3 = 60$ bit

6 farklı şeyi en az kaç bit ile ayırt edebilirim?"

Hesap:

$2^1 = 2$ sembol

$2^2 = 4$ sembol -> yetmez

$2^3 = 8$ sembol -> yeter

O yüzden:

$\lceil \log_2(6) \rceil = 3$

Bu formül:

eşit uzunlukta (fixed-length) kodlama için

SIKIŞTIRMADAN SONRA (Shannon–Fano)

Karakter	Frekans	Kod	Uzunluk
A	9	0	1
B	3	100	3
C	3	101	3
D	2	110	3
E	2	1110	4
F	1	1111	4

$$A \rightarrow 9 \times 1 = 9$$

$$B \rightarrow 3 \times 3 = 9$$

$$C \rightarrow 3 \times 3 = 9$$

$$D \rightarrow 2 \times 3 = 6$$

$$E \rightarrow 2 \times 4 = 8$$

$$F \rightarrow 1 \times 4 = 4$$

$$\text{TOPLAM} = 45 \text{ bit}$$

SIKIŞTIRMA KAZANCI (BIT OLARAK)

Kazanç = Önceki – Sonraki Kazanç = 60 – 45 = 15 bit

Kazanç (%) = $(15 / 60) \times 100 = \%25$

Oran = Önceki / Sonraki = 60 / 45 = 1.33 -- Veri **1.33 kat** küçüldü

SIKIŞTIRMA SONRASI NE ELDE EDERİZ?

Sadece “bit sayısı azaldı” DEMEYİZ.

Somut olarak iki şey çıkar:

A) Bit akışı (encoded data)

B) Kod tablosu (veya ağaç)

İkisi birlikte sıkıştırılmış veridir.

AAAAAAA BBB CCC DD EE F -> Toplam 20 karakter.

Shannon–Fano / Huffman KOD TABLOSU

Karakter	Kod	ENCODED BIT STREAM (GERÇEK ÇIKTI)
----------	-----	-----------------------------------

A	0	A A A A A A B B B C C C D D E E F
---	---	-----------------------------------

B	100	⬇
---	-----	---

C	101	0 0 0 0 0 0 0
---	-----	---------------

D	110	100 100 100
---	-----	-------------

E	1110	101 101 101
---	------	-------------

F	1111	110 110
---	------	---------

		1110 1110
--	--	-----------

		1111
--	--	------

Tek parça bit dizisi:

000000010010010010110110111011101111

İşte sıkıştırılmış veri BU.

DECODER (ÇÖZME) NASIL ÇALIŞIR?

Decoder'ın elinde:

Ağaç (veya kod tablosu)
Bit akışı

DECODING ADIMLARI

Bitleri soldan sağa oku:

0 → A

kökten başa dön

100 → B

kökten başa dön

101 → C

Tam çözme örneği:

0000000 100100100 101101101 110110 11101110 1111



AAAAAAA BBB CCC **DD** EE F

Orijinal veri %100 geri geldi

UYARI

$\log_2(n)$ → “kaç sembol var?” problemi

Huffman → “en az bit ile nasıl kodlarım?” problemi

Veri sıkıştırma sistemi, veriyi daha az yer kaplayacak şekilde temsil etmeyi amaçlayan ve birbiriyle uyumlu çalışan temel bileşenlerden oluşur. Bu sistemde **kaynak modeli**, veride hangi sembollerin ya da desenlerin daha sık tekrarlandığını analiz ederek istatistiksel bir yapı çıkarır; böylece hangi bilginin daha kısa kodlarla temsil edilmesi gerektiğini belirler. **Kodlayıcı (encoder)**, kaynak modelinden elde edilen bu bilgiyi kullanarak veriyi daha verimli bir biçimde kodlar ve boyutunu küçültür. İletim ya da depolama sürecinden sonra **kod çözücü (decoder)** ise aynı model ve kuralları kullanarak sıkıştırılmış veriyi tekrar orijinal hâline (veya kayıplı sistemlerde yaklaşık hâline) dönüştürür. Bu üç temel bileşen birlikte çalışarak hem depolama alanından tasarruf sağlar hem de veri iletimini daha hızlı ve maliyet etkin hâle getirir.

KAYNAKÇA

Veri Yapıları ve Algoritmalar, 12. Baskı, Rifat Çölkesen

Nejat Yumuşak, fatih Adak Veri C++ ile veri yapıları, 3.baskı

C++ ile programlama, Dokuzuncu baskıdan çeviri, Palme Yayınları, Pau Deitel, Harvey Deitel, Çeviri Editörü Cemil Öz

Veri Yapıları, Hakan Kutucu

Algoritma ve Programlamaya giriş, Fahri Vatansever

Programlama ve Veri Yapılarına Giriş C,C++ ve Java ile, Sadi Evren Şeker

<https://chatgpt.com/>

<https://gemini.google.com/>

<https://notebooklm.google.com/>